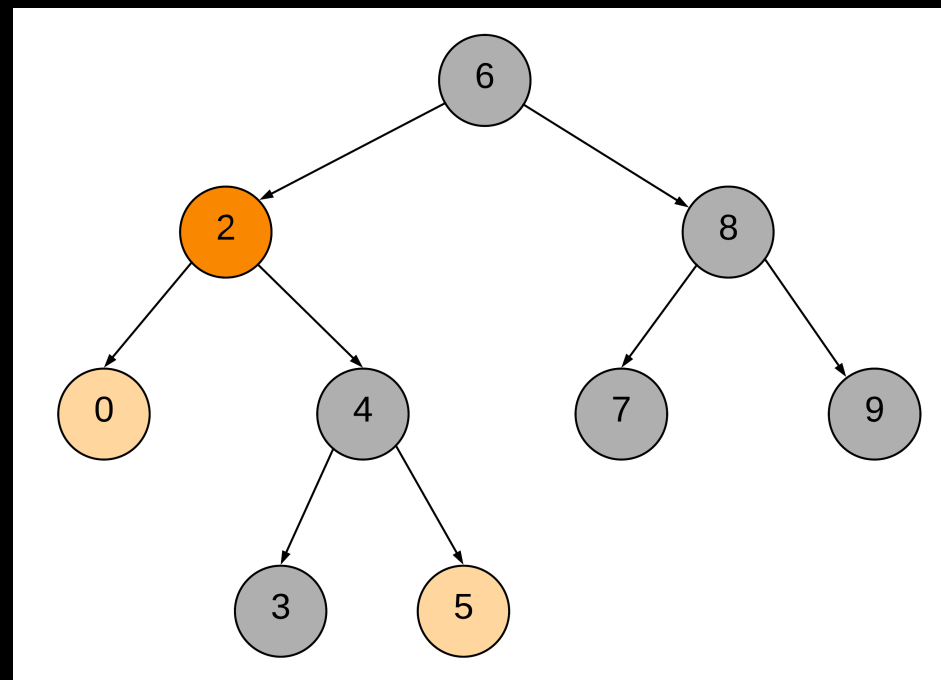


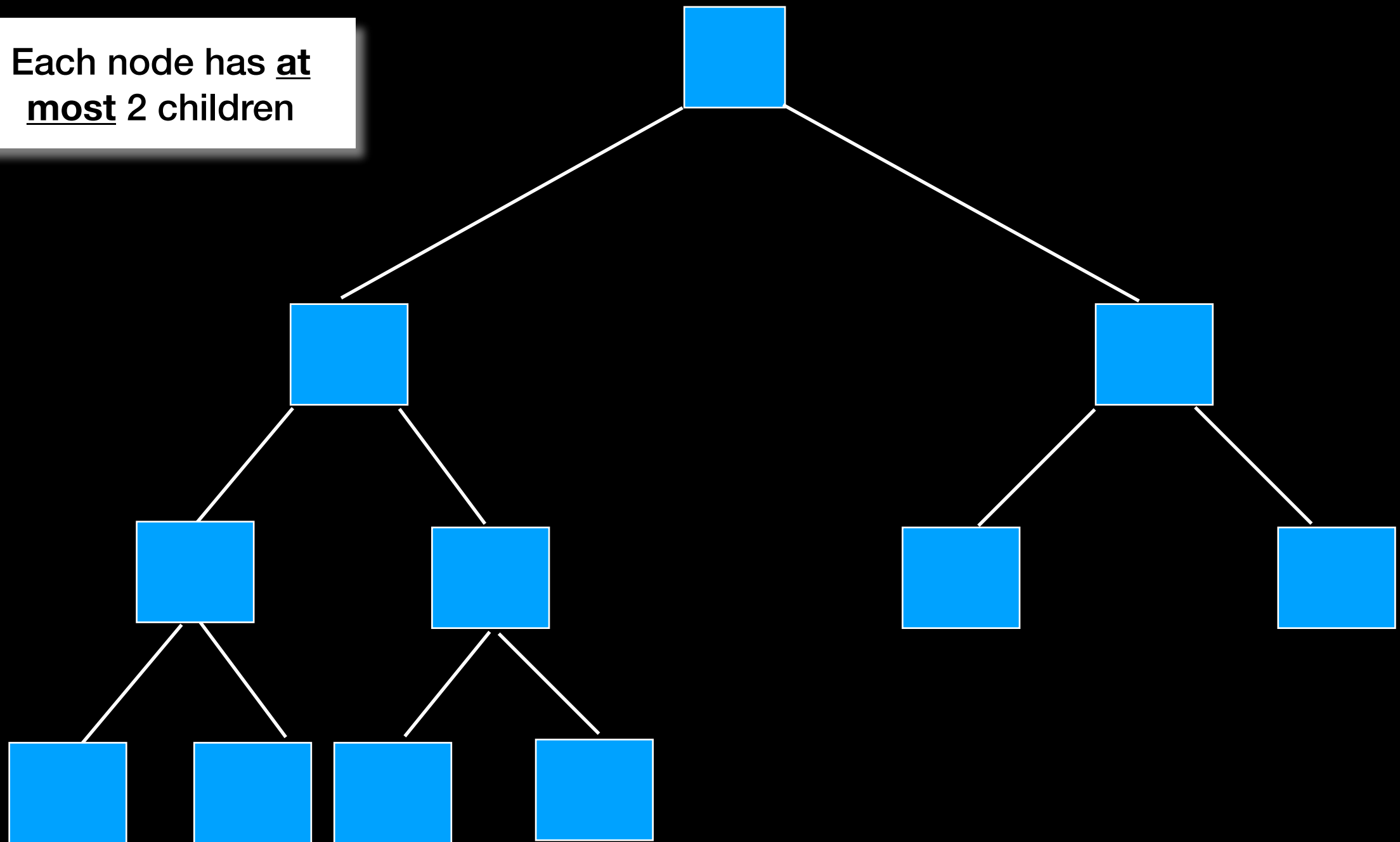
Binary Search Tree (BST)



Tiziana Ligorio
Hunter College of The City University of New York

Recap: Binary Tree

Each node has at most 2 children



Recap: Structure

Full:

- Non-leaves have exactly 2 children
- Each node has left and right subtree of same h
- All leaves at level h

Complete:

- Full up to level $h-1$
- Level h filled from left to right
- All nodes at $h-2$ and above have exactly 2 children

Balanced:

- For each node, left and right subtree height differ by at most 1

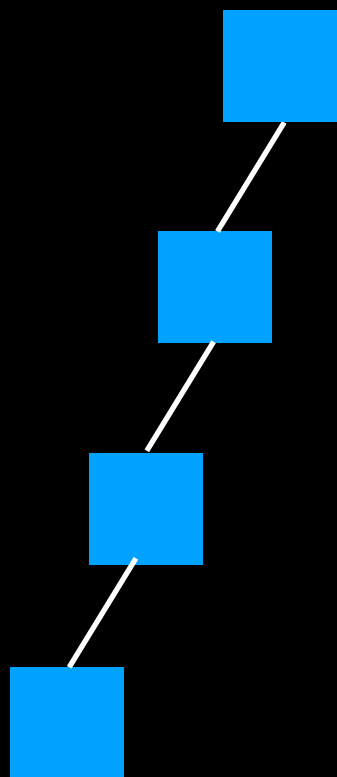
Recap: Max/Min Height

n nodes

every node 1 child

$h = n$

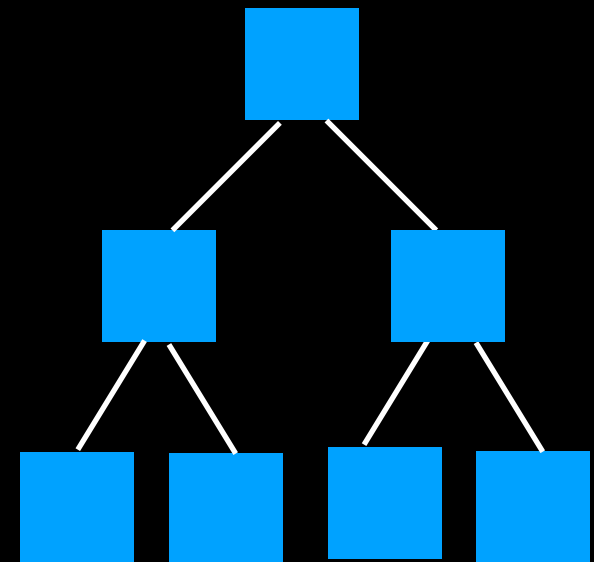
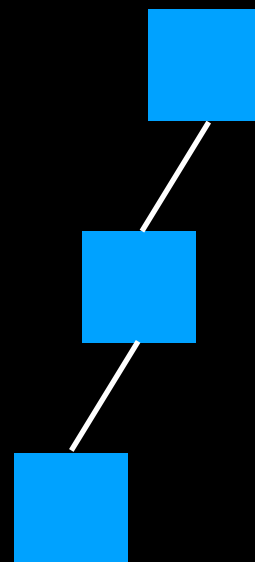
Essentially a chain



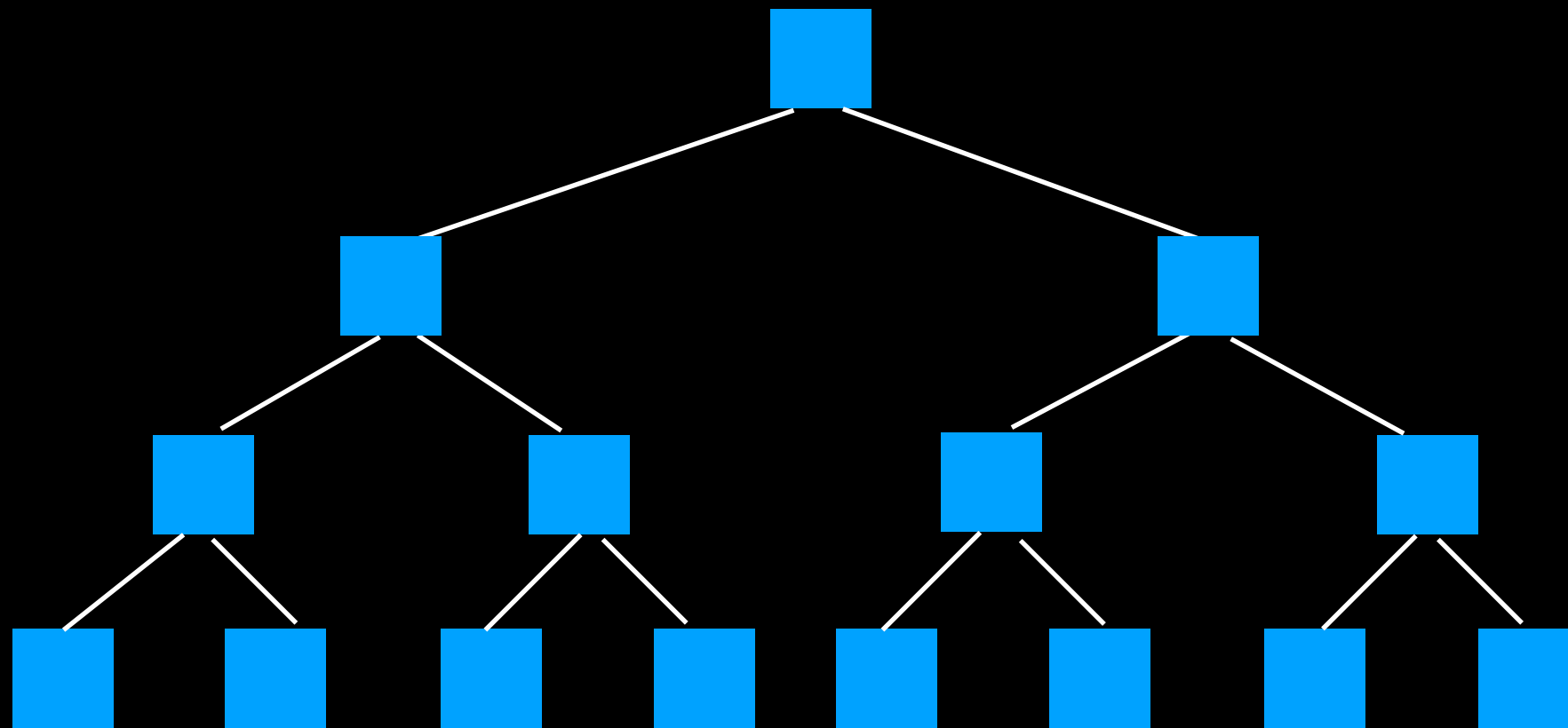
Binary tree of height h can have up to $n = 2^h - 1$

For example for $h = 3$, $1 + 2 + 4 = 7 = 2^3 - 1$

$h = \log(n+1)$ for a **full binary tree**



Recap



...

In a full tree:

h	n @ level	Total n
1	$1 = 2^0$	$1 = 2^1 - 1$
2	$2 = 2^1$	$3 = 2^2 - 1$
3	$4 = 2^2$	$7 = 2^3 - 1$
4	$8 = 2^3$	$15 = 2^4 - 1$
h	2^{h-1}	$2^h - 1$

Recap

```
#ifndef BinaryTree_H_
#define BinaryTree_H_

template<class T>
class BinaryTree
{
public:
    BinaryTree(); // constructor
    BinaryTree(const BinaryTree<T>& tree); // copy constructor
    ~BinaryTree(); // destructor
    bool isEmpty() const;
    size_t getHeight() const;
    size_t getNumberOfNodes() const;
    void add(const T& new_item);
    void remove(const T& new_item);
    T find(const T& item) const;
    void clear();

    void preorderTraverse(Visitor<T>& visit) const;
    void inorderTraverse(Visitor<T>& visit) const;
    void postorderTraverse(Visitor<T>& visit) const;

    BinaryTree& operator= (const BinaryTree<T>& rhs);

private: // implementation details here
}; // end BST

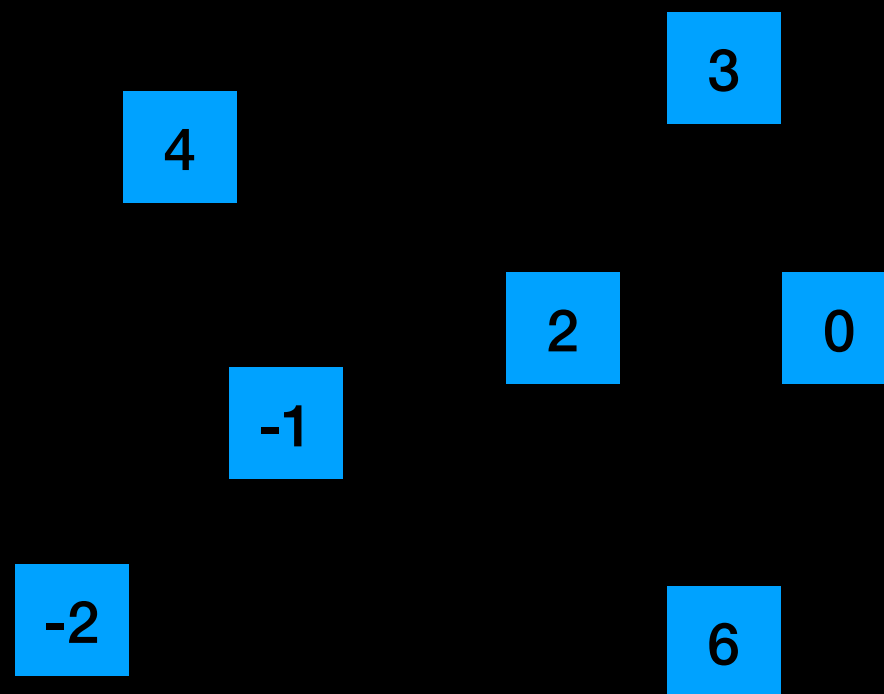
#include "BinaryTree.cpp"
#endif // BinaryTree_H_
```

How might you add
Will determine the tree structure

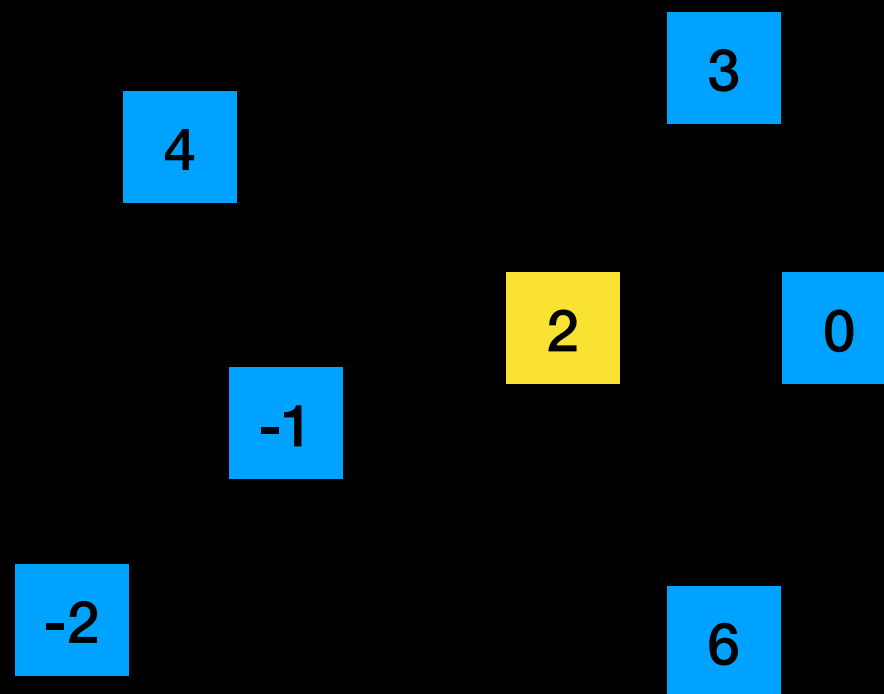
This is an abstract class from which
we can derive desired behavior
keeping the traversal general

Considerations

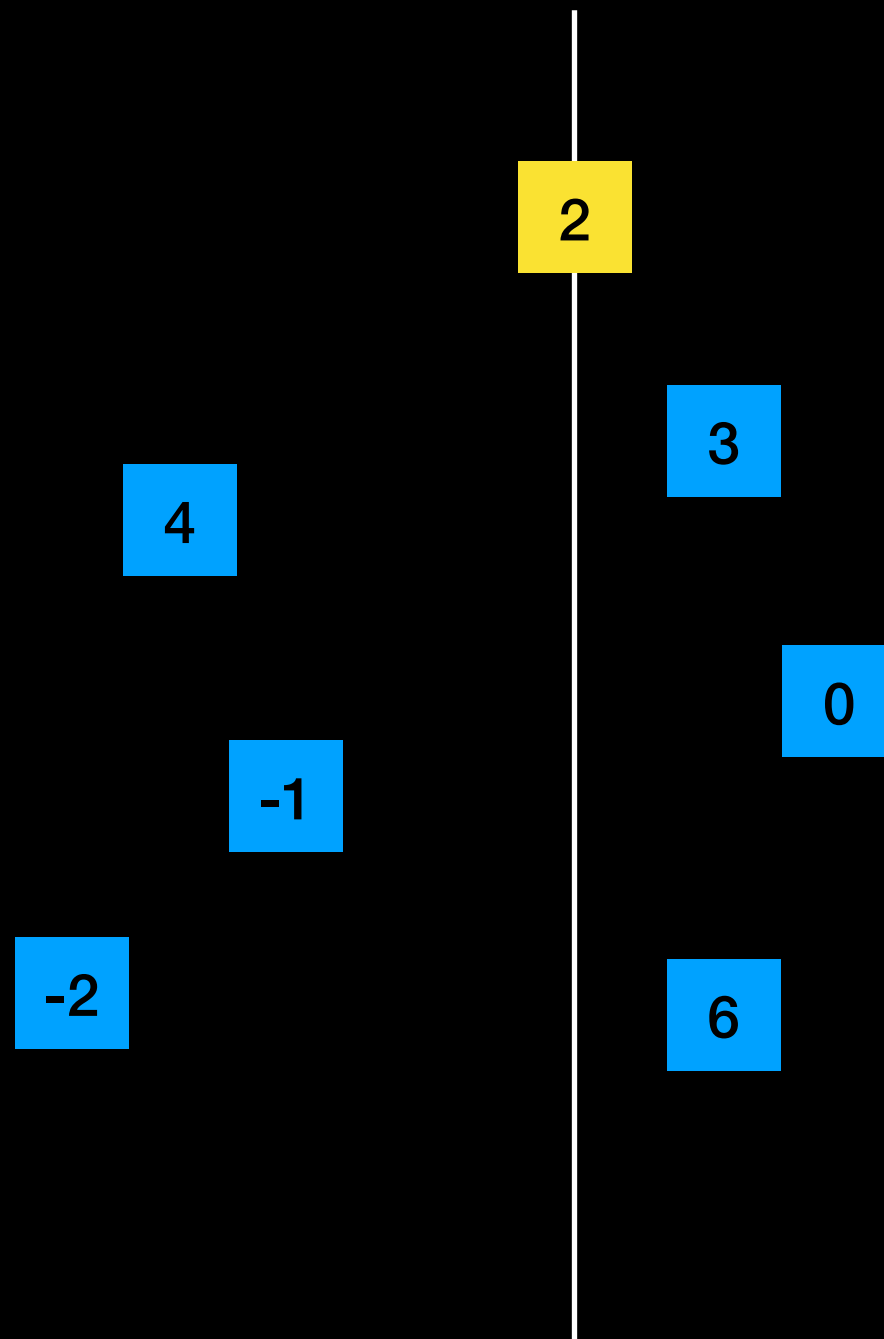
A Different Approach



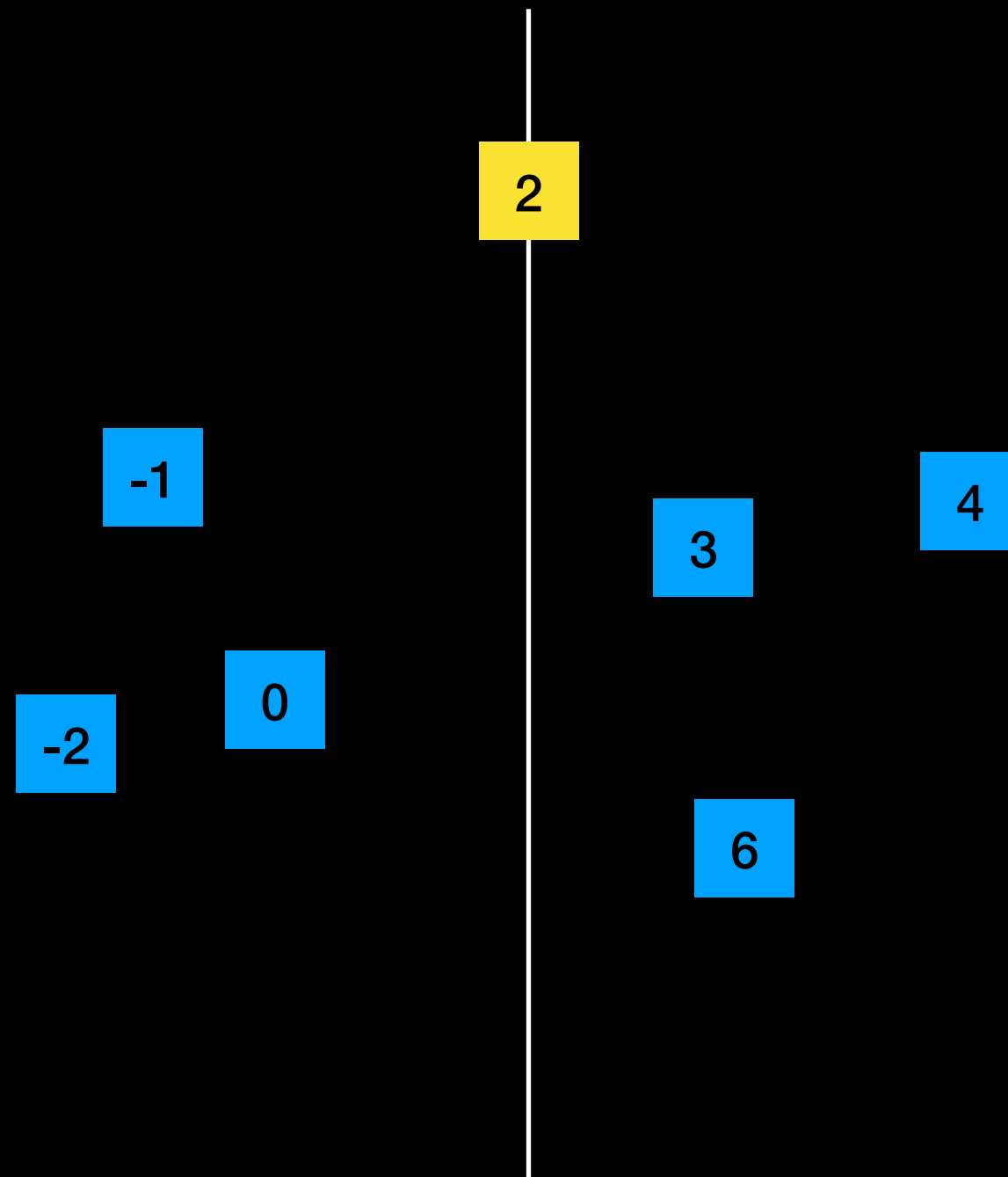
A Different Approach



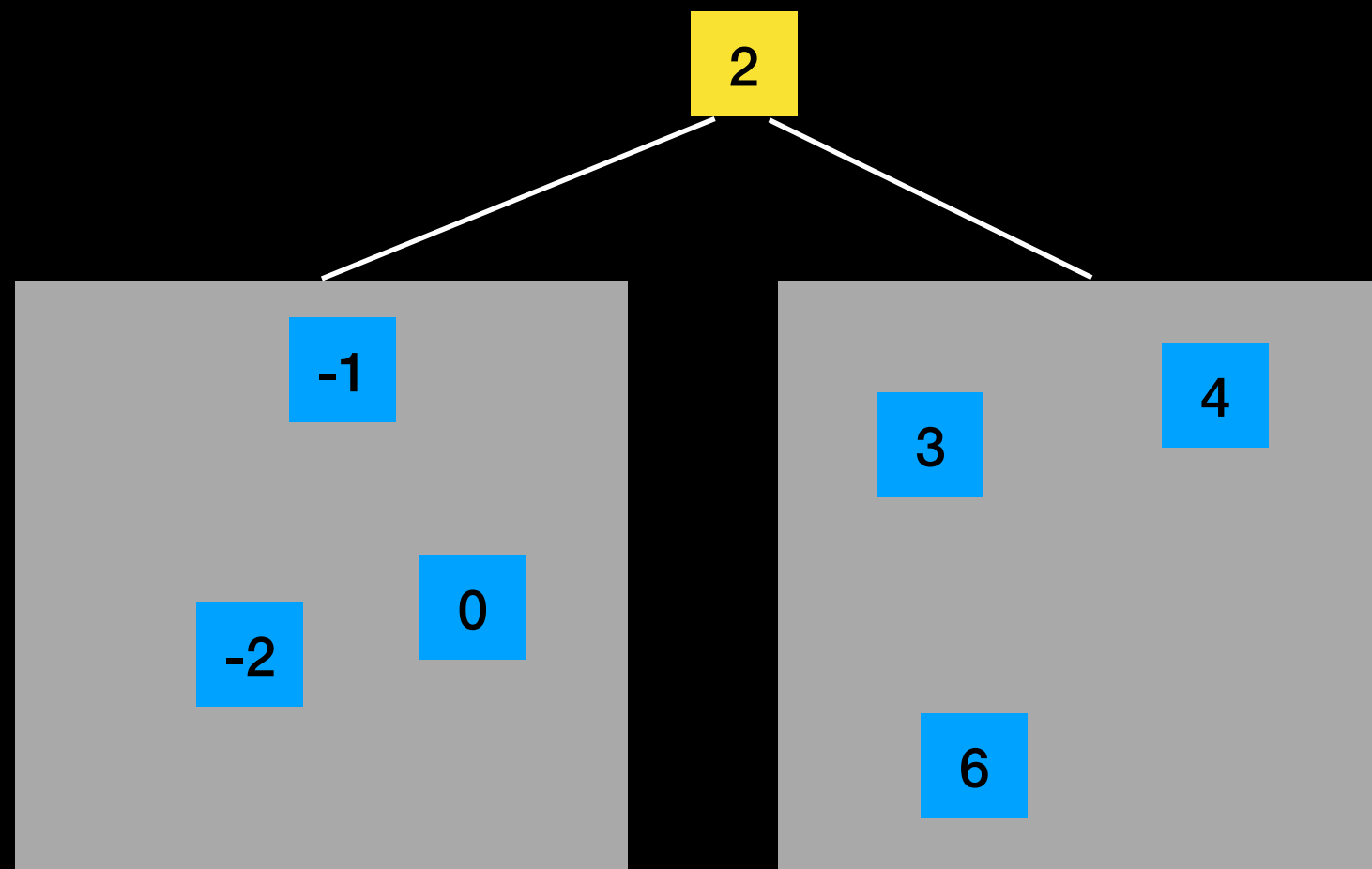
A Different Approach



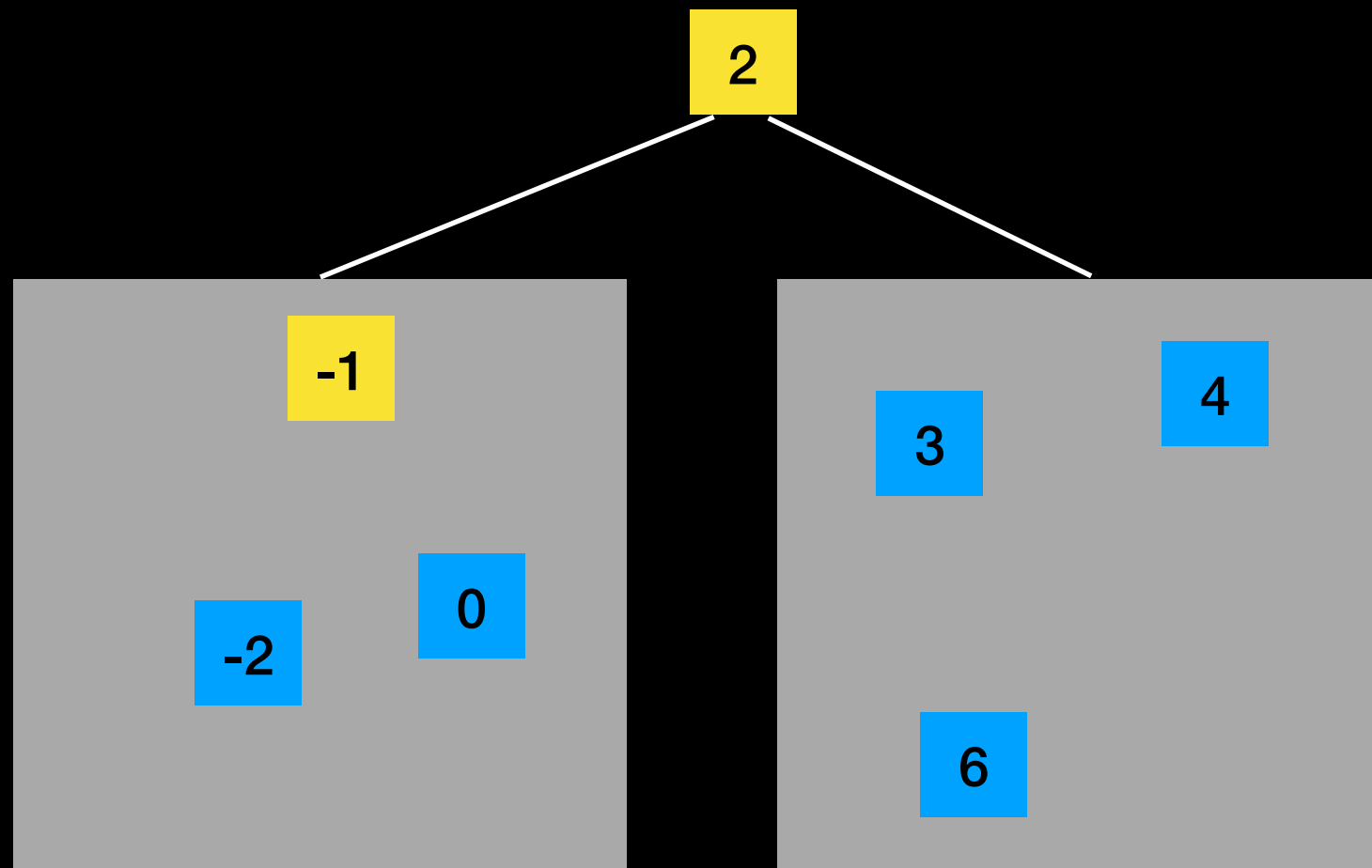
A Different Approach



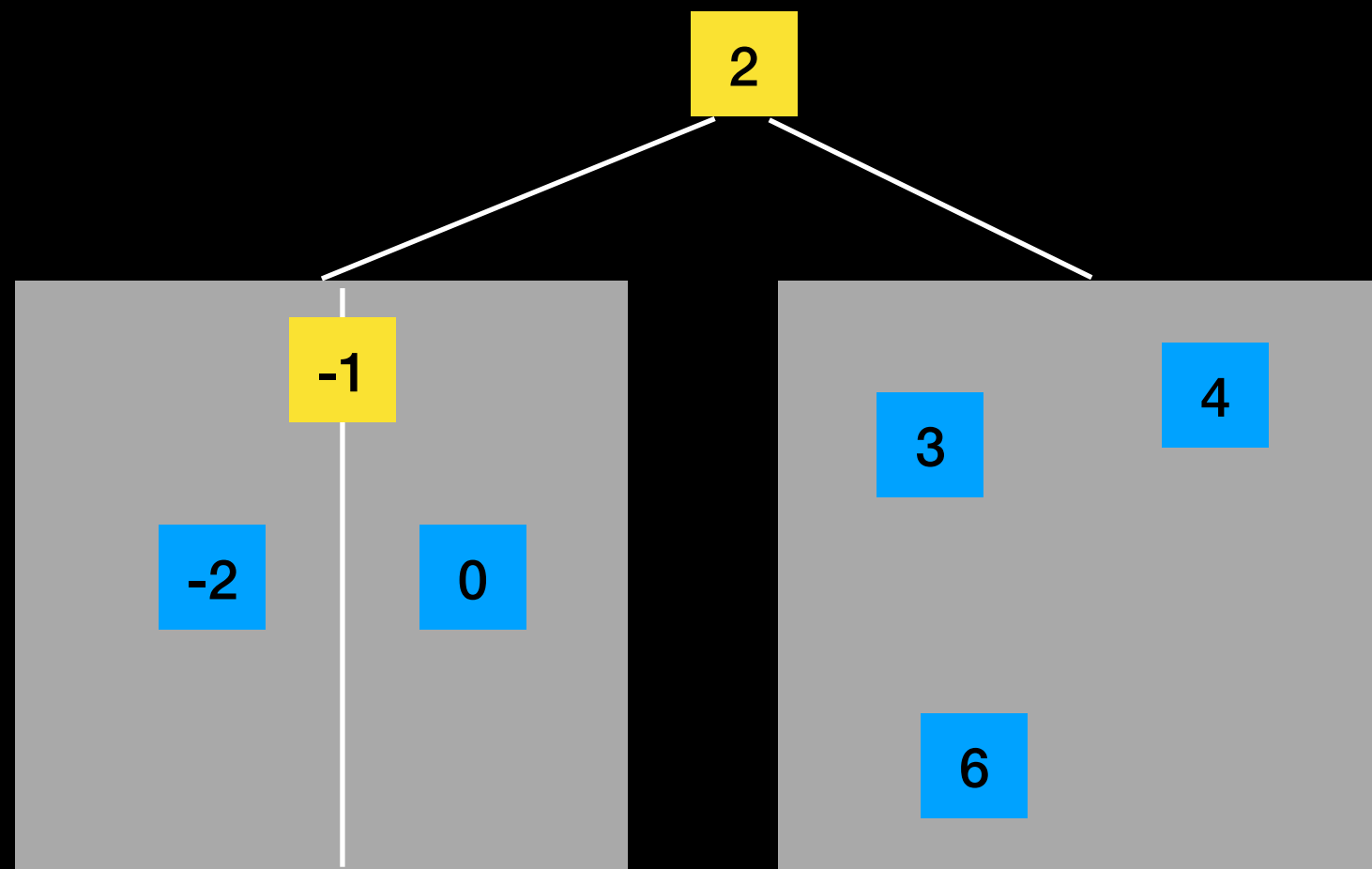
A Different Approach



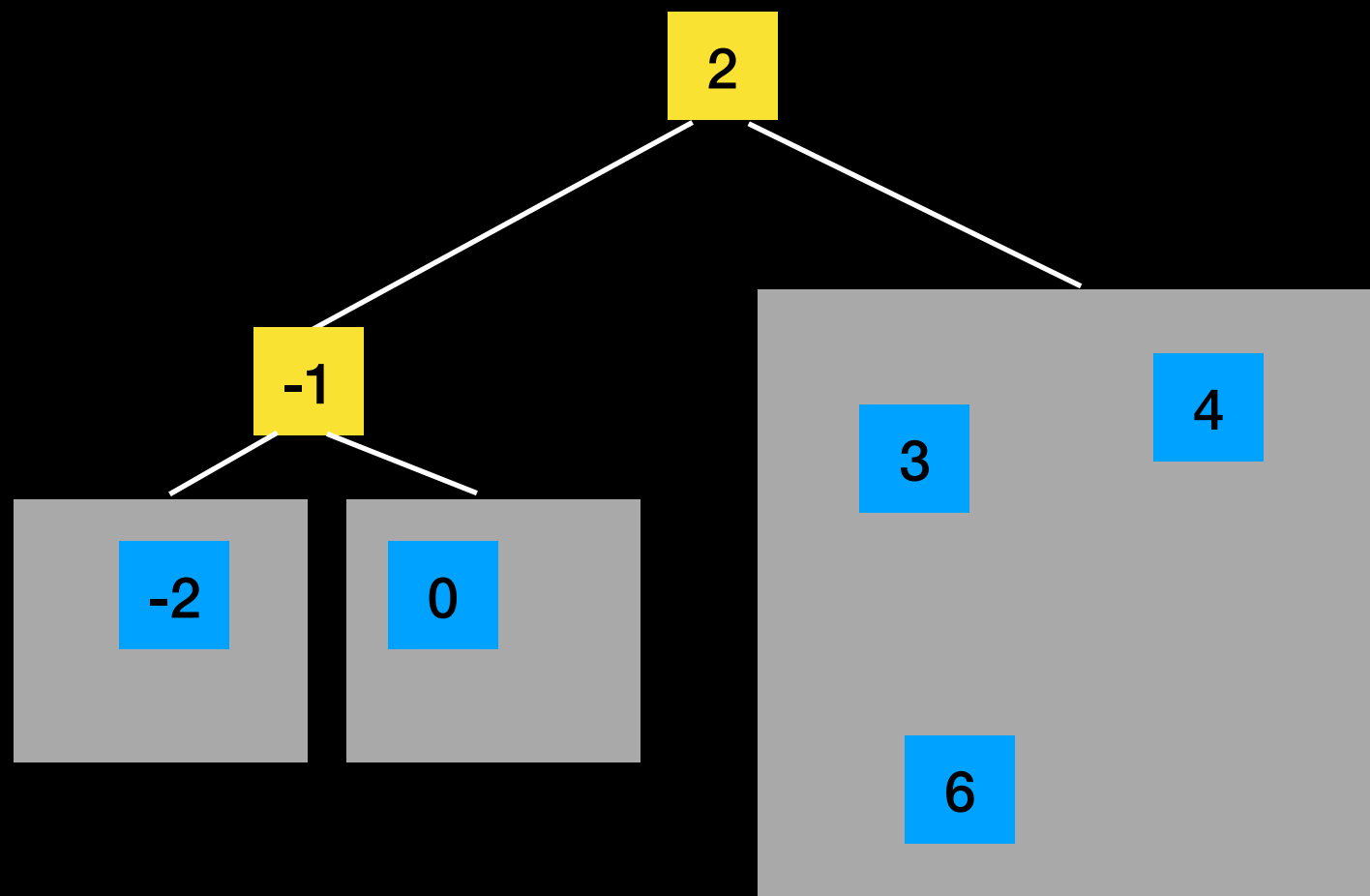
A Different Approach



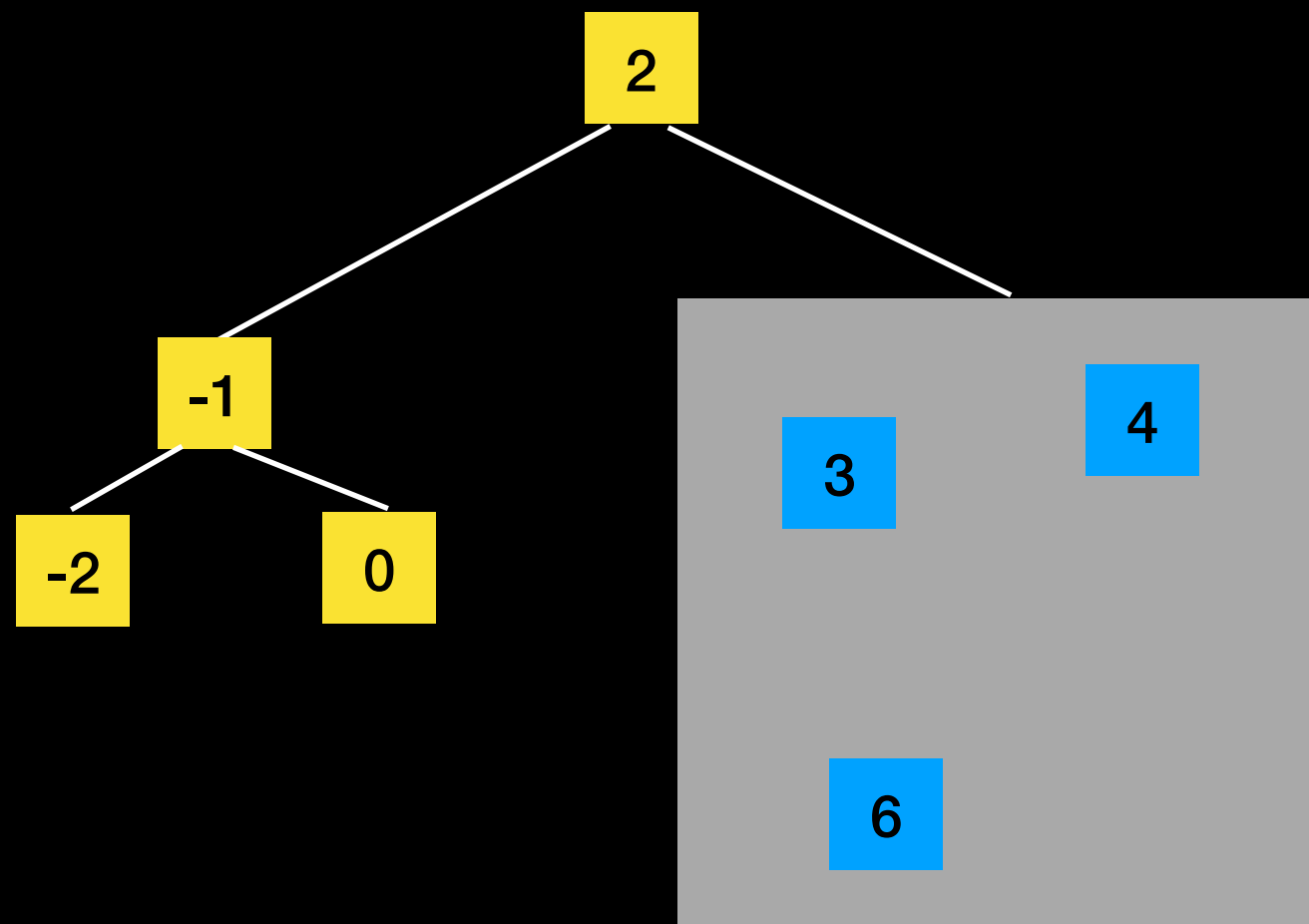
A Different Approach



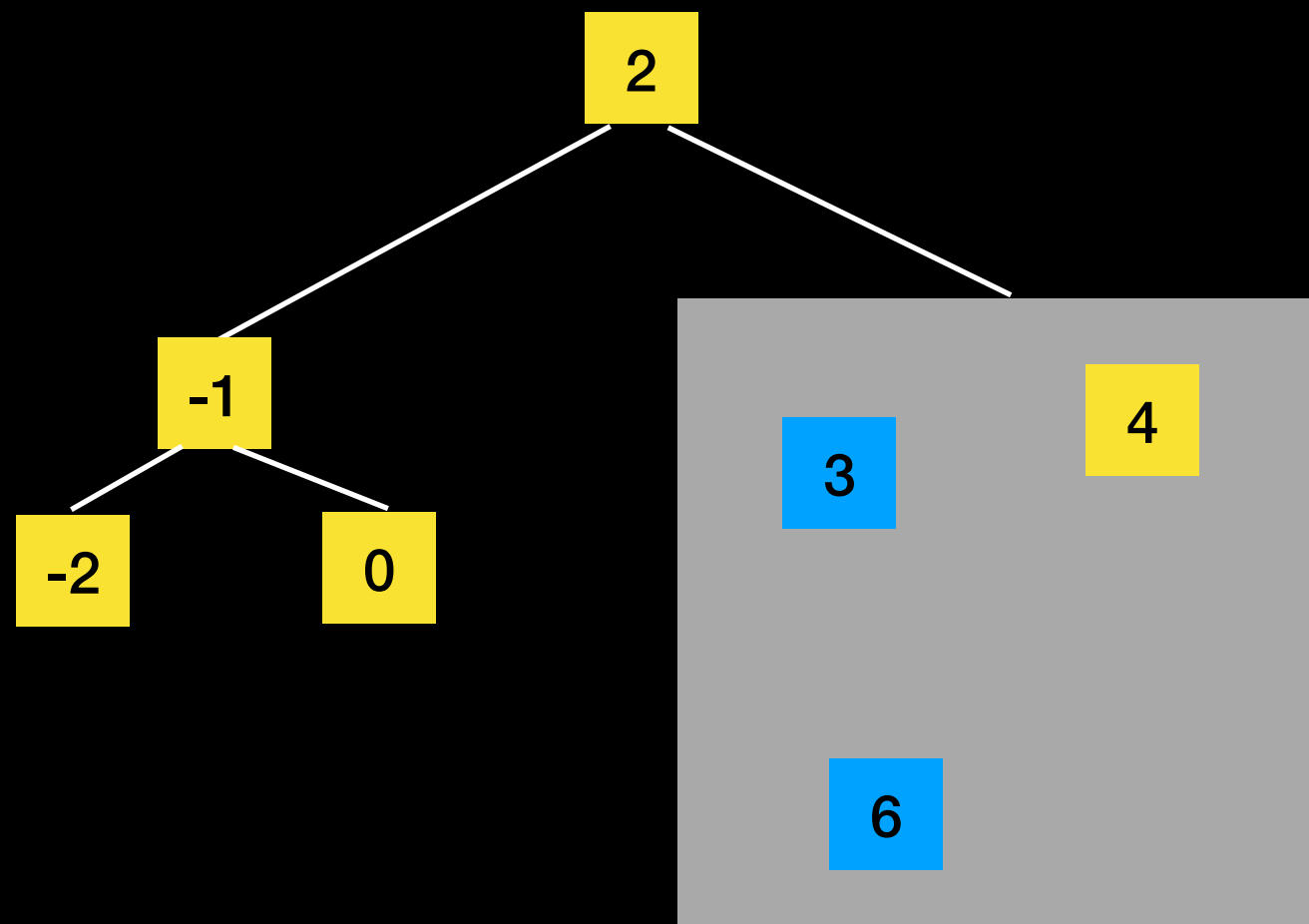
A Different Approach



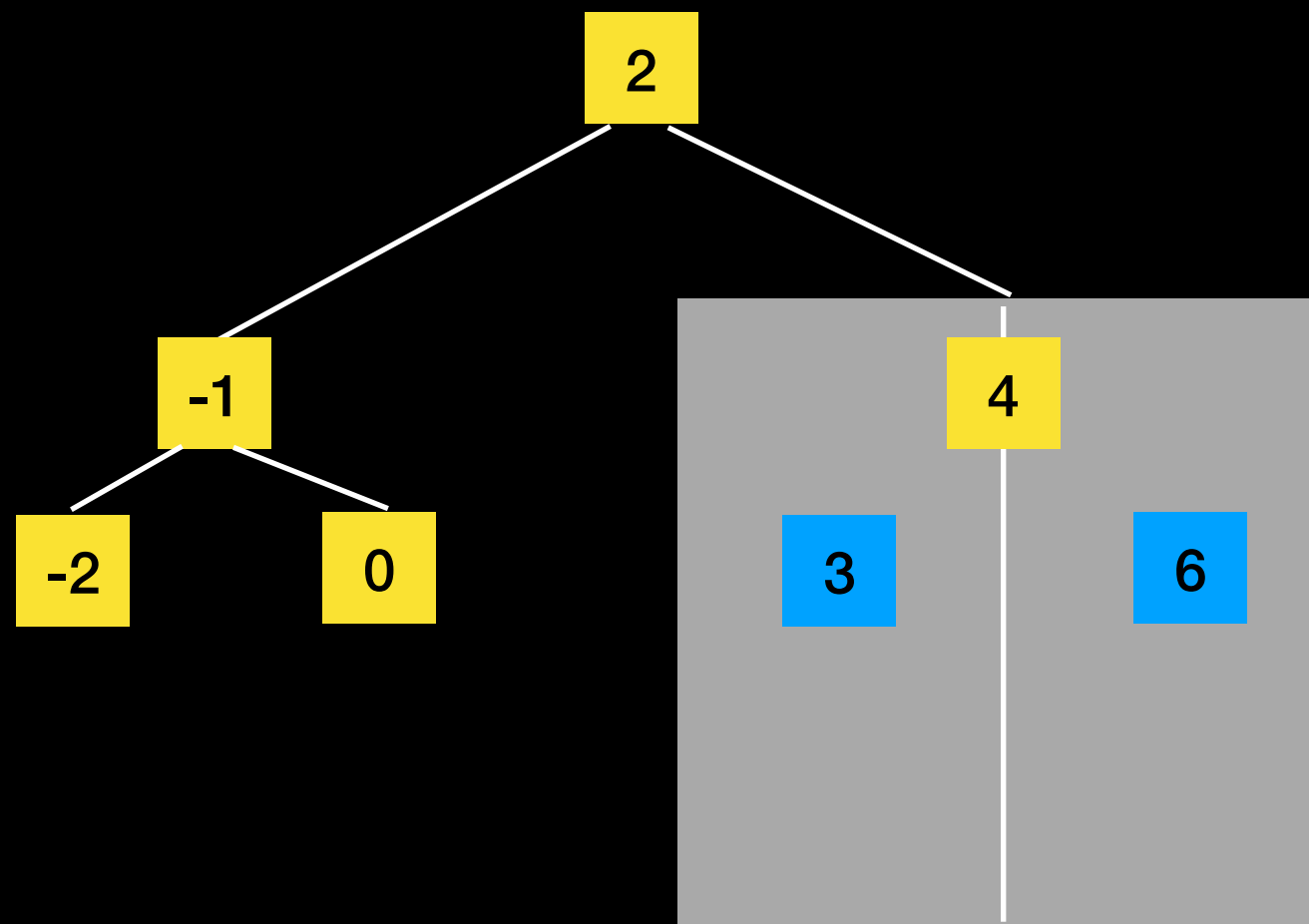
A Different Approach



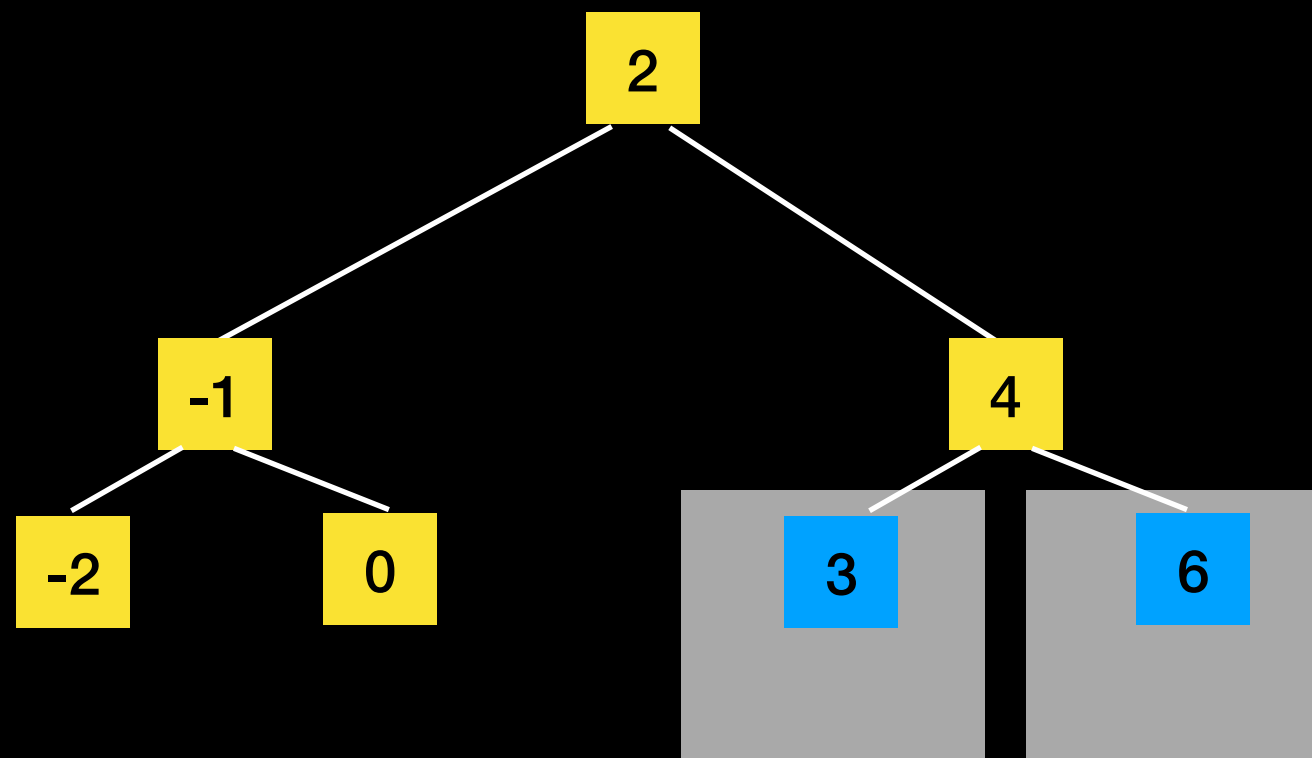
A Different Approach



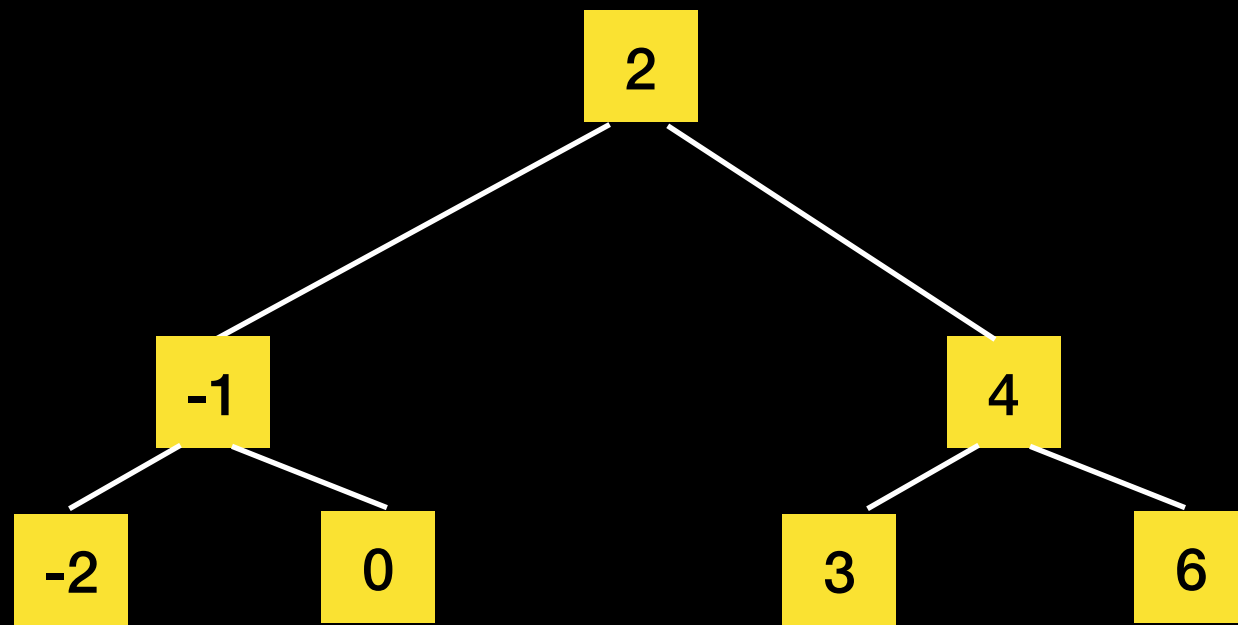
A Different Approach



A Different Approach

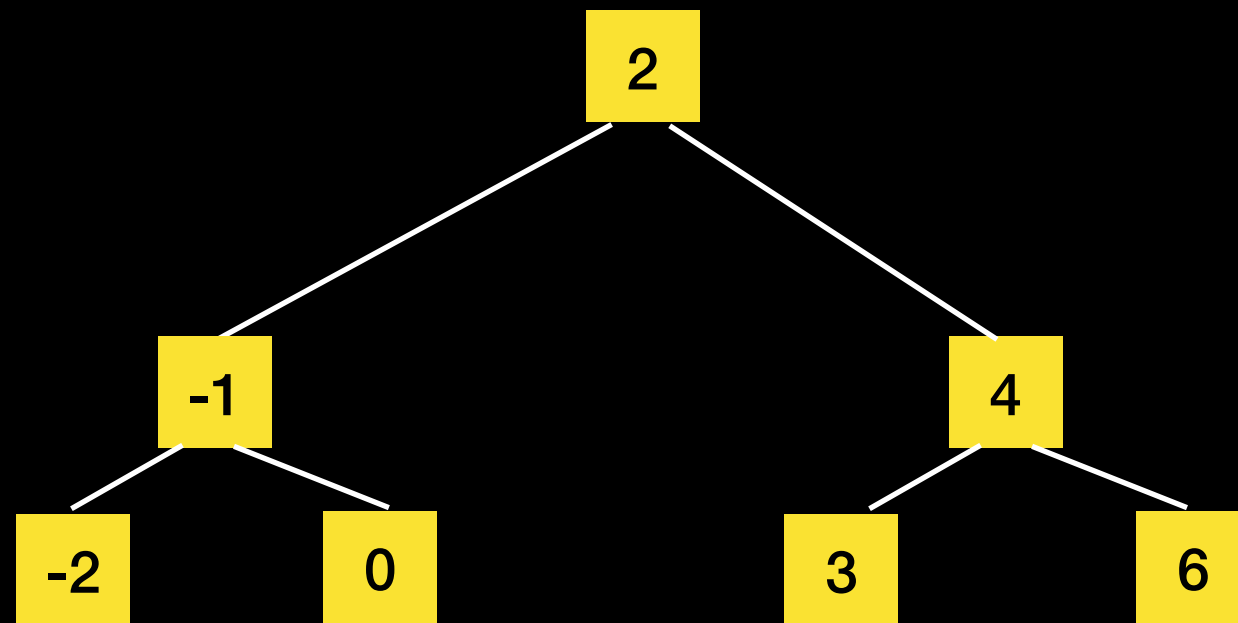


A Different Approach



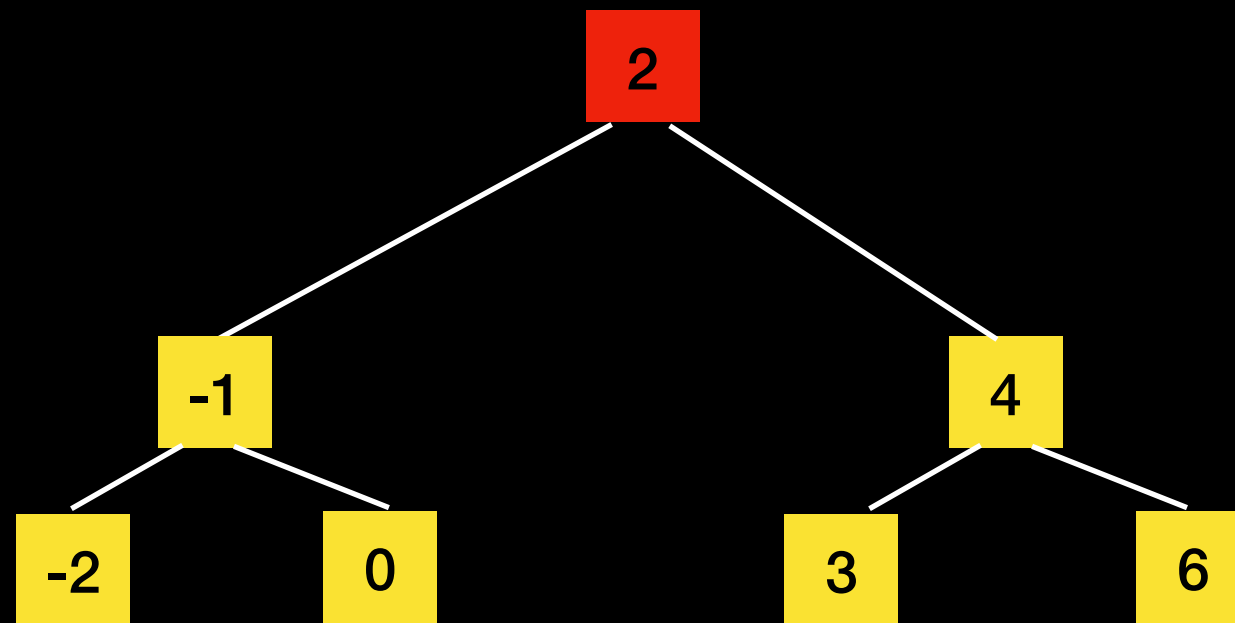
A Different Approach

Find 5



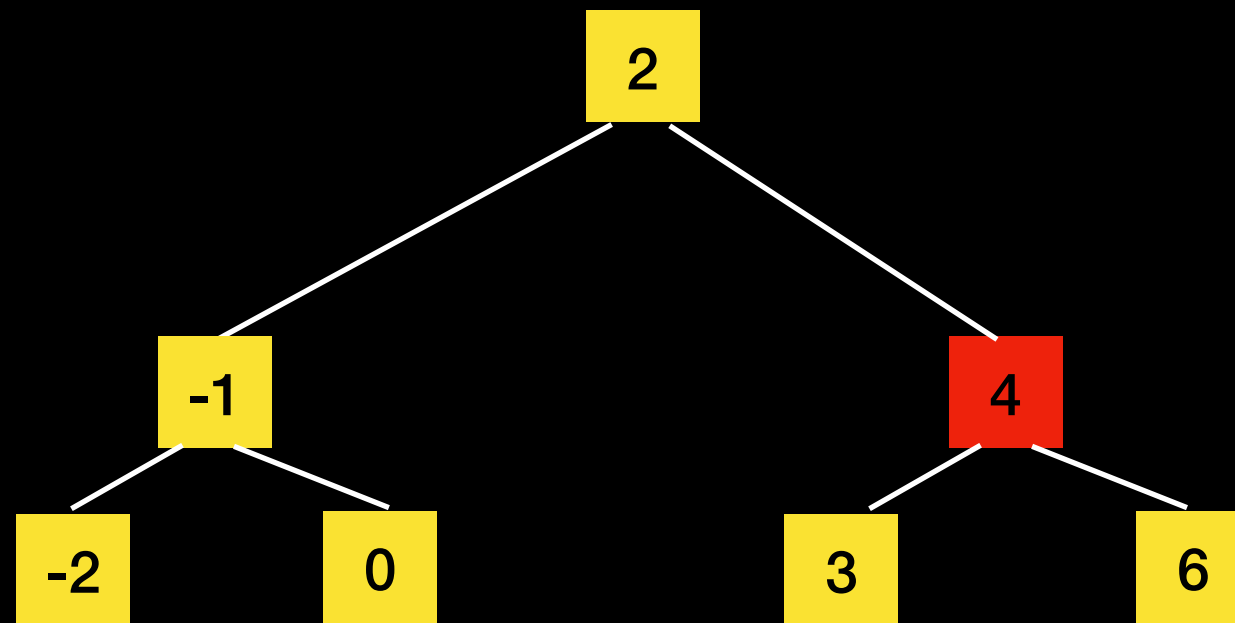
A Different Approach

Find 5



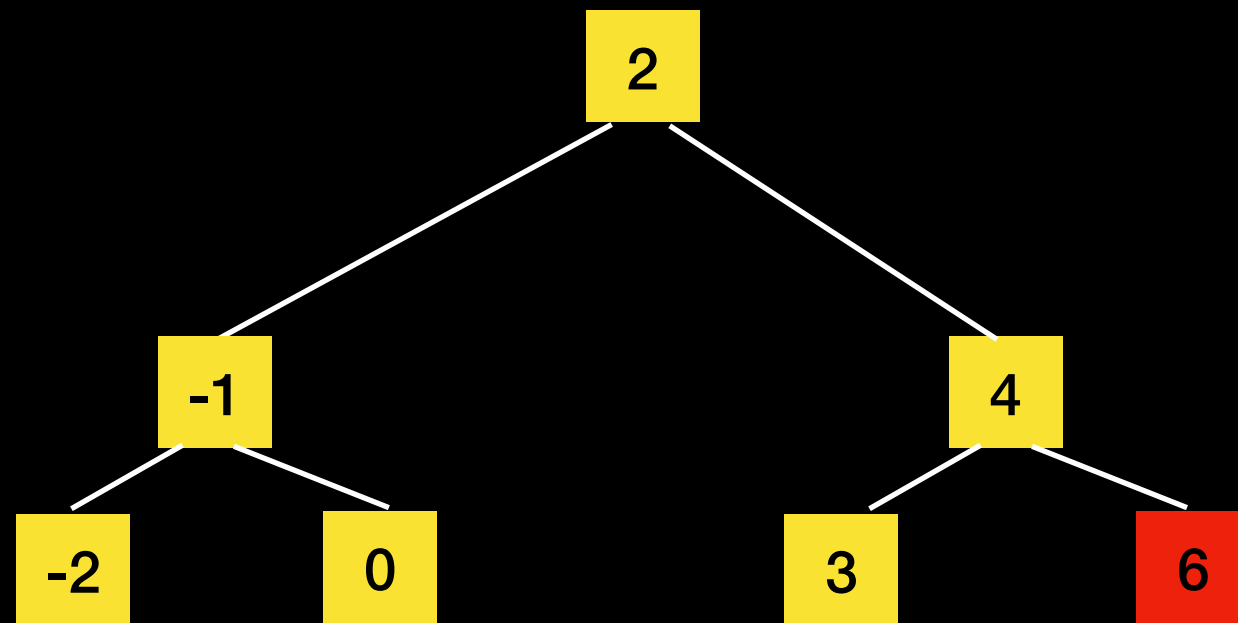
A Different Approach

Find 5



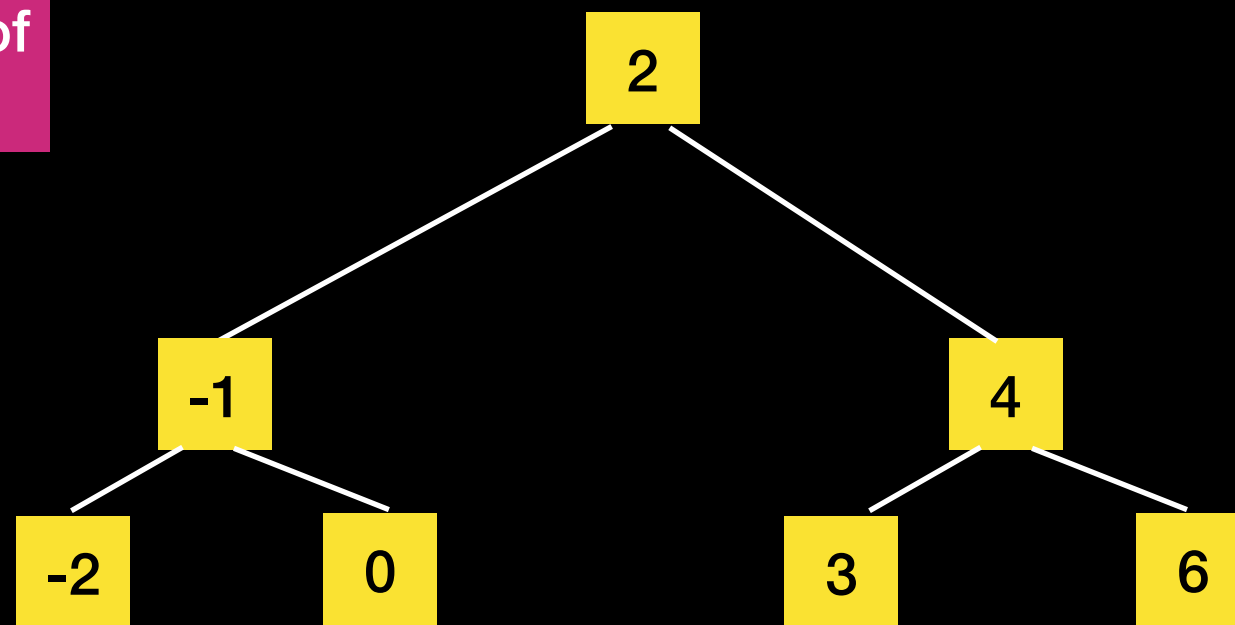
A Different Approach

Find 5



A Different Approach

What's special about the shape of this tree?



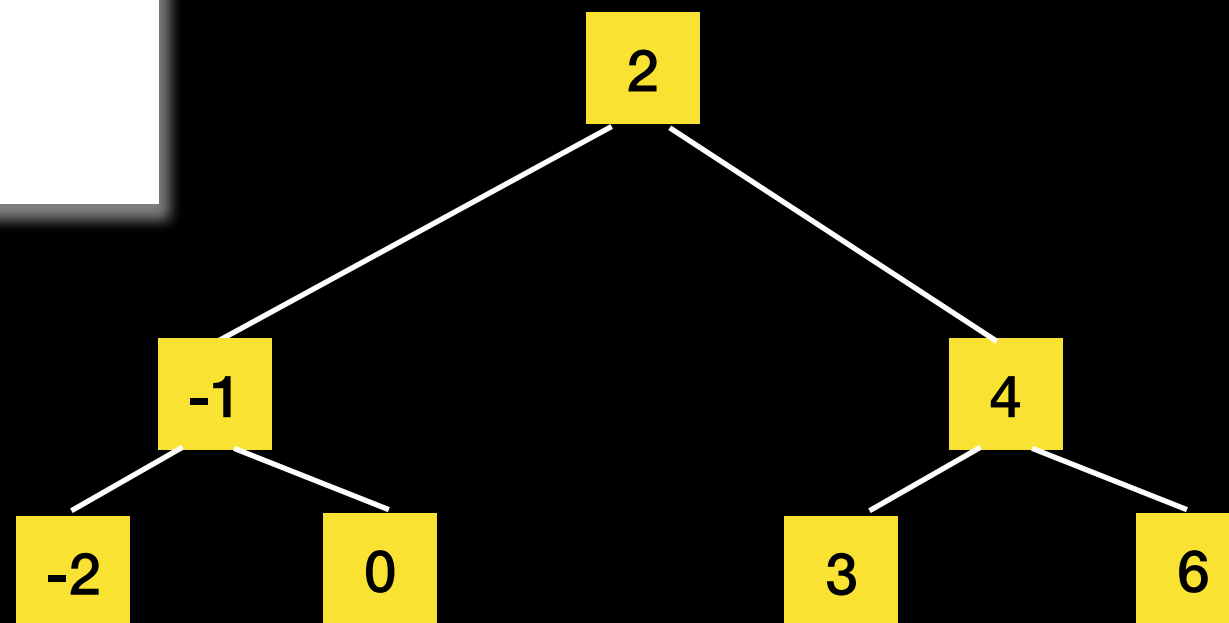
Binary Search Tree

Structural Property:

For each node n

$n > \text{all values in } T_L$

$n < \text{all values in } T_R$



BST Formally

Let S be a set of values upon which a **total ordering relation** $<$, is defined. For example, S can be the set of integers.

A **binary search tree (BST)** T for the ordered set $(S, <)$ is a binary tree with the following properties:

- Each node of T has a value. If p and q are **nodes**, then we write $p < q$ to mean that the value of p is less than the value of q .
- For each node $n \in T$, if p is a node in the left subtree of n , then $p < n$.
- For each node $n \in T$, if p is a node in the right subtree of n , then $n < p$.
- For each element $s \in S$ there exists a node $n \in T$ such that $s = n$.

Binary Search Tree

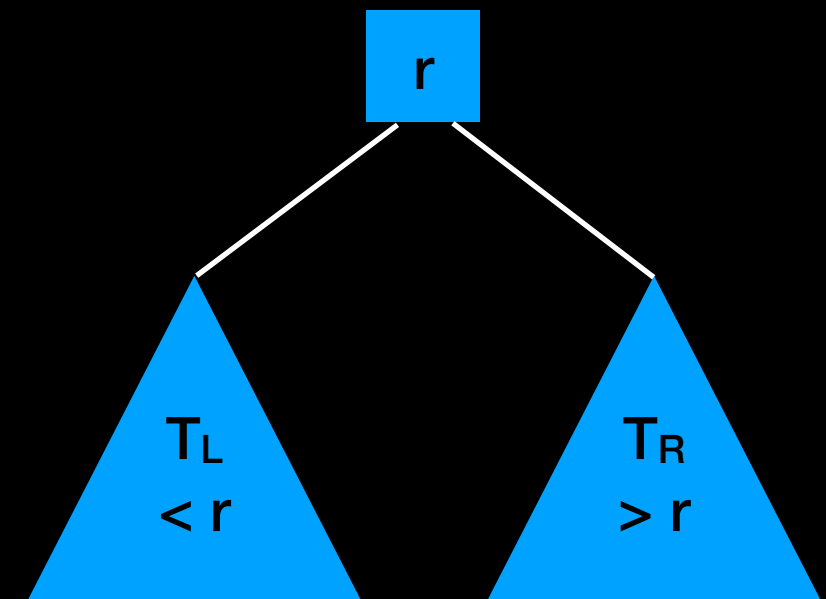
Structural Property:

For each node n

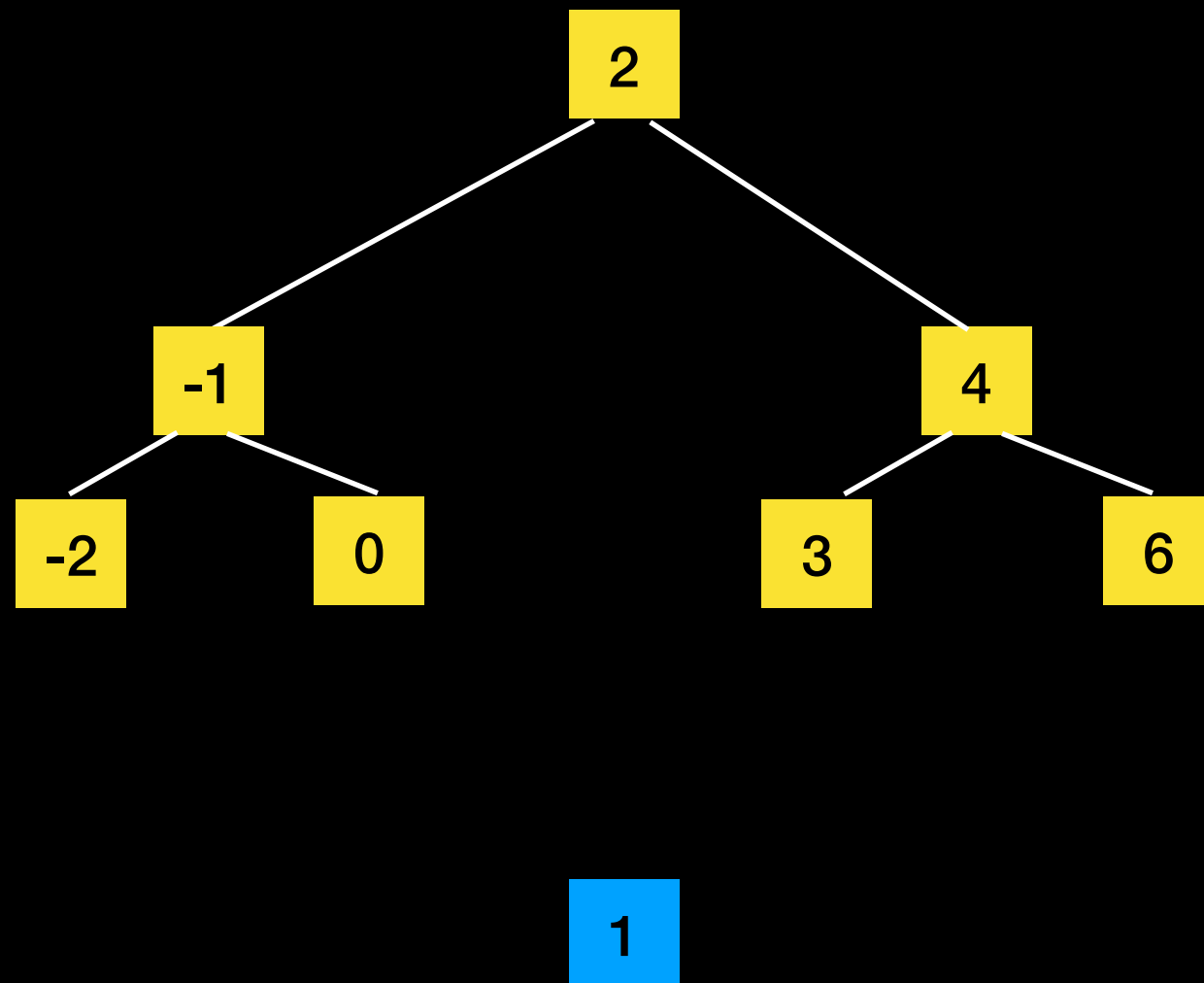
$n >$ all values in T_L

$n <$ all values in T_R

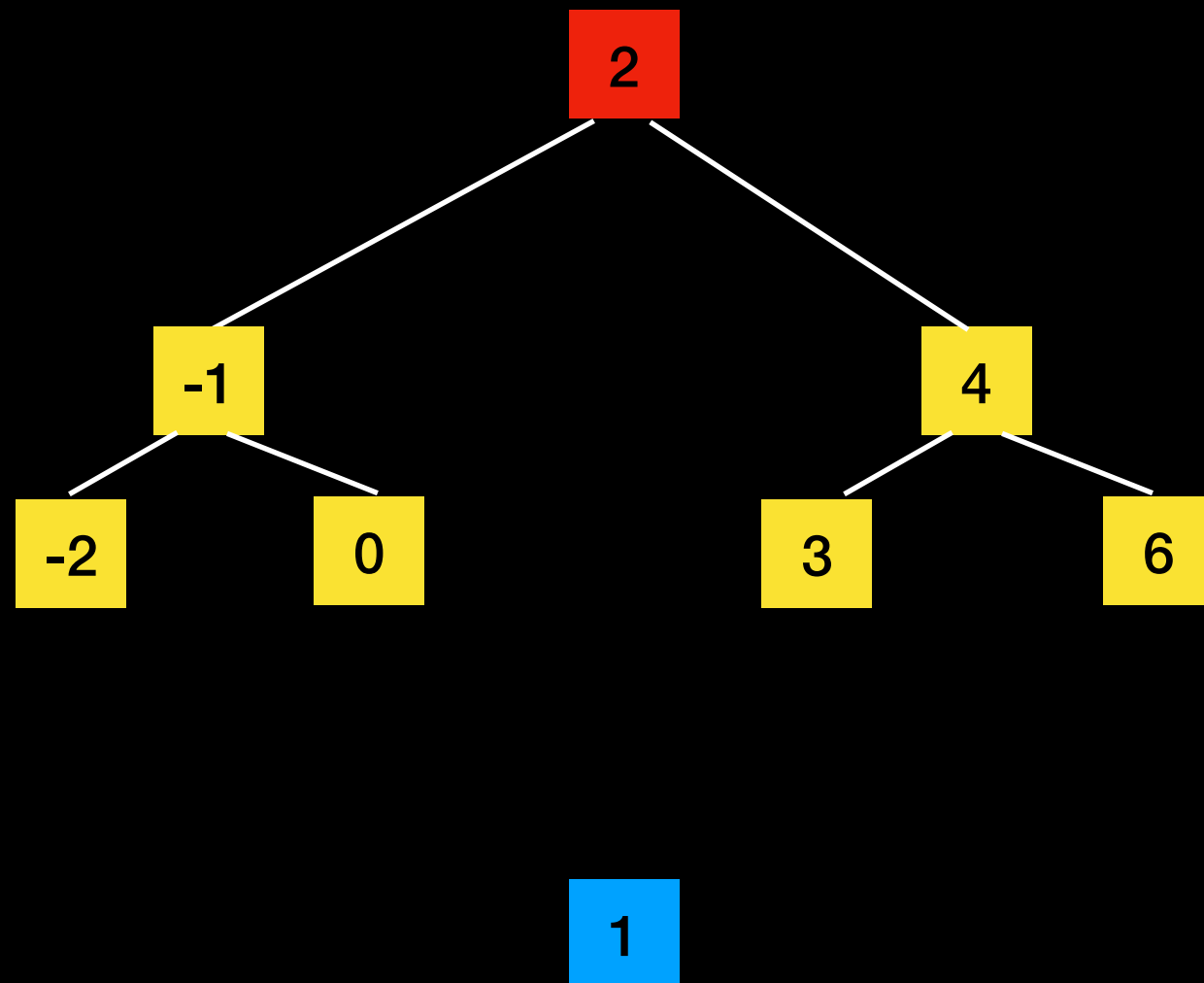
```
search(bs_tree, item)
{
    if (bs_tree is empty) //base case
        item not found
    else if (item == root)
        return root
    else if (item < root)
        search( $T_L$ , item)
    else // item > root
        search( $T_R$ , item)
}
```



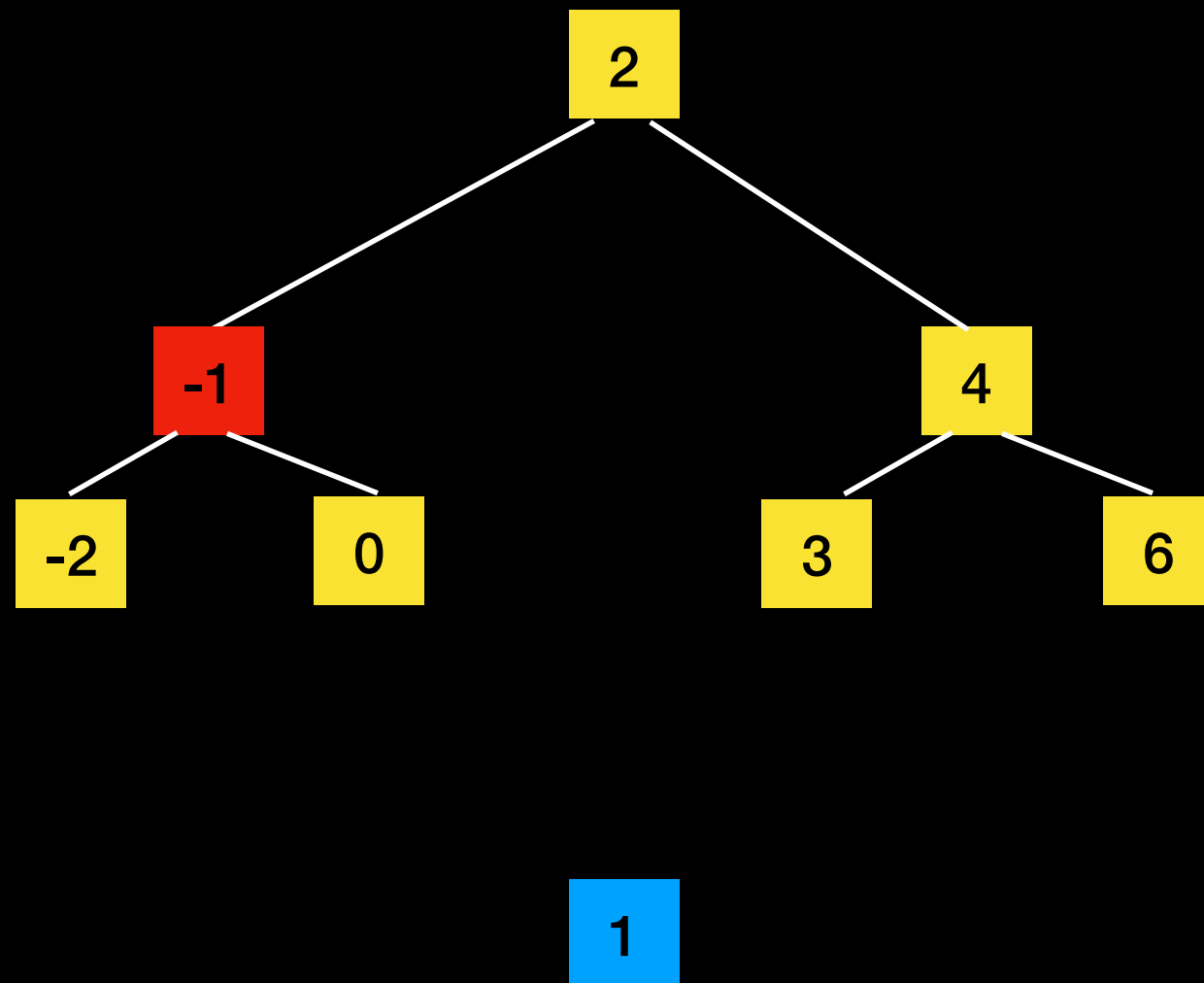
Inserting into a BST



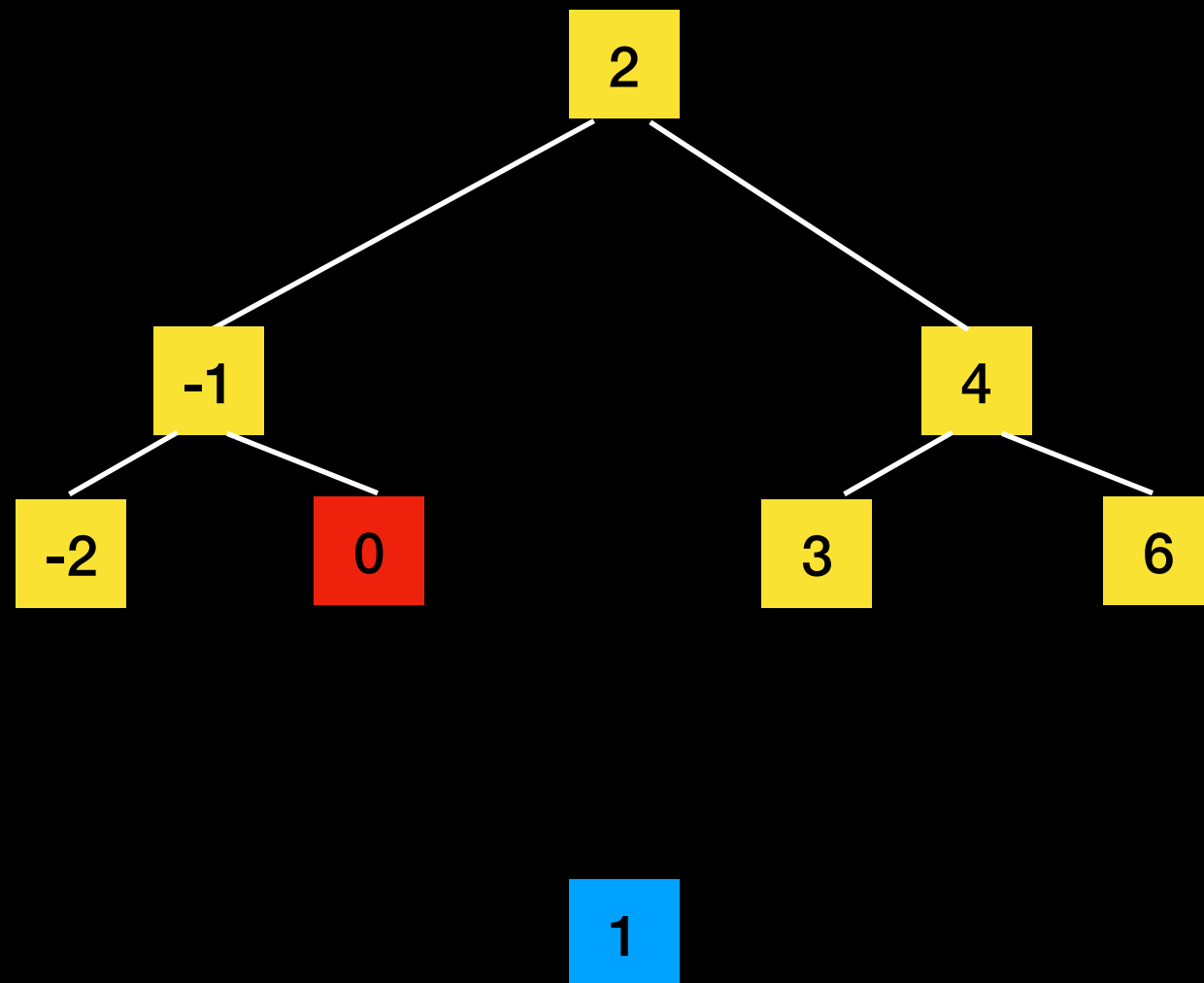
Inserting into a BST



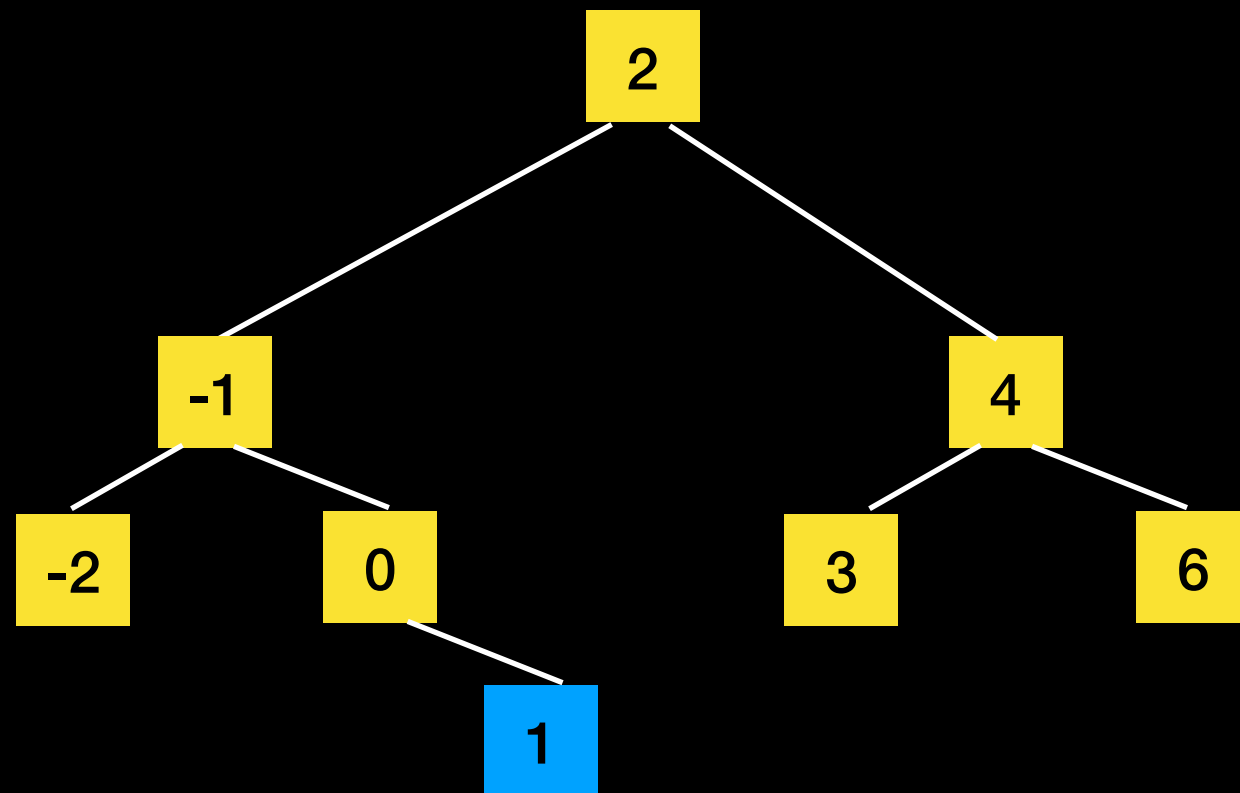
Inserting into a BST



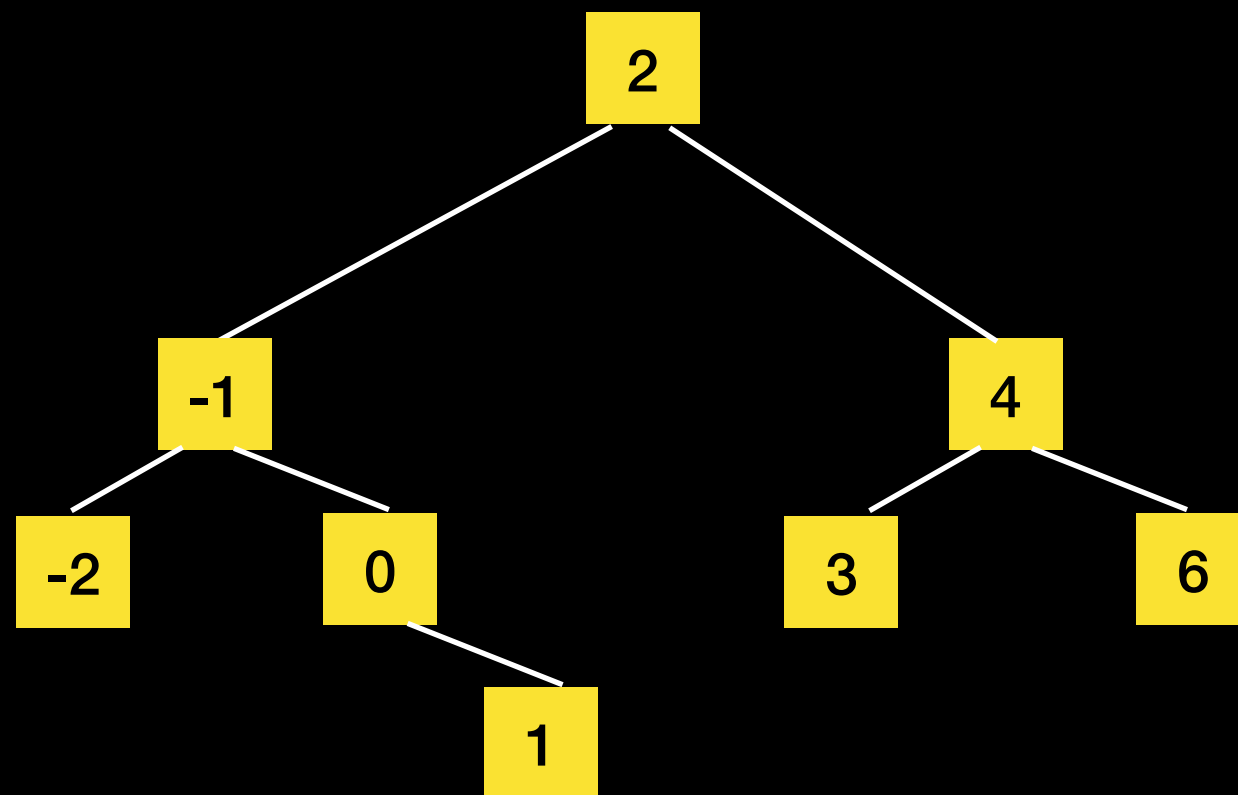
Inserting into a BST



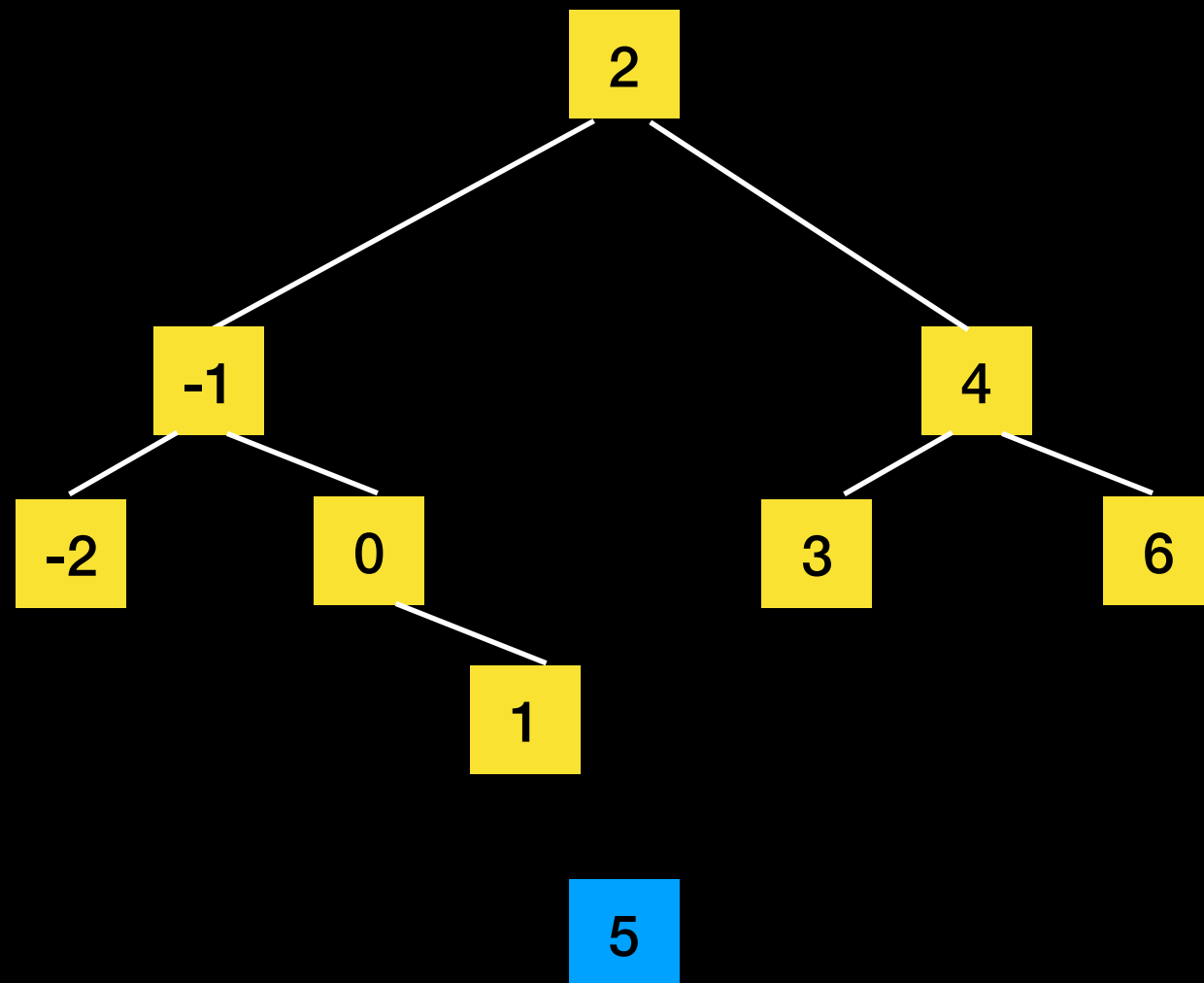
Inserting into a BST



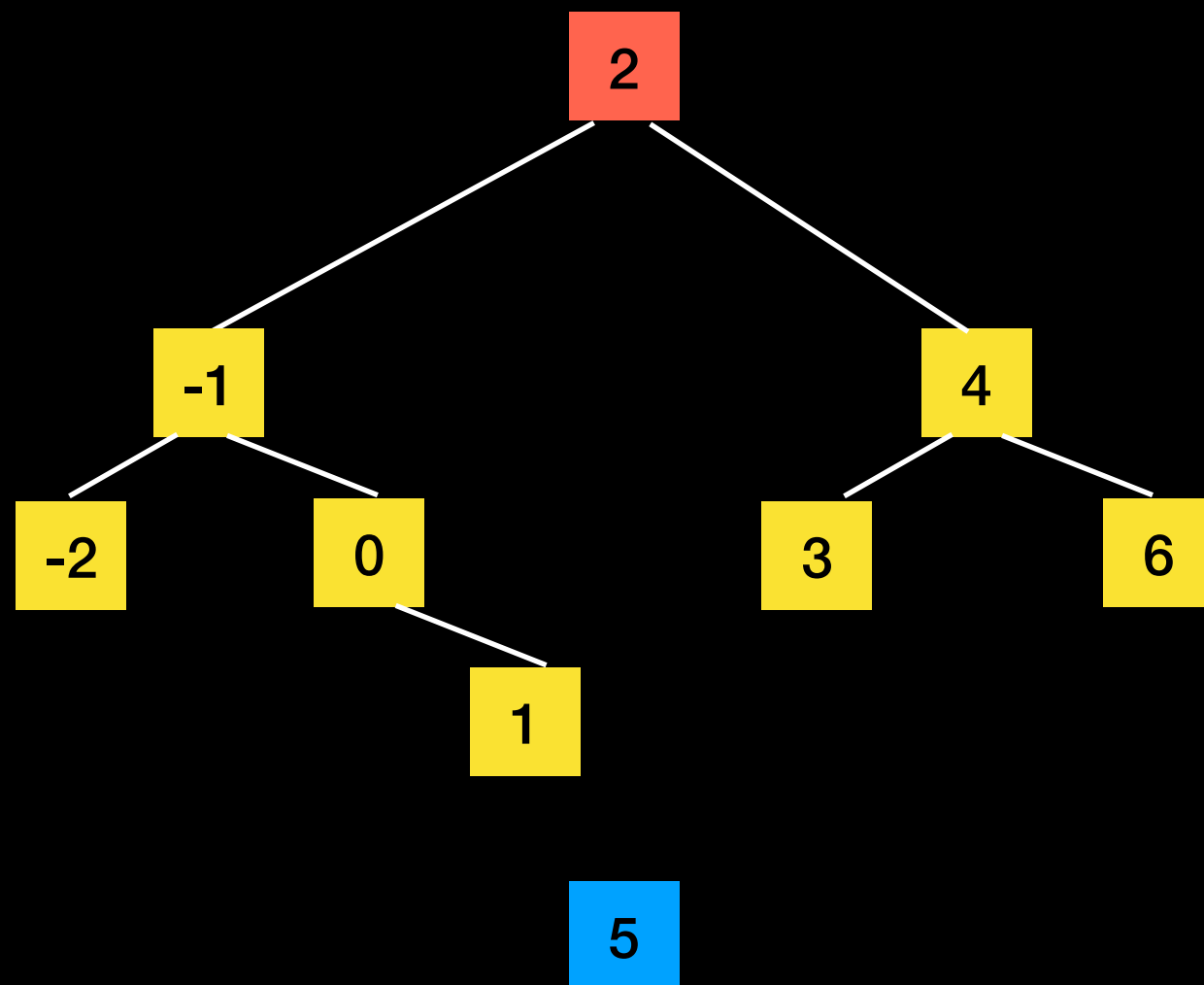
Inserting into a BST



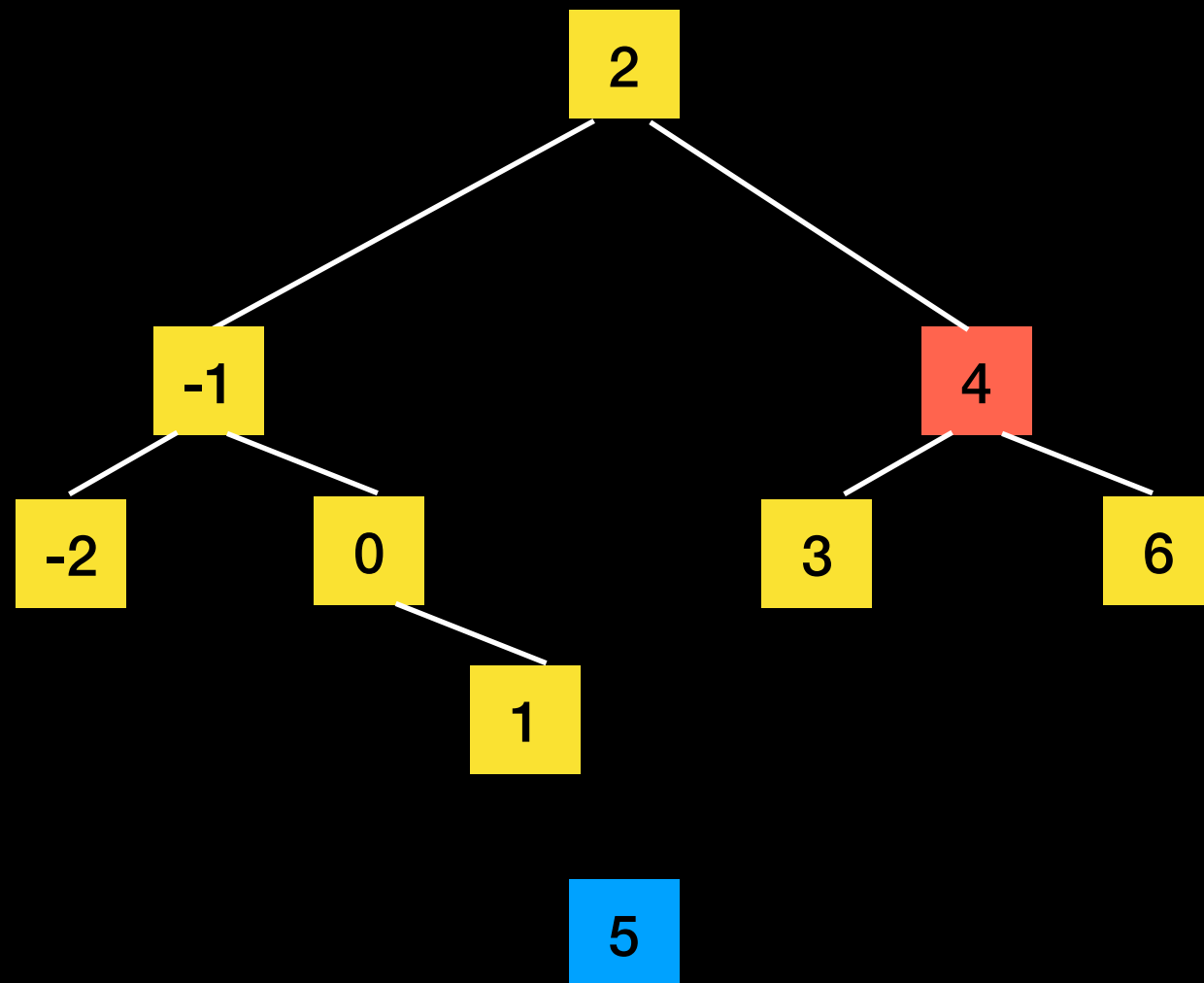
Inserting into a BST



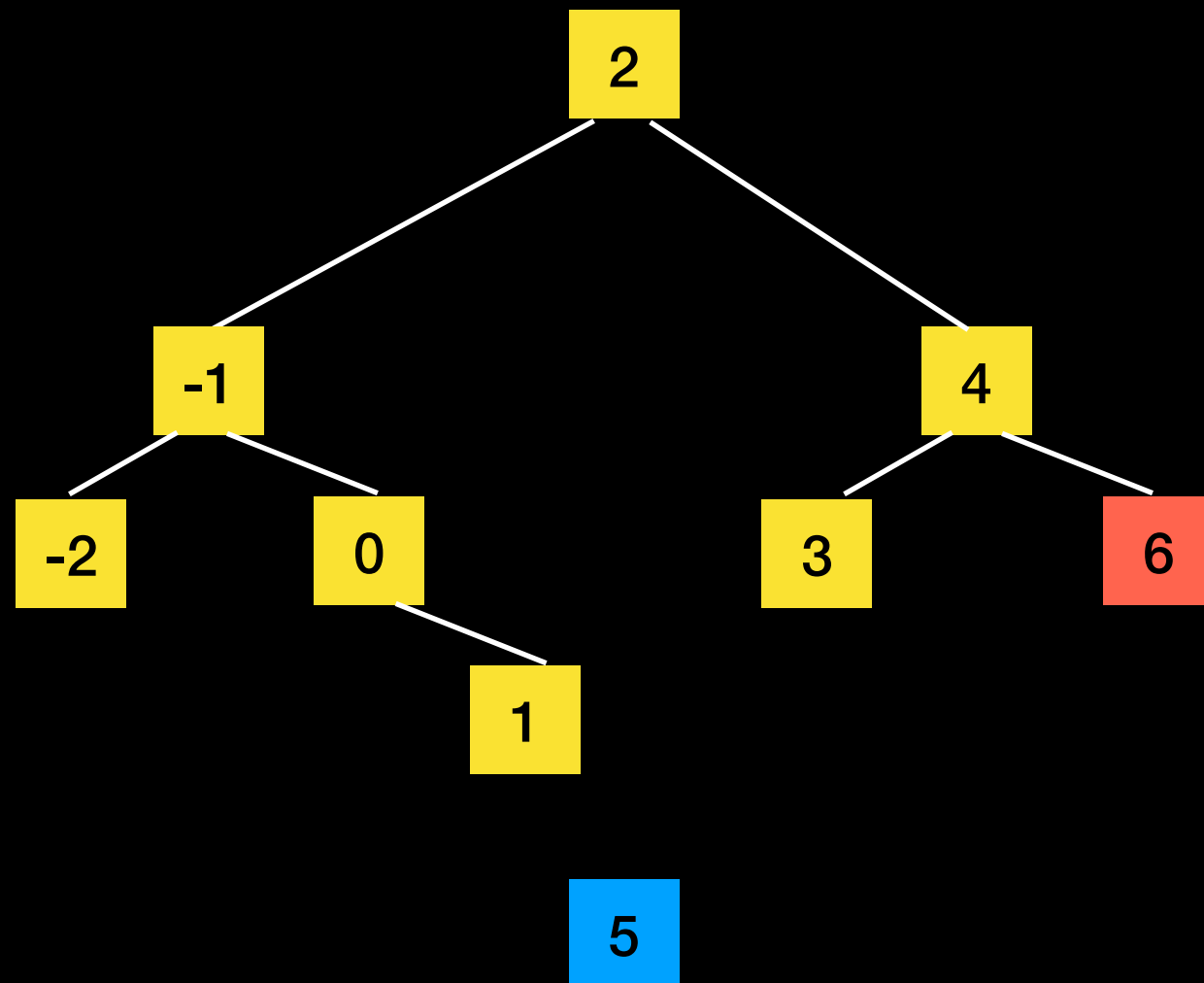
Inserting into a BST



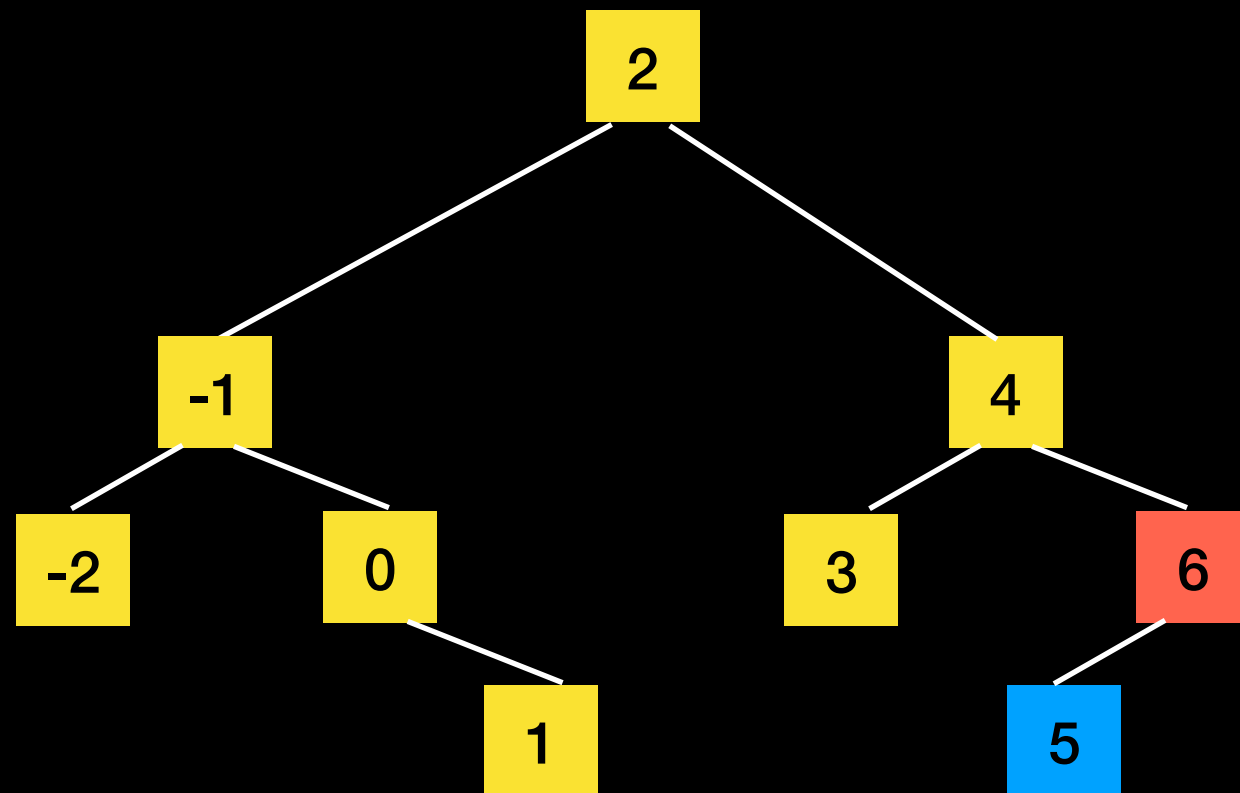
Inserting into a BST



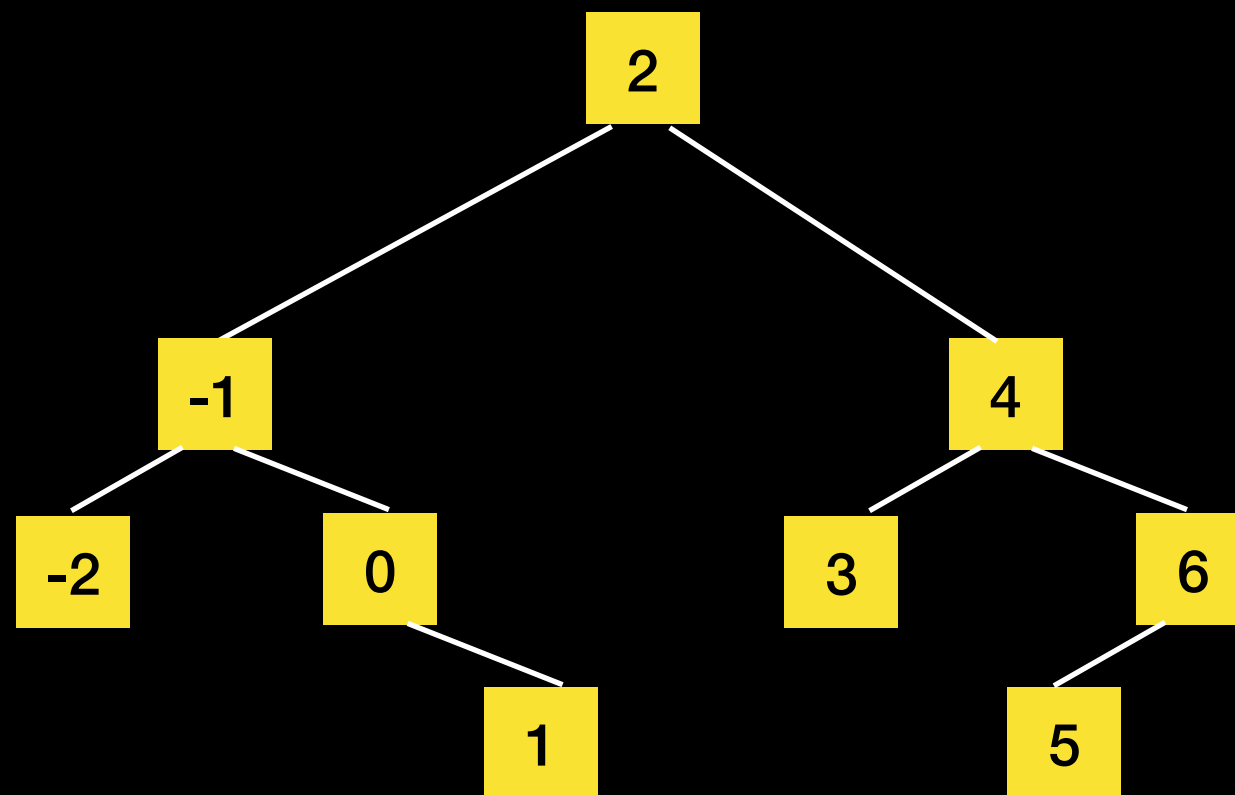
Inserting into a BST



Inserting into a BST

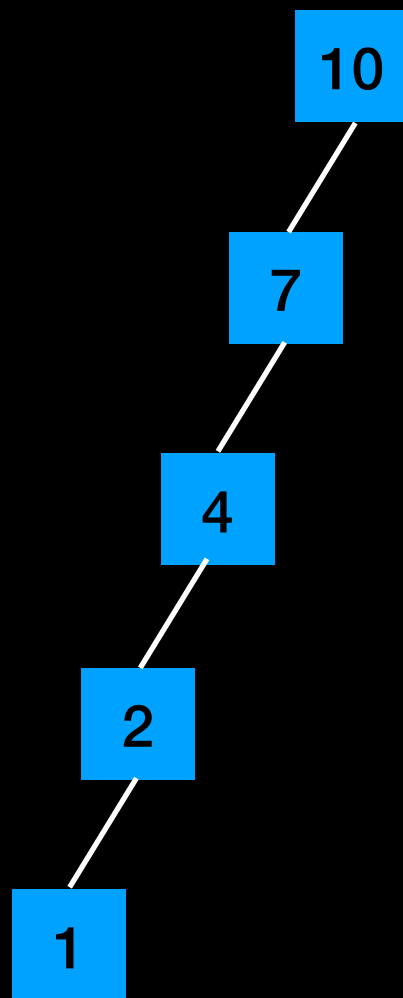


Inserting into a BST

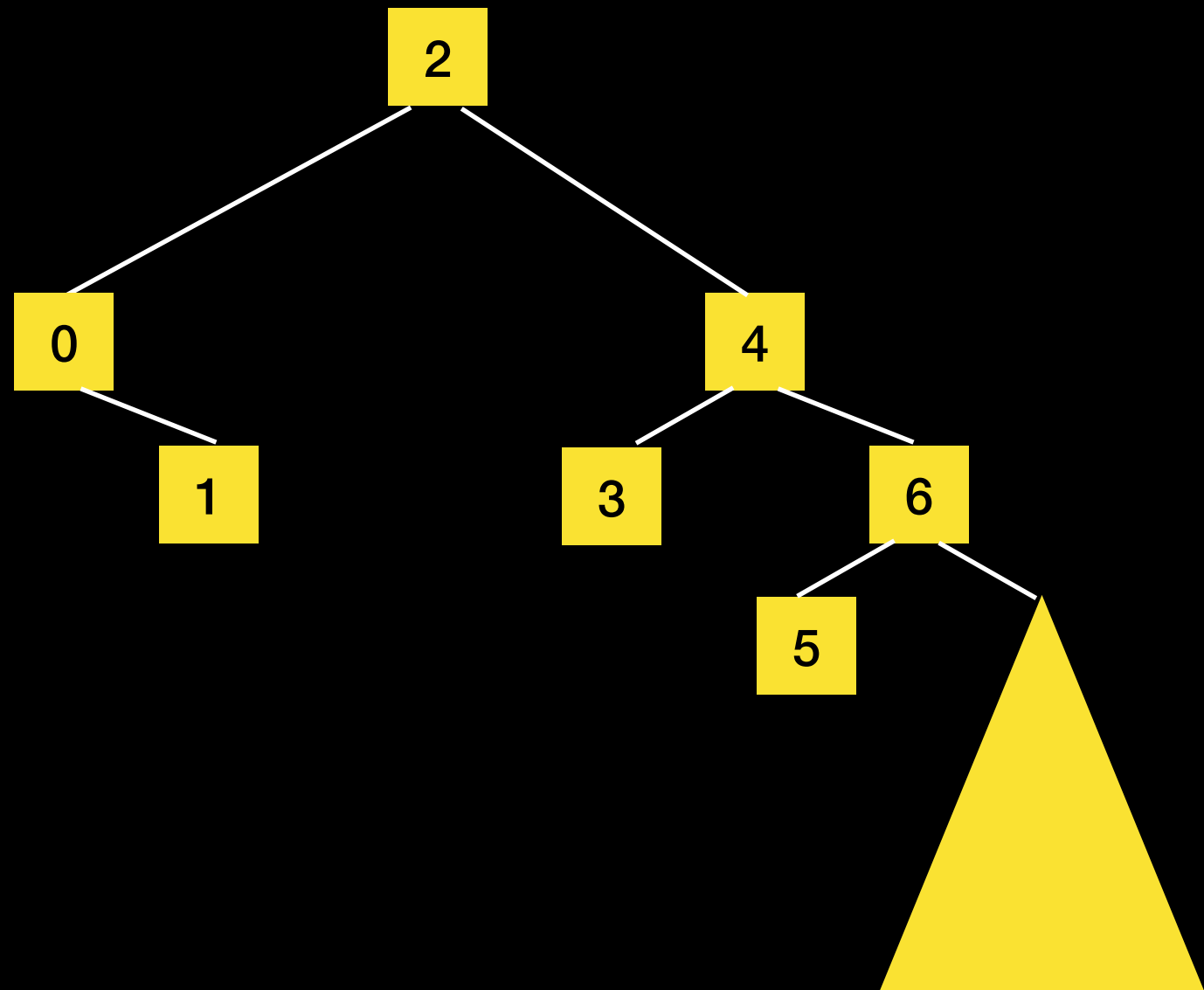
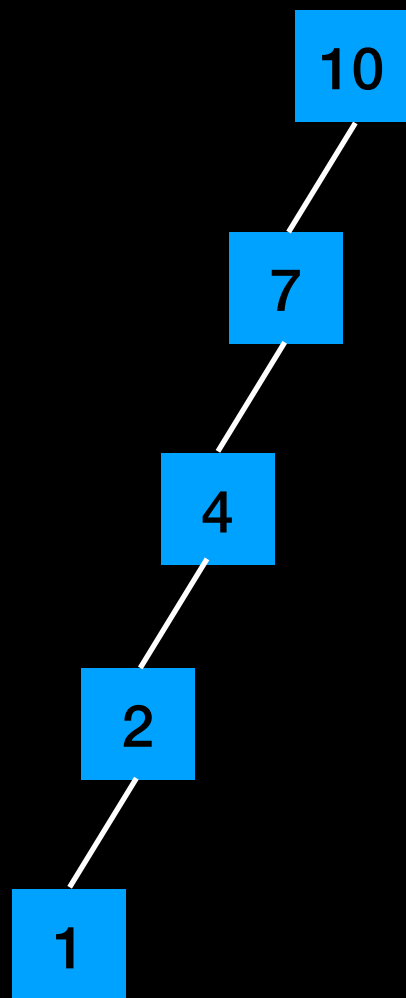


You **Grow** a tree with BST property, you don't get to restructure it
(Self-balancing trees (e.g. Red-Black trees) will do that, perhaps in CSCI 335)

Growing a BST

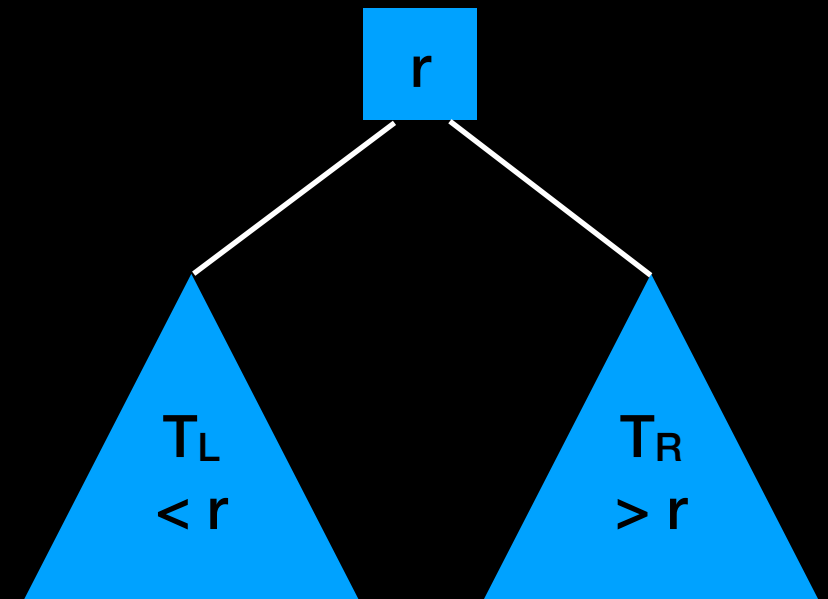


Growing a BST



Inserting into a BST

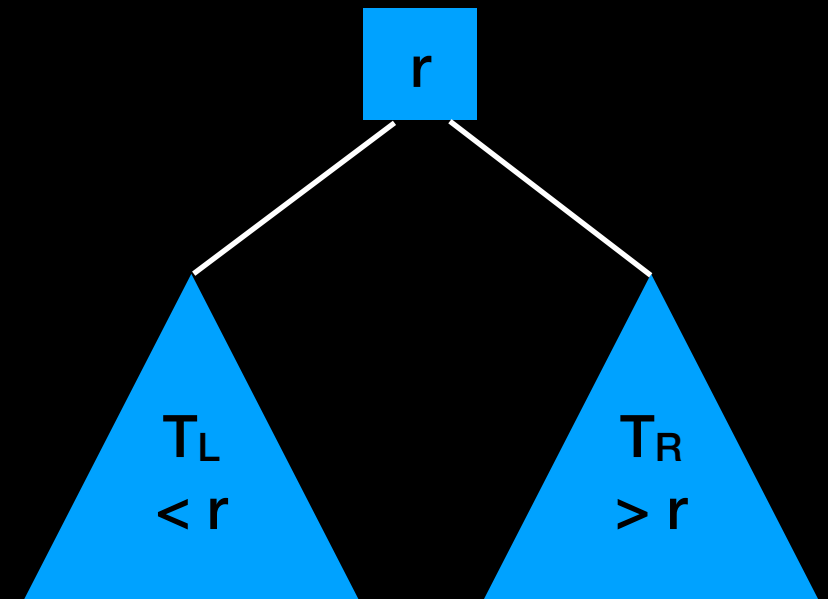
```
add(bs_tree, item)
{
    if (bs_tree is empty) //base case
        make item the root
    else if (item < root)
        add(TL, item)
    else // item > root
        add(TR, item)
}
```



Traversing a BST

Same as traversing
any binary tree

Which type of
traversal is special
for a BST?

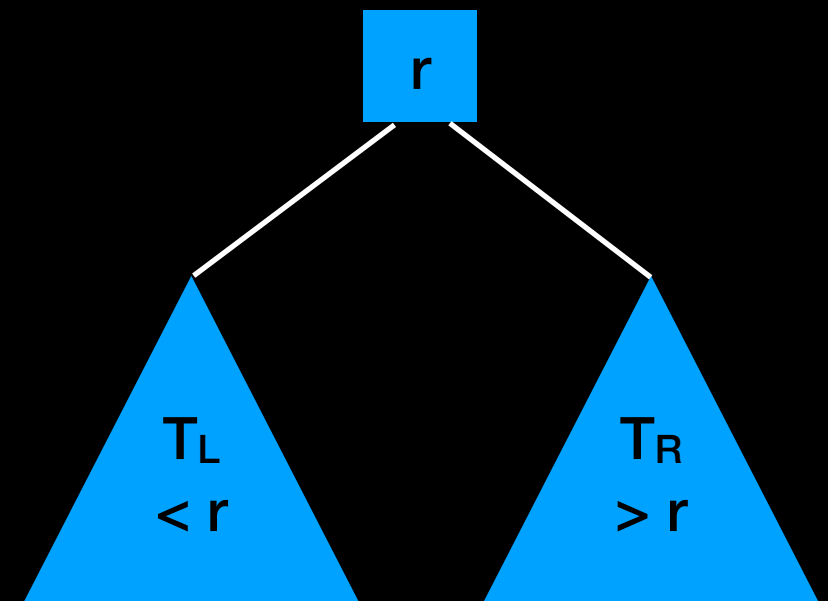


Traversing a BST

Same as traversing
any binary tree

```
inorder(bs_tree)
{
    //implicit base case
    if (bs_tree is not empty)
    {
        inorder(TL)
        visit the root
        inorder(TR)
    }
}
```

Visits nodes in sorted
ascending order



Efficiency of BST

Searching is key to most operations

Think about the structure and height of the tree

Efficiency of BST

Searching is key to most operations

Think about the structure and height of the tree

$O(h)$

What is the **maximum height**?

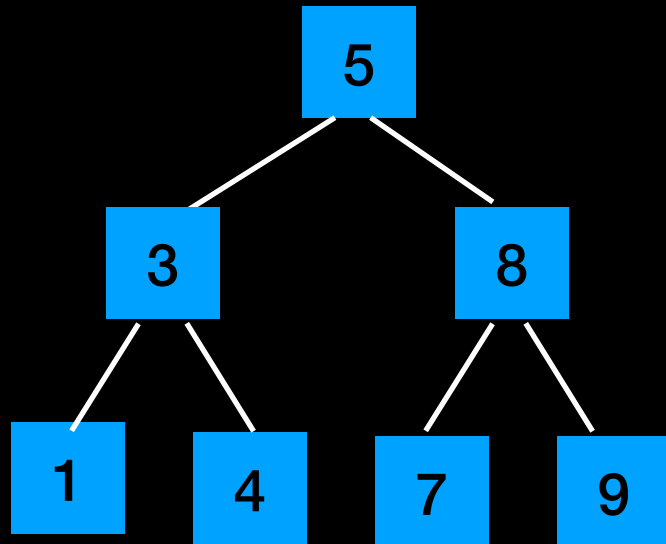
What is the **minimum height**?

Tree Structure

$n = 7$

$h = 3$

Full BST

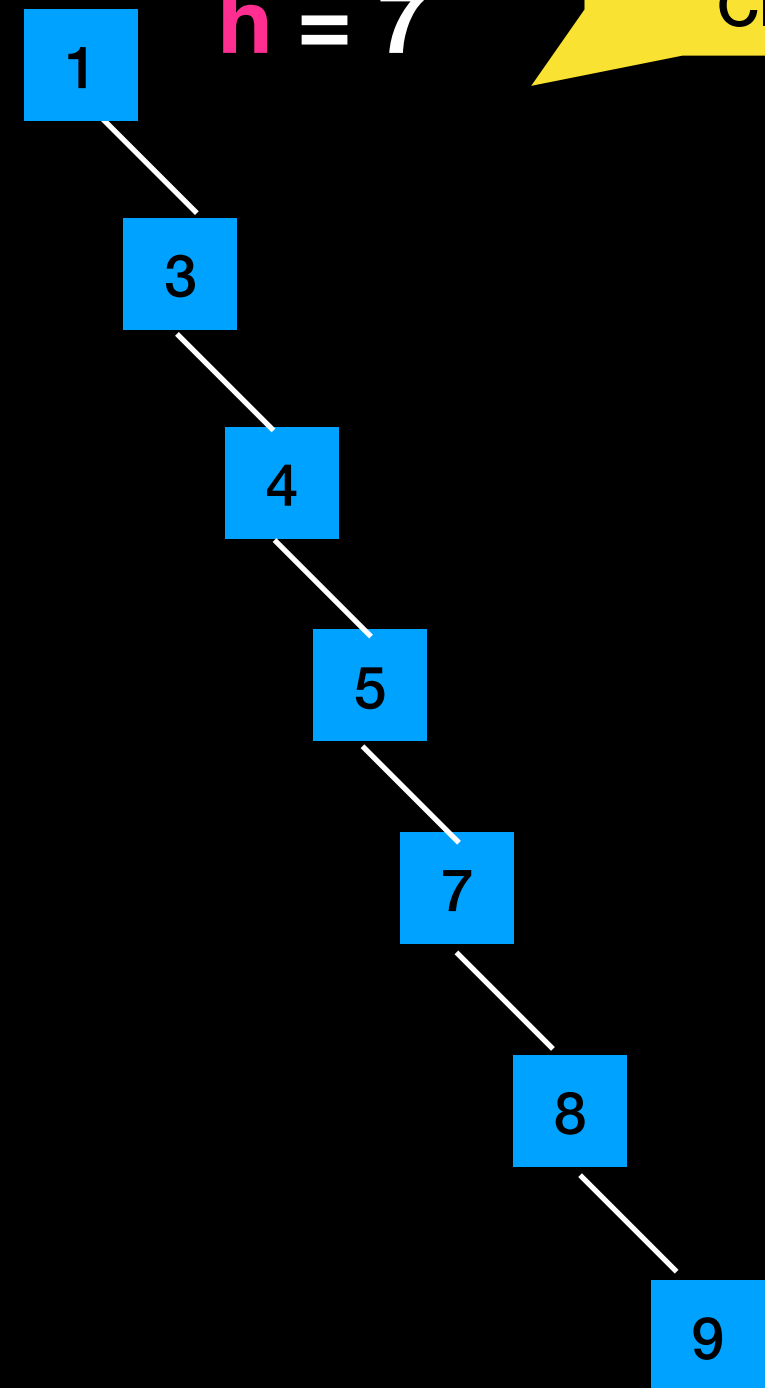


n nodes

$$\log(n+1) \leq h \leq n$$

$h = 7$

Chain



Operation	In Full Tree	Worst-case
Search	$O(\log n)$	$O(h)$
Add	$O(\log n)$	$O(h)$
Remove	$O(\log n)$	$O(h)$
Traverse	$O(n)$	$O(n)$

BST Operations

```

#ifndef BST_H_
#define BST_H_

template<class T>
class BST
{
public:
    BST(); // constructor
    BST(const BST<T>& tree); // copy constructor
    ~BST(); // destructor
    bool isEmpty() const;
    size_t getHeight() const;
    size_t getNumberOfNodes() const;
    void add(const T& new_item);
    void remove(const T& new_item);
    T find(const T& item) const;
    void clear();

    void preorderTraverse(Visitor<T>& visit) const;
    void inorderTraverse(Visitor<T>& visit) const;
    void postorderTraverse(Visitor<T>& visit) const;

    BST& operator= (const BST<T>& rhs);

private: // implementation details here
}; // end BST

#include "BST.cpp"
#endif // BST_H_

```

Looks a lot like a
BinaryTree

Might you inherit
from it?

What would you
override?

This is an abstract class from which
we can derive desired behavior
keeping the traversal general

```

#ifndef BST_H_
#define BST_H_

template<class T>
class BST
{
public:
    BST(); // constructor
    BST(const BST<T>& tree); // copy constructor
    ~BST(); // destructor
    bool isEmpty() const;
    size_t getHeight() const;
    size_t getNumberOfNodes() const;
    void add(const T& new_item);
    void remove(const T& new_item);
    T find(const T& item) const;
    void clear();

    void preorderTraverse(Visitor<T>& visit) const;
    void inorderTraverse(Visitor<T>& visit) const;
    void postorderTraverse(Visitor<T>& visit) const;

    BST& operator= (const BST<T>& rhs);

private: // implementation details here
}; // end BST

#include "BST.cpp"
#endif // BST_H_

```

Looks a lot like a
BinaryTree

Might you inherit
from it?

What would you
override?

This is an abstract class from which
we can derive desired behavior
keeping the traversal general

BST Implementation

Tiziana Ligorio

Hunter College of The City University of New York

Recap

```
#ifndef BST_H_
#define BST_H_
#include <memory>

template<class T>
class BST
{
public:
    BST(); // constructor
    BST(const BST<T>& tree); // copy constructor
    ~BST(); // destructor
    bool isEmpty() const;
    size_t getHeight() const;
    size_t getNumberOfNodes() const;
    void add(const T& new_item);
    void remove(const T& new_item);
    T find(const T& item) const;
    void clear();

    void preorderTraverse(Visitor<T>& visit) const;
    void inorderTraverse(Visitor<T>& visit) const;
    void postorderTraverse(Visitor<T>& visit) const;

    BST& operator= (const BST<T>& rhs);

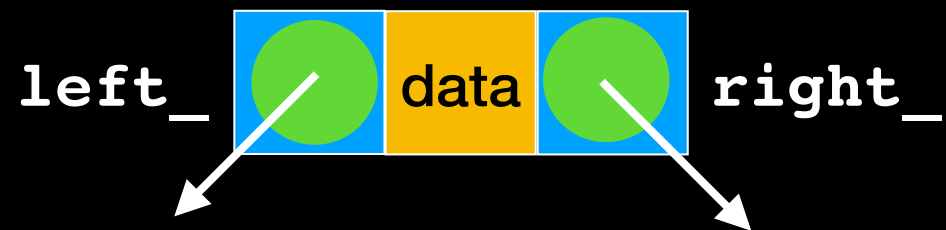
private:
    std::shared_ptr<BinaryNode<T>> root_ptr_;
}; // end BST

#include "BST.cpp"
#endif // BST_H_
```

Let's try something new and use `shared_ptr`:
A bit of extra syntax at declaration but then you use them as regular pointers with less cleaning up

To implement this as a linked structure what do we need to change in our previous implementation of nodes???

BinaryNode




```
#ifndef BinaryNode_H_
#define BinaryNode_H_
#include <memory>
```

Standard library for
shared_ptr

```
template<class T>
class BinaryNode
{
public:
    BinaryNode();
    BinaryNode(const T& an_item, std::shared_ptr<BinaryNode<T>> left,
               std::shared_ptr<BinaryNode<T>> right);
    void setItem(const T& an_item);
    T getItem() const;

    bool isLeaf() const;

    auto getLeftChildPtr() const;
    auto getRightChildPtr() const;

    void setLeftChildPtr(std::shared_ptr<BinaryNode<T>> left_ptr);
    void setRightChildPtr(std::shared_ptr<BinaryNode<T>> right_ptr);

private:
    T item_; // Data portion
    std::shared_ptr<BinaryNode<T>> left_; // Pointer to left child
    std::shared_ptr<BinaryNode<T>> right_; // Pointer to right child
}; // end BST

#include "BinaryNode.cpp"
#endif // BinaryNode_H_
```

```

template<class T>
BinaryNode(const T& an_item, std::shared_ptr<BinaryNode<T>> left,
           std::shared_ptr<BinaryNode<T>> right): item_{an_item},
           left_{left}, right_{right}{ } // end constructor

```

```

template<class T>
bool BinaryNode<T>::isLeaf() const
{
    return ((left_ == nullptr) && (right_ == nullptr));
} // end isLeaf

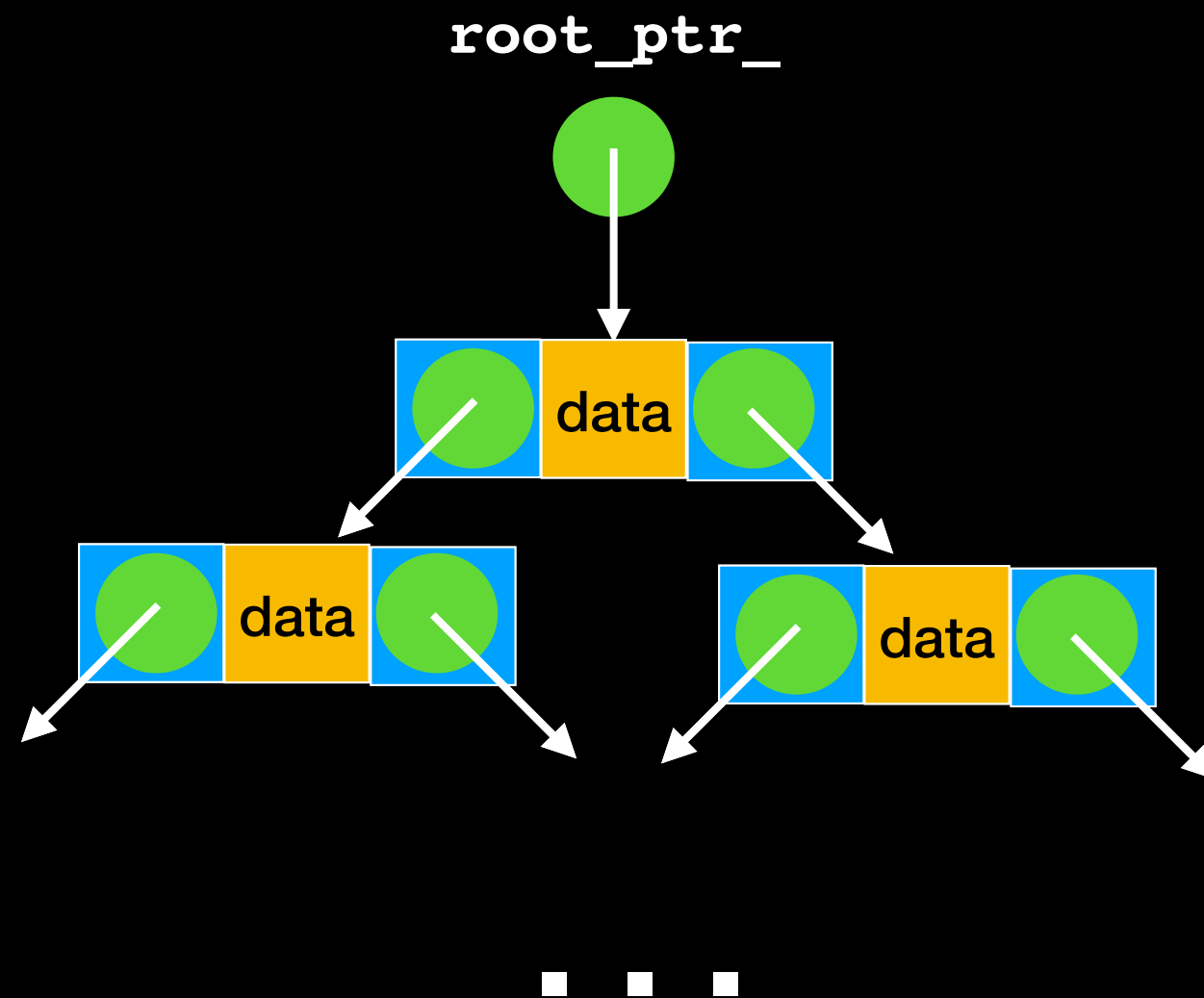
```

```

template<class T>
void BinaryNode<T>::setLeftChildPtr(std::shared_ptr<BinaryNode<T>> left_ptr)
{
    left_ = left_ptr;
} // end setLeftChildPtr

```

BST



```

#ifndef BST_H_
#define BST_H_
#include <memory>

template<class T>
class BST
{
public:
    BST(); // constructor
    BST(const BST<T>& tree); // copy constructor
    ~BST(); // destructor
    bool isEmpty() const;
    size_t getHeight() const;
    size_t getNumberOfNodes() const;
    void add(const T& new_item);
    void remove(const T& new_item);
    T find(const T& item) const;
    void clear();

    void preorderTraverse(Visitor<T>& visit) const;
    void inorderTraverse(Visitor<T>& visit) const;
    void postorderTraverse(Visitor<T>& visit) const;

    BST& operator= (const BST<T>& rhs);

private:
    std::shared_ptr<BinaryNode<T>> root_ptr_;
}; // end BST

#include "BST.cpp"
#endif // BST_H_

```

In the spirit of safe programming, our interface is generic and not tied to implementation. Many of these methods will use helper functions, which should be private (or protected if you envision inheritance). I do not include them here in the interface for lack of space.

Copy Constructor

root_ptr of
this object

root_ptr of tree: the
object I'm going to copy

```
template<class T>
BST<T>::BST(const BST<T>& tree)
{
    root_ptr_ = copyTree(tree.root_ptr_); // Call helper function
} // end copy constructor
```

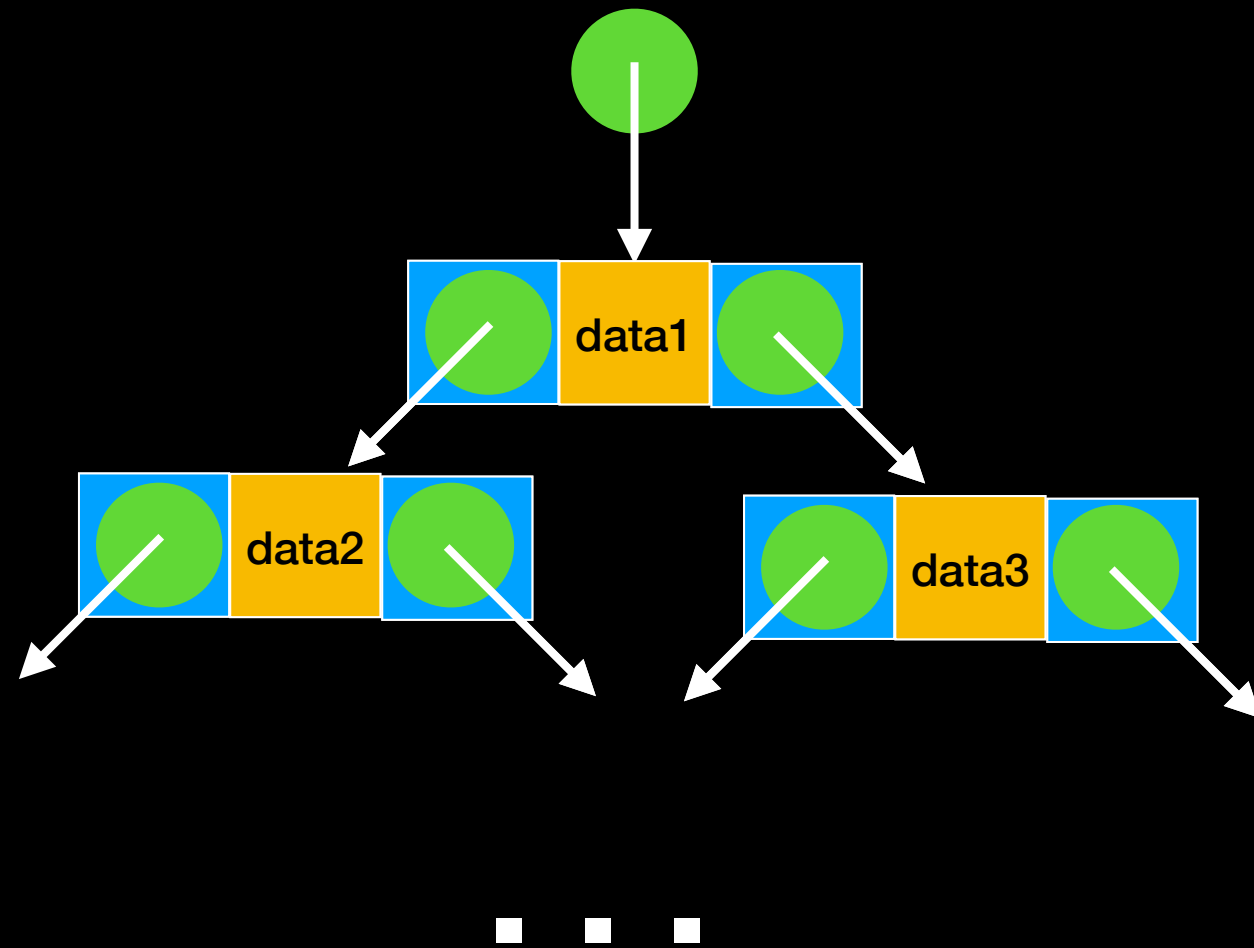
Safe programming: the public
method does not take pointer
parameter.

Only protected/private methods
have access to pointers and
may modify tree structure

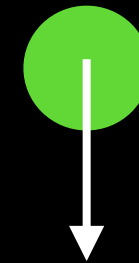
I can use the . operator to access a
private member variable because it
is s within the class definition.

copyTree(old_tree_root_ptr)

old_tree_root_ptr

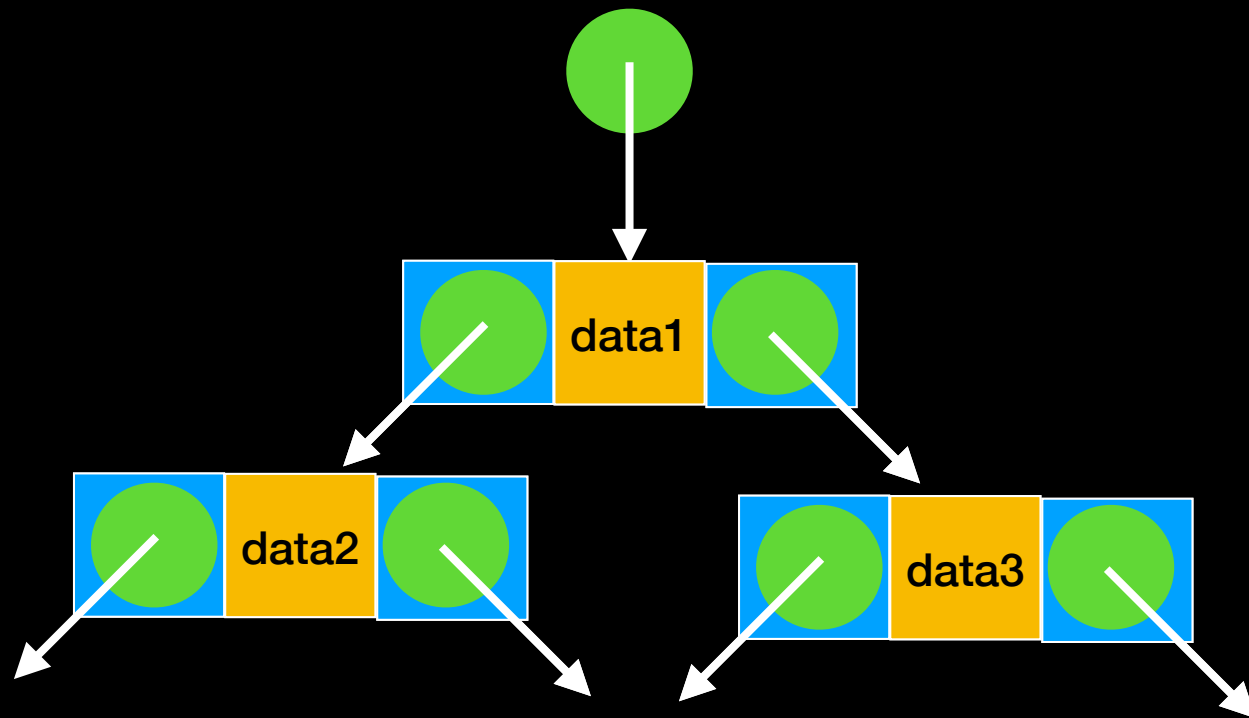


new_tree_ptr

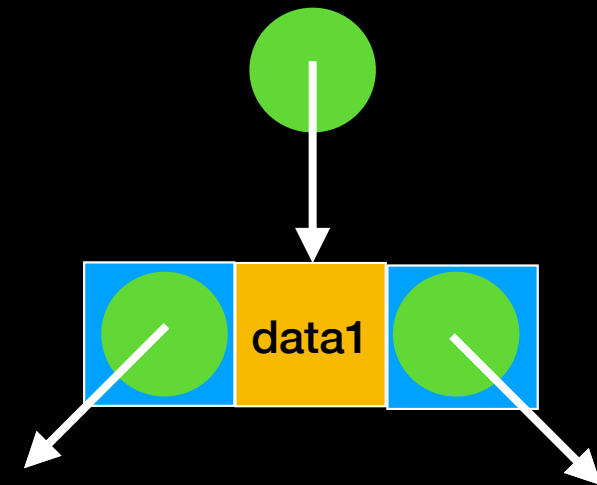


copyTree(`old_tree_root_ptr`)

`old_tree_root_ptr`

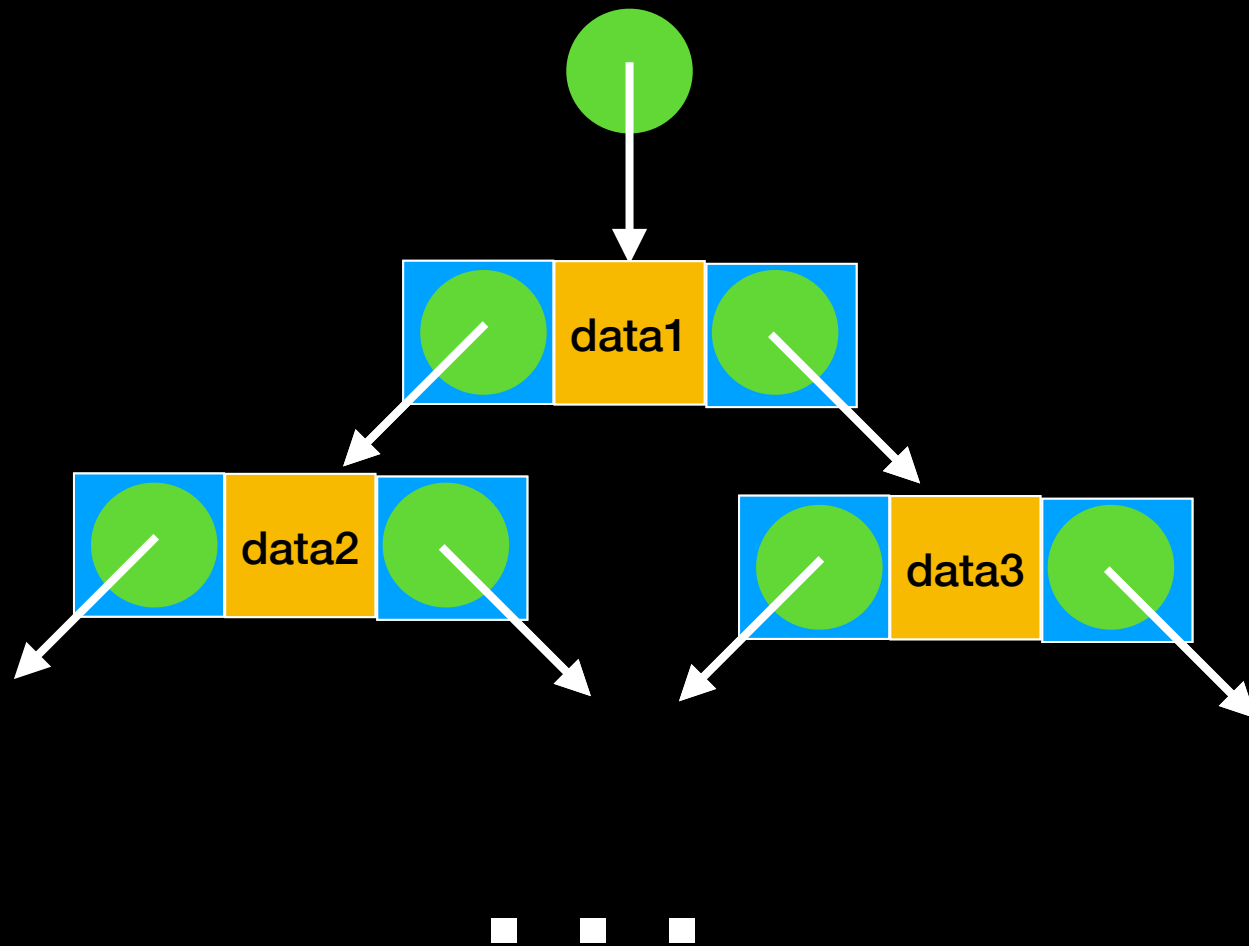


`new_tree_ptr`

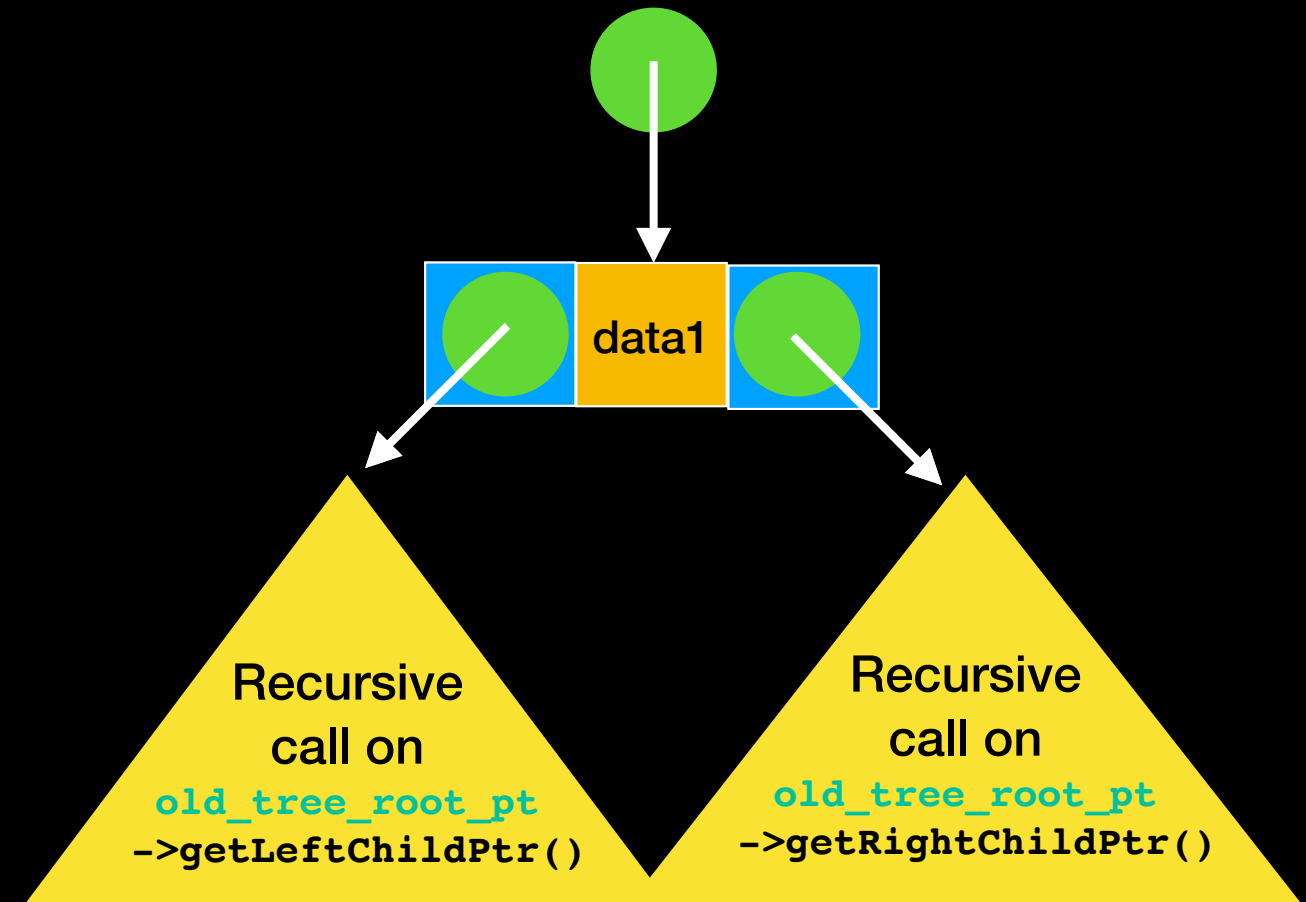


copyTree(`old_tree_root_ptr`)

`old_tree_root_ptr`



`new_tree_ptr`



Copy Constructor Helper Function

Returning
shared_ptr,
cleaner to use
auto return type:
-std=c++14

```
template<class T>
auto BST<T>::copyTree(const std::shared_ptr<BinaryNode<T>> old_tree_root_ptr) const
{
    std::shared_ptr<BinaryNode<T>> new_tree_ptr;

    // Copy tree nodes during a preorder traversal
    if (old_tree_root_ptr != nullptr)
    {
        // Copy node
        new_tree_ptr = std::make_shared<BinaryNode<T>>(old_tree_root_ptr
                                                         ->getItem(), nullptr, nullptr);
        new_tree_ptr->setLeftChildPtr(copyTree(old_tree_root_ptr->getLeftChildPtr()));
        new_tree_ptr->setRightChildPtr(copyTree(old_tree_root_ptr
                                                 ->getRightChildPtr()));
    } // end if

    return new_tree_ptr;
} // end copyTree
```

Recall: this is the syntax
for allocating a “new”
object with shared_ptr
pointing to it

Recursive Calls:
**Don't want to tie interface
to recursive implementation:
Use helper function**

Preorder Traversal Scheme:
copy each node as soon as it
is visited to make exact copy

Move Constructor

How would you implement it?

Move the root!

Move Constructor

rvalue
reference

```
template<class T>
BST<T>::BST(const BST<T>&& tree)
{
    root_ptr_ = tree.root_ptr_;
    tree.root_ptr_.reset();
} // end move constructor
```

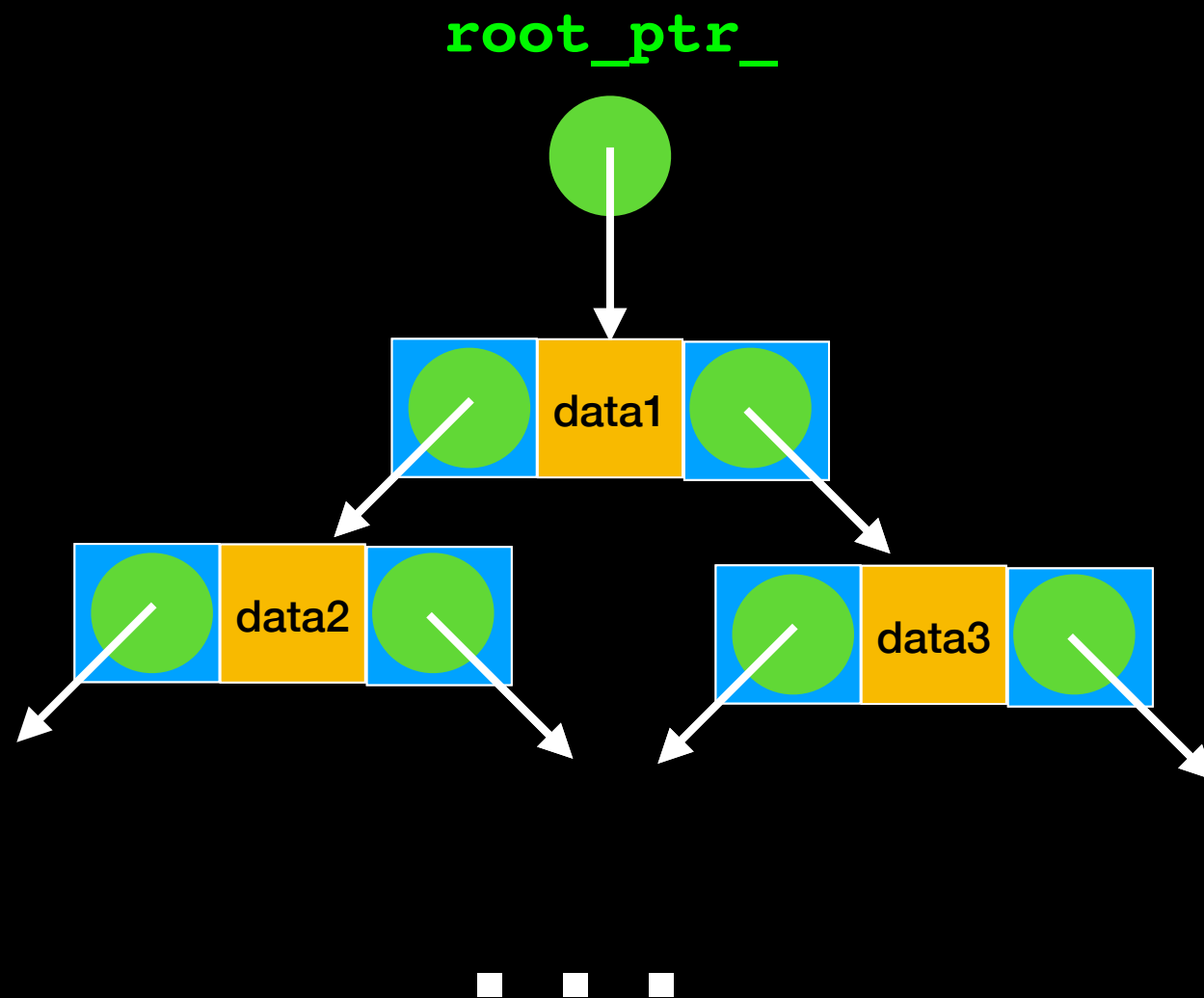
Destructor

```
template<class T>
BST<T>::~~BST( )
{
    destroyTree(root_ptr_); // Call helper function
} // end destructor
```

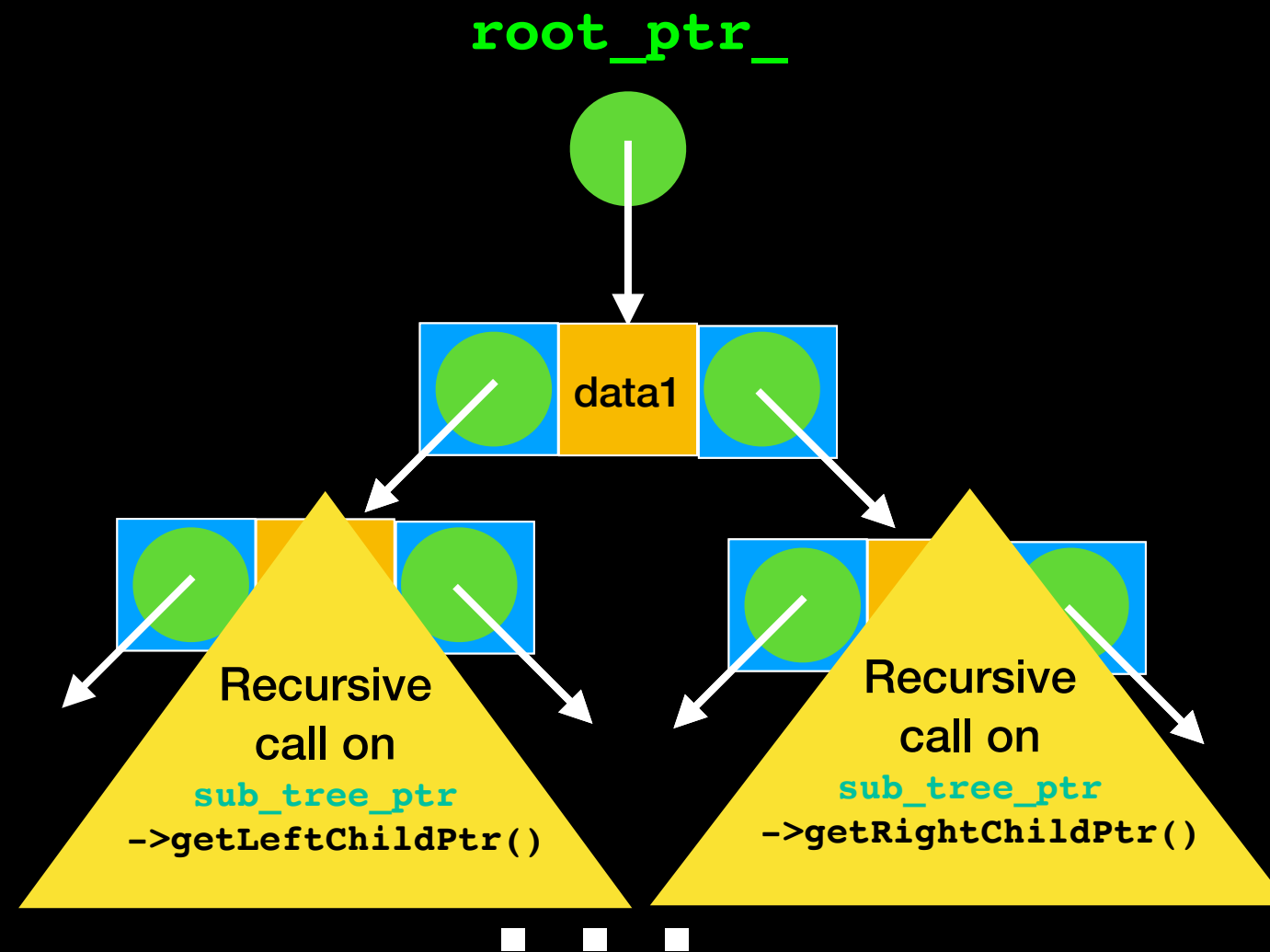
Safe programming: the public method does not take pointer parameter.

Only protected/private methods have access to pointers and may modify tree structure

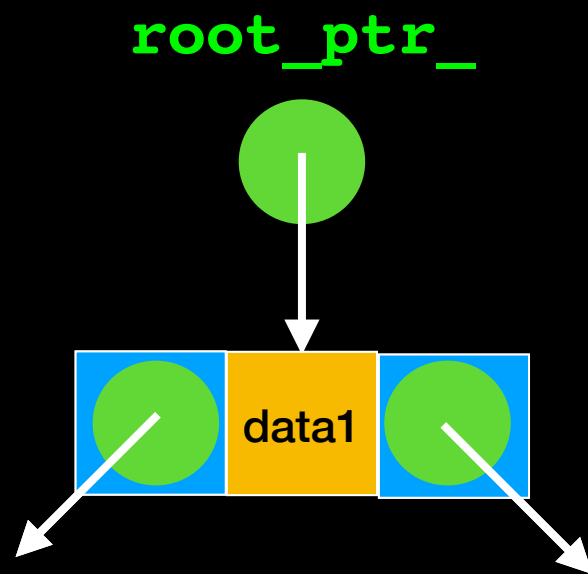
```
destroyTree(sub_tree_ptr)
```



destroyTree(**sub_tree_ptr**)



```
destroyTree(sub_tree_ptr)
```



```
root_ptr_.reset()
```



```
destroyTree(sub_tree_ptr)
```

root_ptr_



Destructor Helper Function

```
template<class T>
void BST<T>::destroyTree(std::shared_ptr<BinaryNode<T>> sub_tree_ptr)
{
    if (sub_tree_ptr != nullptr)
    {
        → destroyTree(sub_tree_ptr->getLeftChildPtr());
        → destroyTree(sub_tree_ptr->getRightChildPtr());
        sub_tree_ptr.reset(); // same as sub_tree_ptr = nullptr for smart pointers
    } // end if
} // end destroyTree
```

Notice: all we have to do is set the `shared_ptr` to `nullptr` with `reset()` and it will take care of deleting the node.

PostOrder Traversal Scheme:
Delete node only after deleting both of its subtrees

clear

```
template<class T>
void BST<T>::clear()
{
    destroyTree(root_ptr_); // Call helper method
} // end clear
```

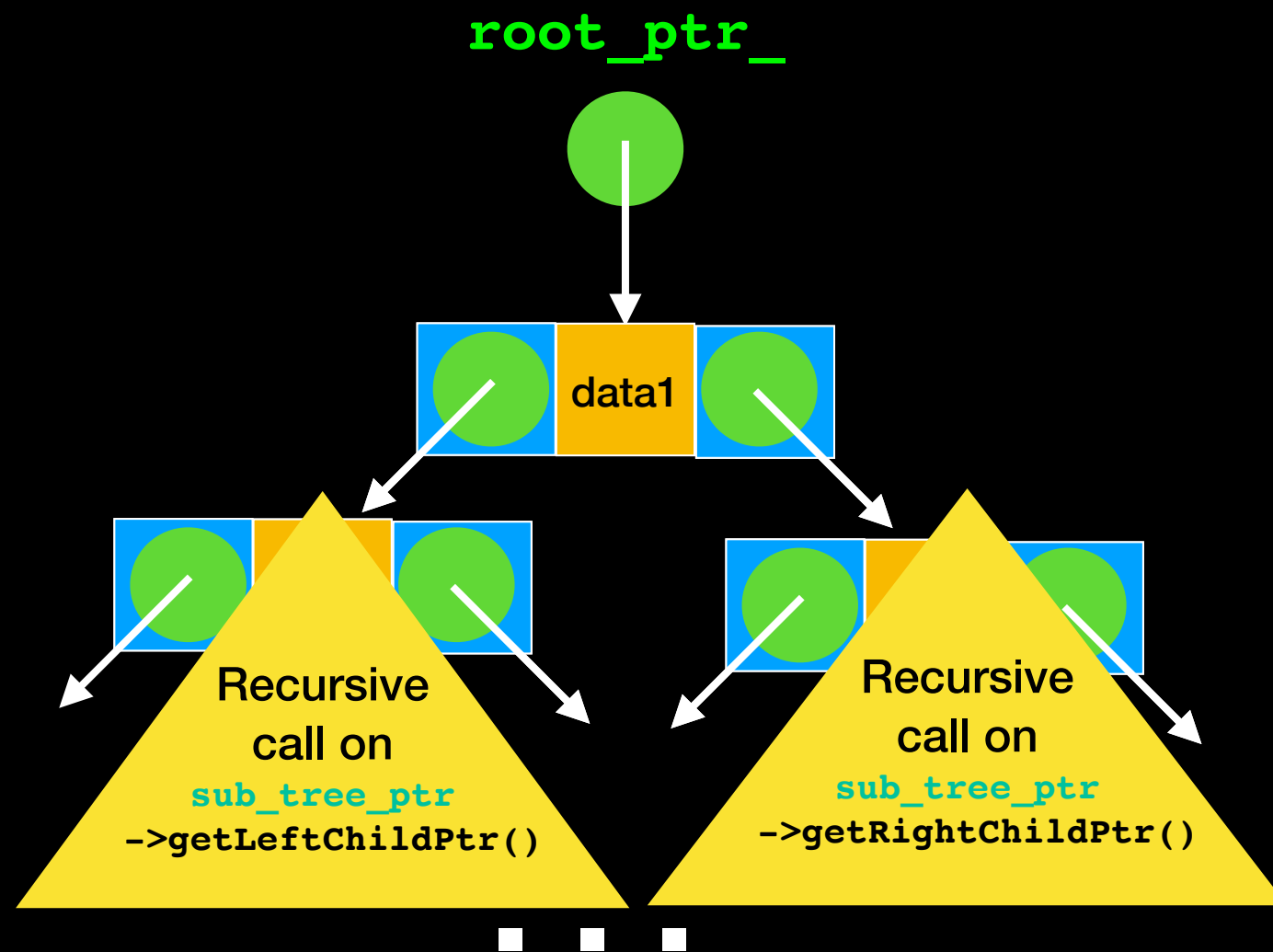
Safe programming: the public method does not take pointer parameter.
Only protected/private methods have access to pointers and may modify tree structure

getHeight

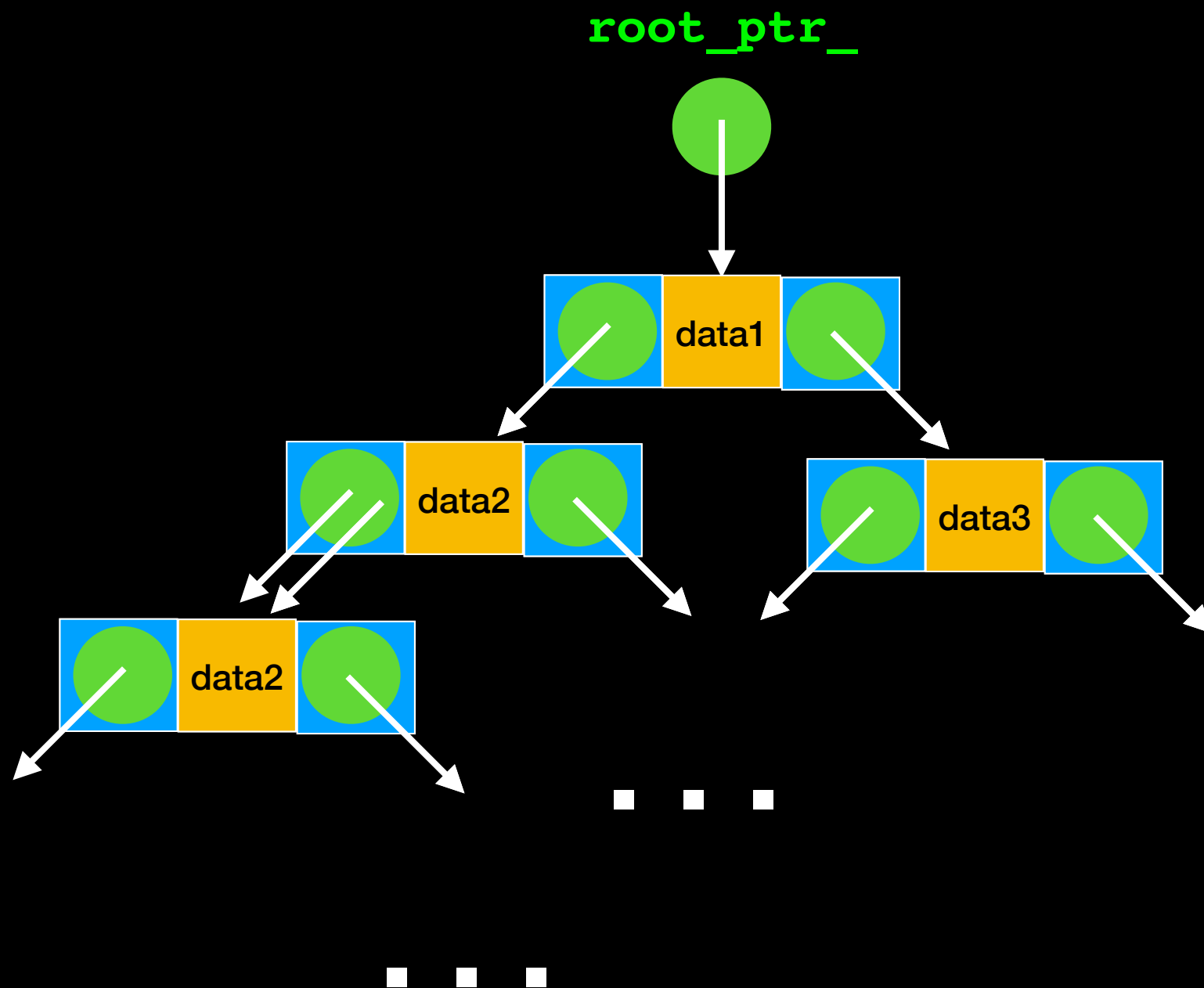
```
template<class T>
int BST<T>::getHeight() const
{
    return getHeightHelper(root_ptr_);
} // end getHeight
```

Safe programming: the public method does not take pointer parameter.
Only protected/private methods have access to pointers and may modify tree structure

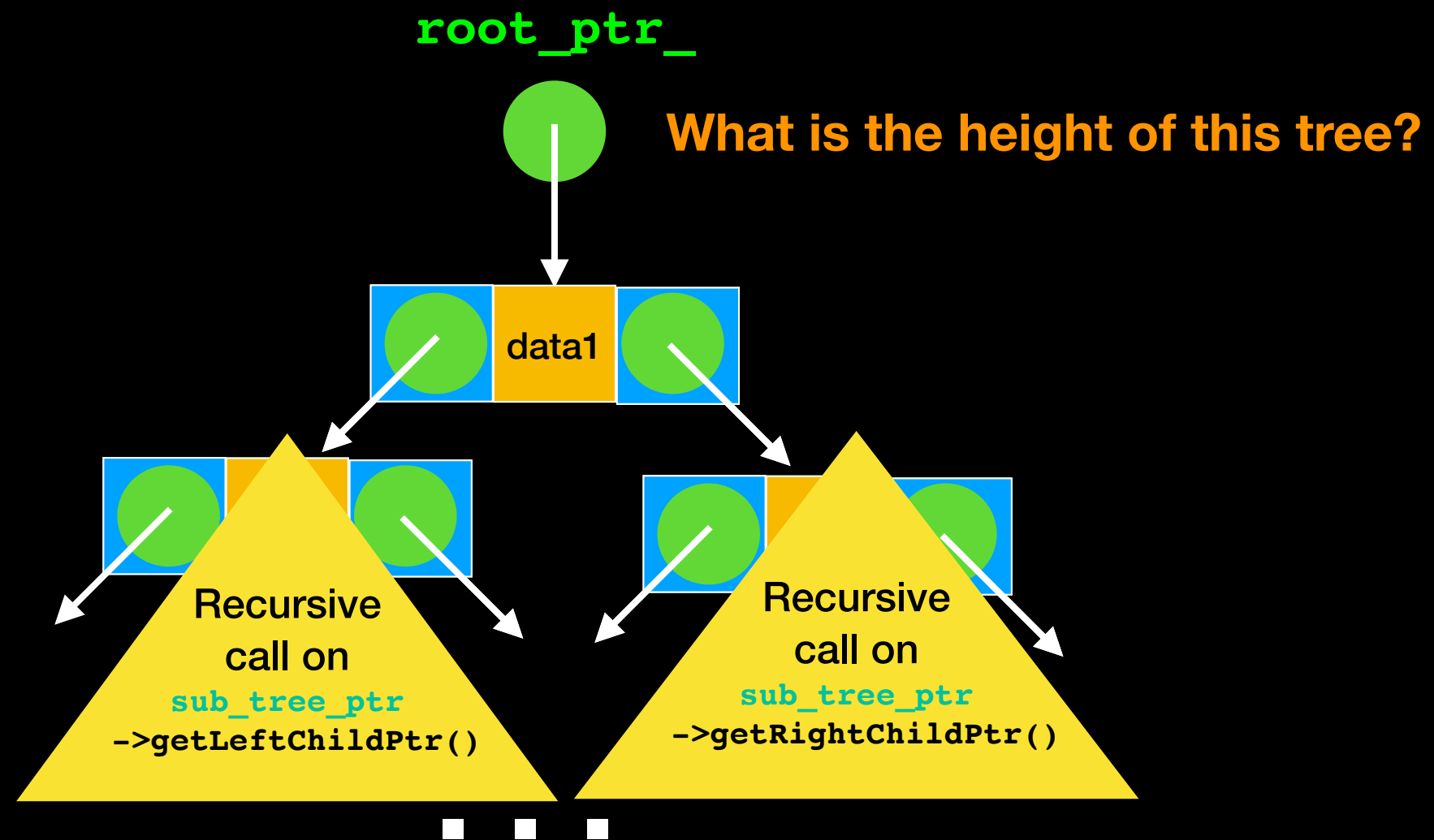
getHeightHelper(**sub_tree_ptr**)



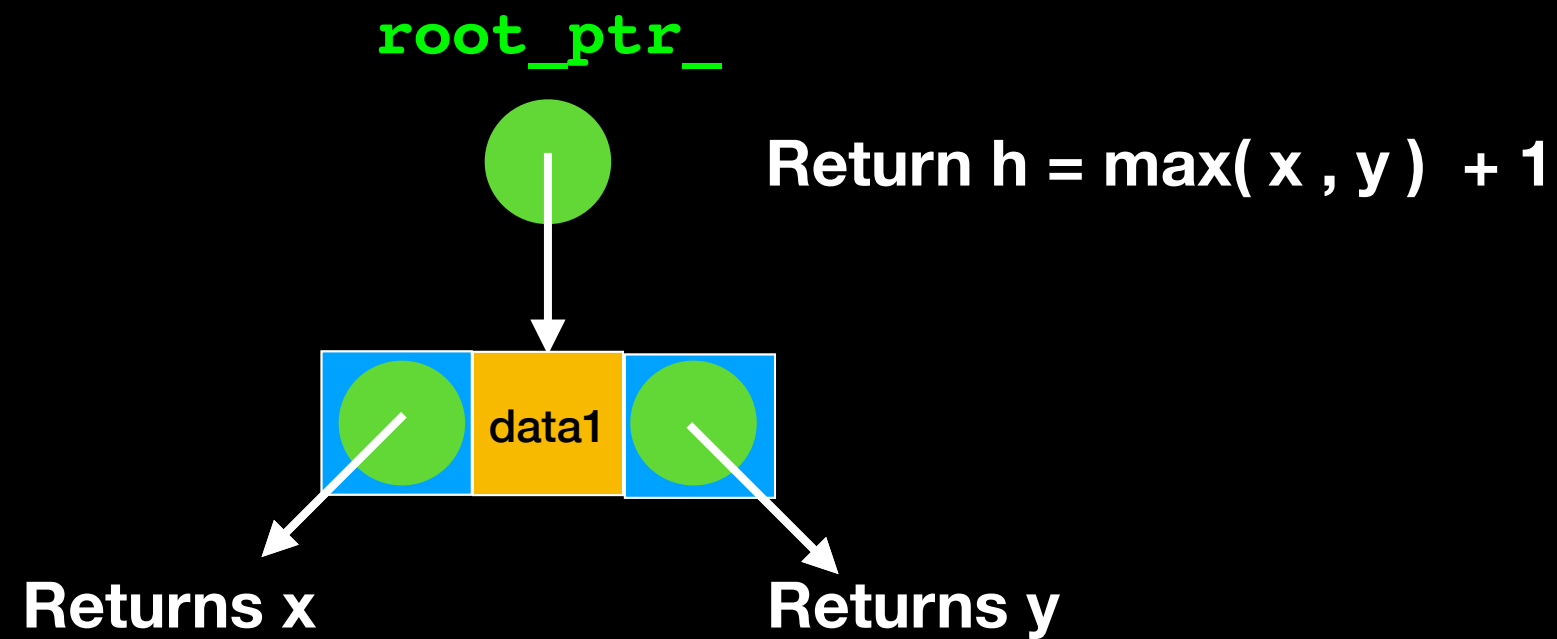
getHeightHelper(sub_tree_ptr)



getHeightHelper(**sub_tree_ptr**)

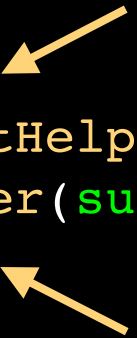


getHeightHelper(**sub_tree_ptr**)



getHeightHelper(sub_tree_ptr)

```
template<class T>
int BST<T>::getHeightHelper(std::shared_ptr<BinaryNode<T>> sub_tree_ptr) const
{
    if (sub_tree_ptr == nullptr)
        return 0;
    else
        return 1 + std::max(getHeightHelper(sub_tree_ptr->getLeftChildPtr()),
                             getHeightHelper(sub_tree_ptr->getRightChildPtr()));
} // end getHeightHelper
```





Similarly: implement these at home!!!

```
int BST<T>::getNumberOfNodes() const  
{ //try it at home!!!!}
```

```
int BST<T>::getNumberOfNodesHelper(std::shared_ptr  
<BinaryNode<T>> sub_tree_ptr) { //try it at home!!!!}
```

add and remove

Key methods: determine order of data

Distinguish between different types of Binary Trees

Implement the BST structural property

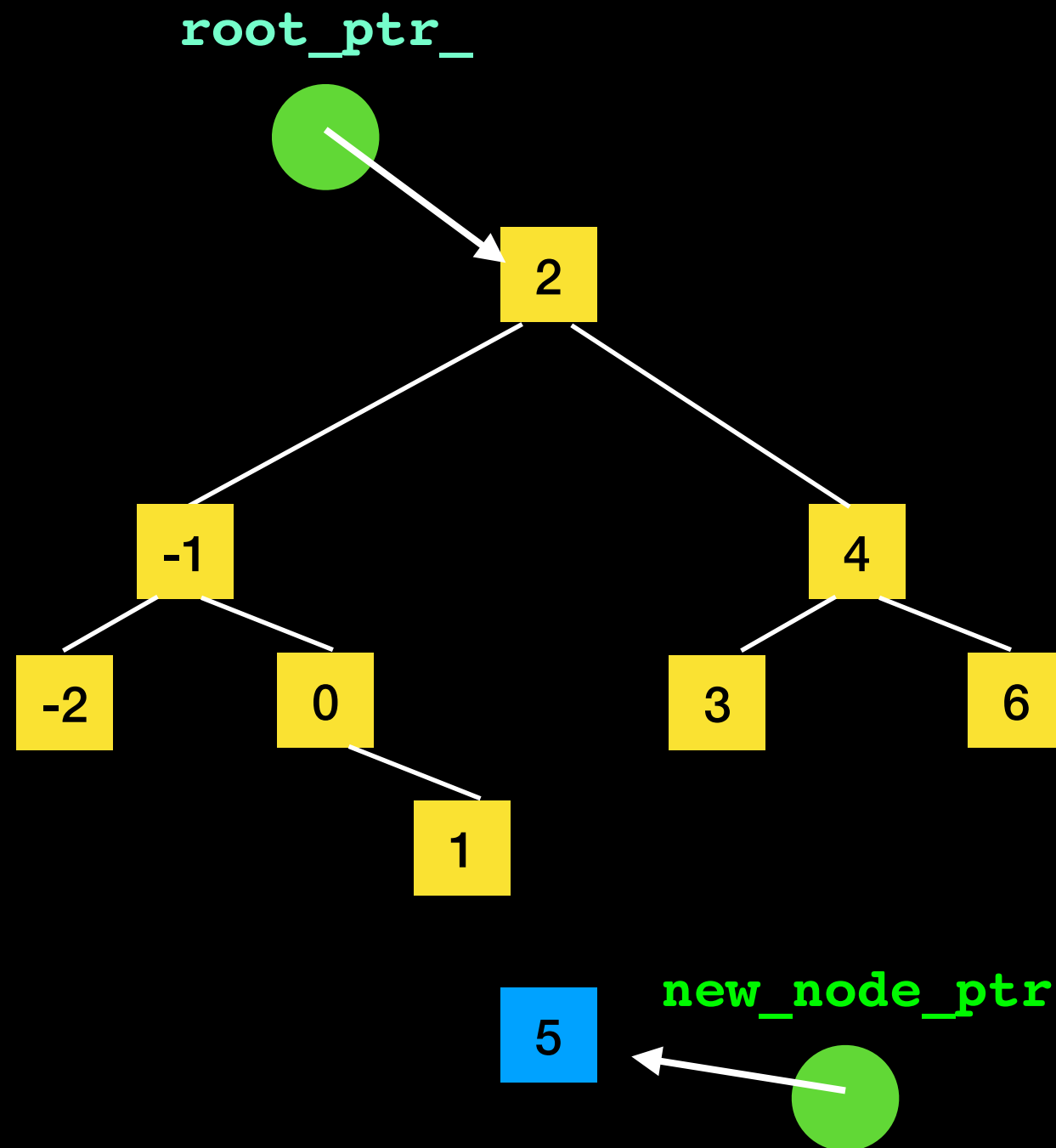
add

```
template<class T>
void BST<T>::add(const T& new_item)
{
    auto new_node_ptr =
        std::make_shared<BinaryNode<T>>(new_item);
    root_ptr_ = placeNode(root_ptr_, new_node_ptr);
} // end add
```

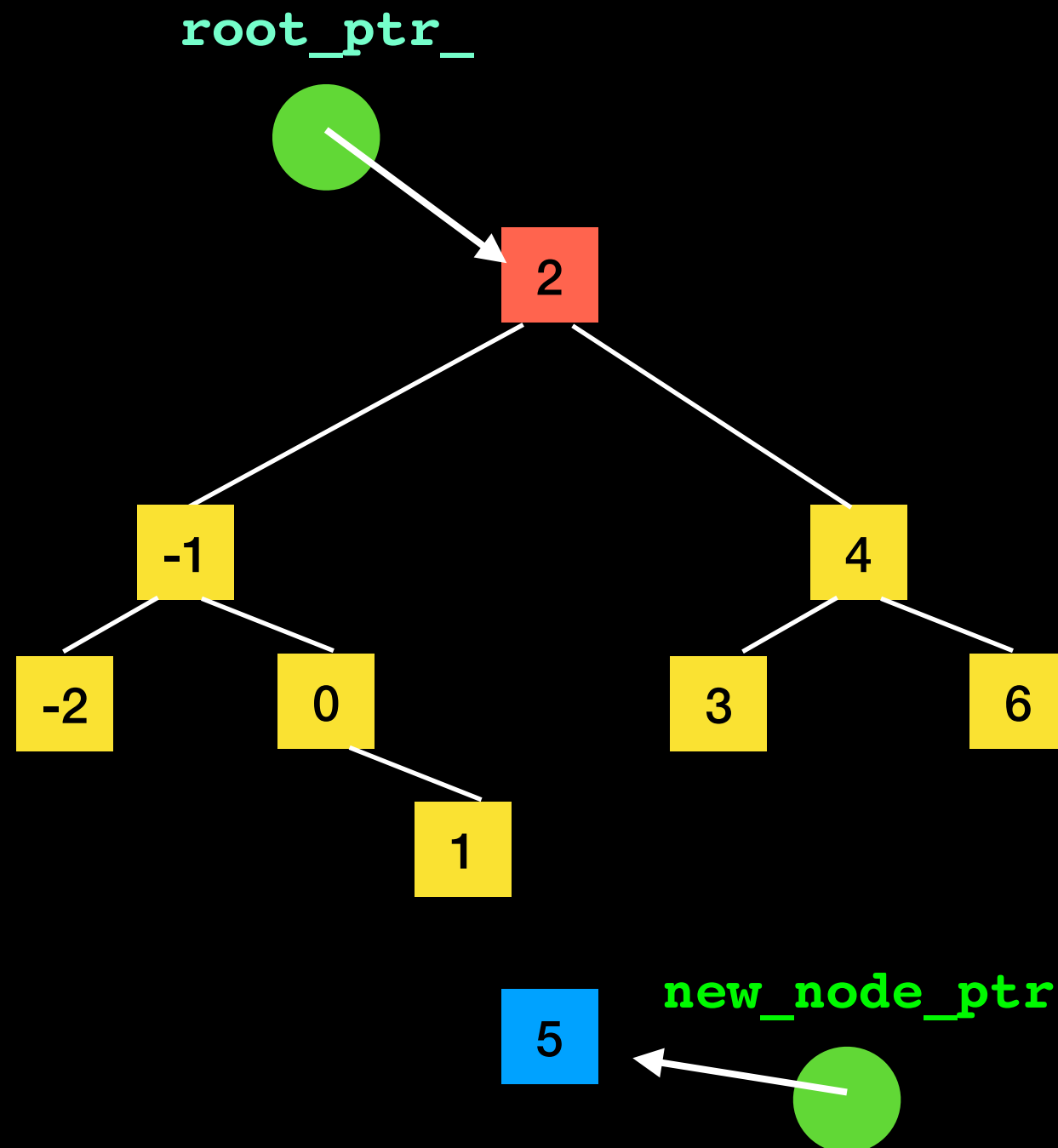
Safe programming: the public method does not take pointer parameter.

Only protected/private methods have access to pointers and may modify tree structure

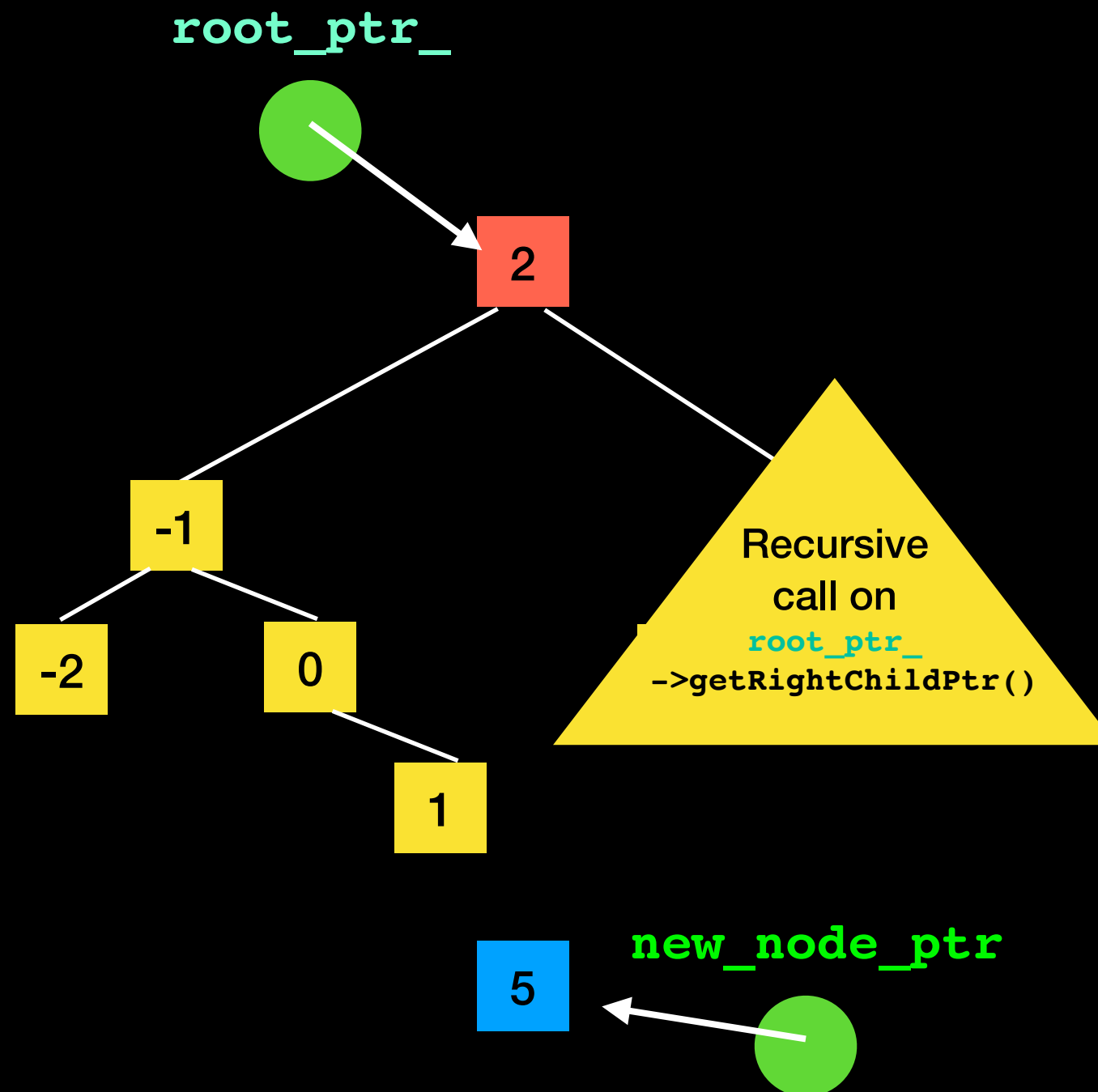
```
placeNode(root_ptr_, new_node_ptr);
```



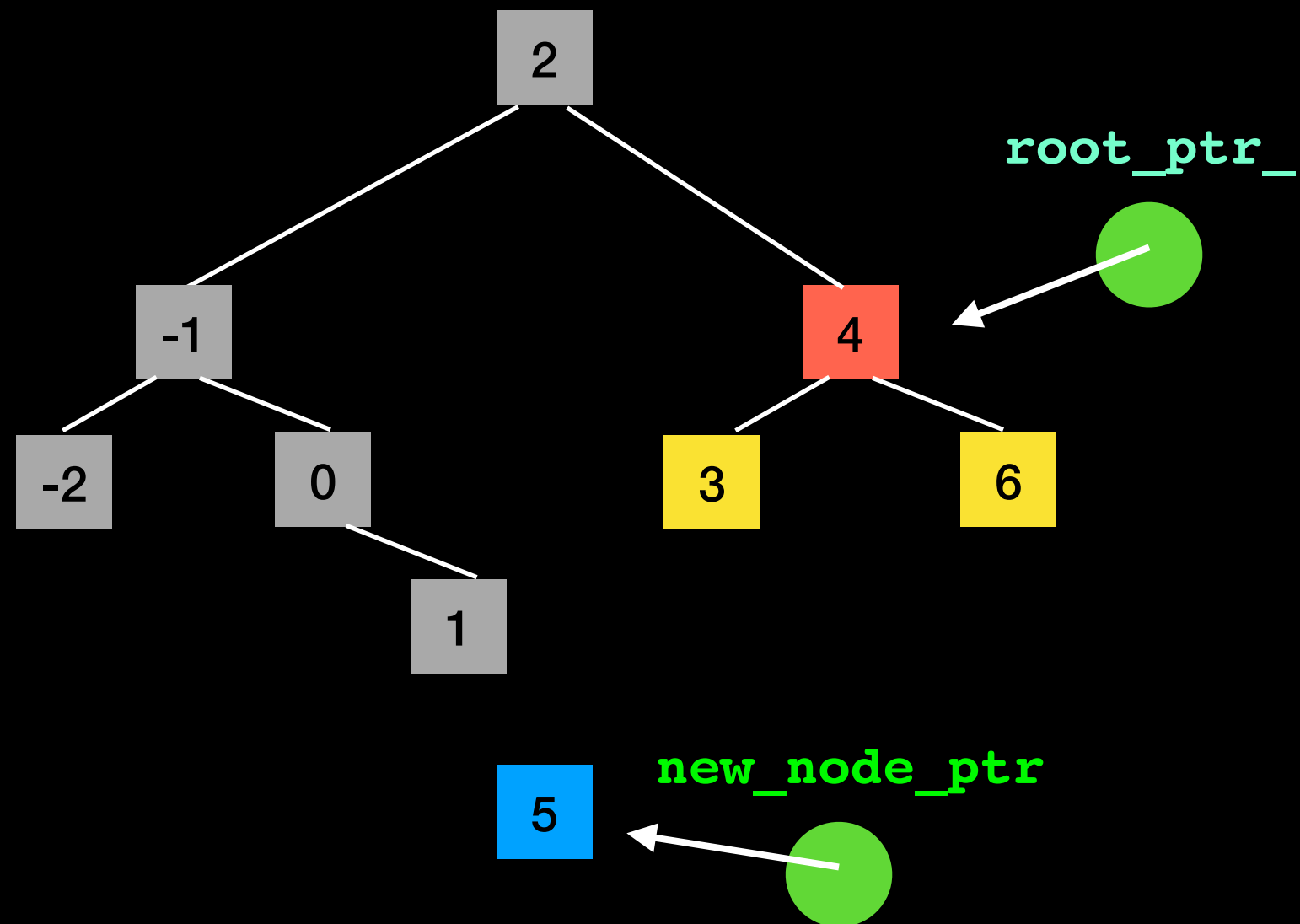
```
placeNode(root_ptr_, new_node_ptr);
```



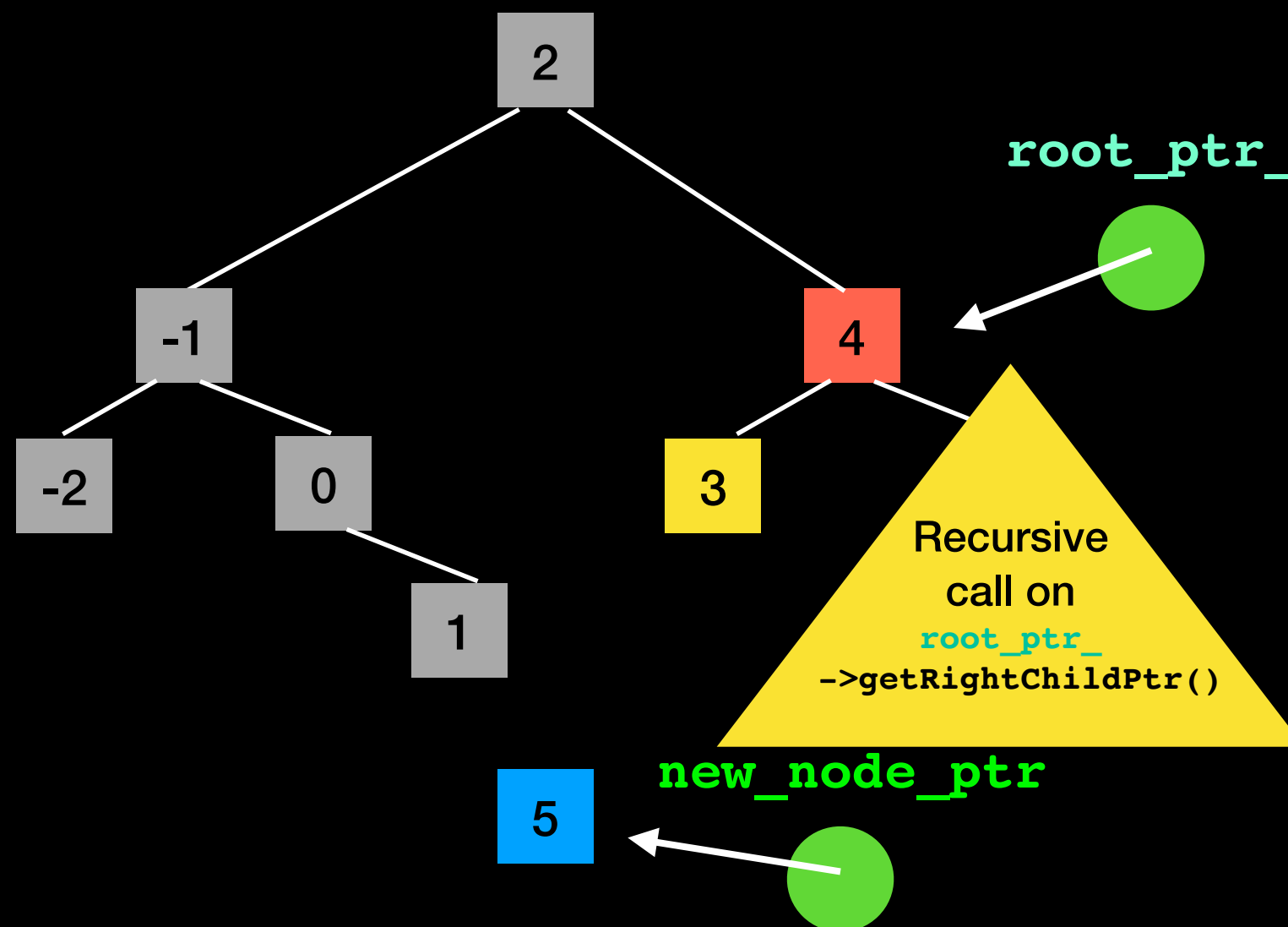
```
placeNode(root_ptr_, new_node_ptr);
```



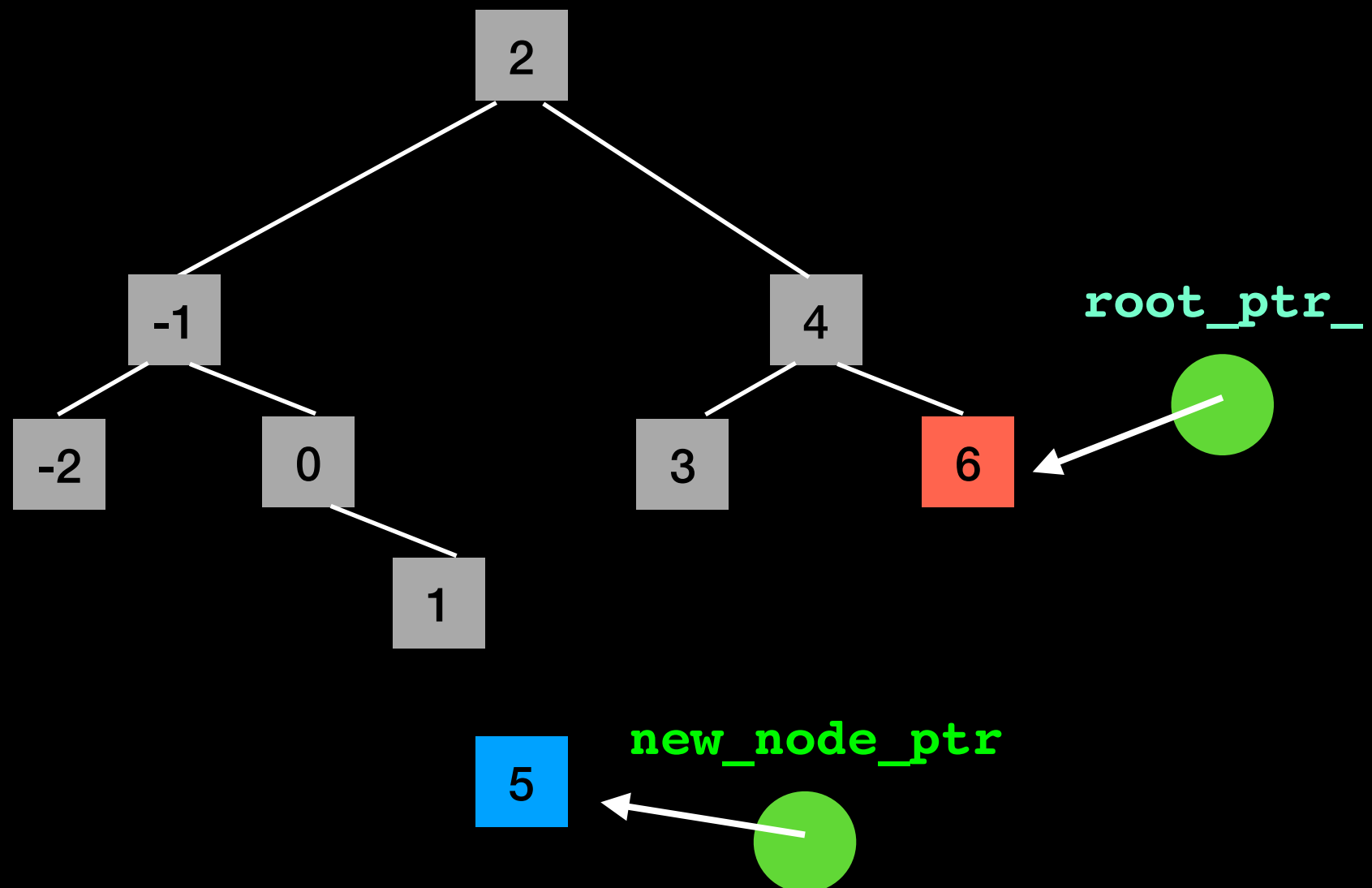
```
placeNode(root_ptr_, new_node_ptr);
```



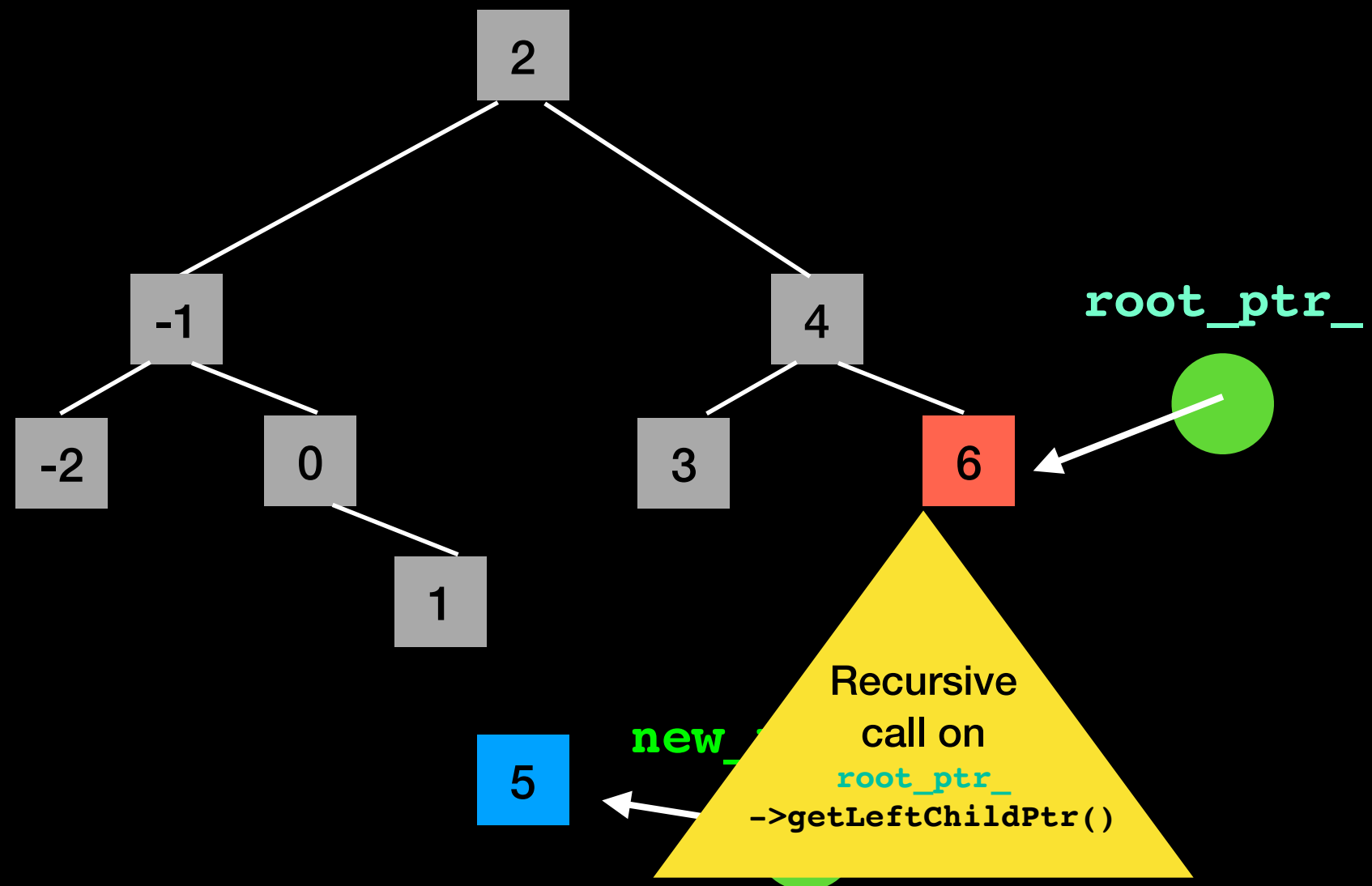

```
placeNode(root_ptr_, new_node_ptr);
```



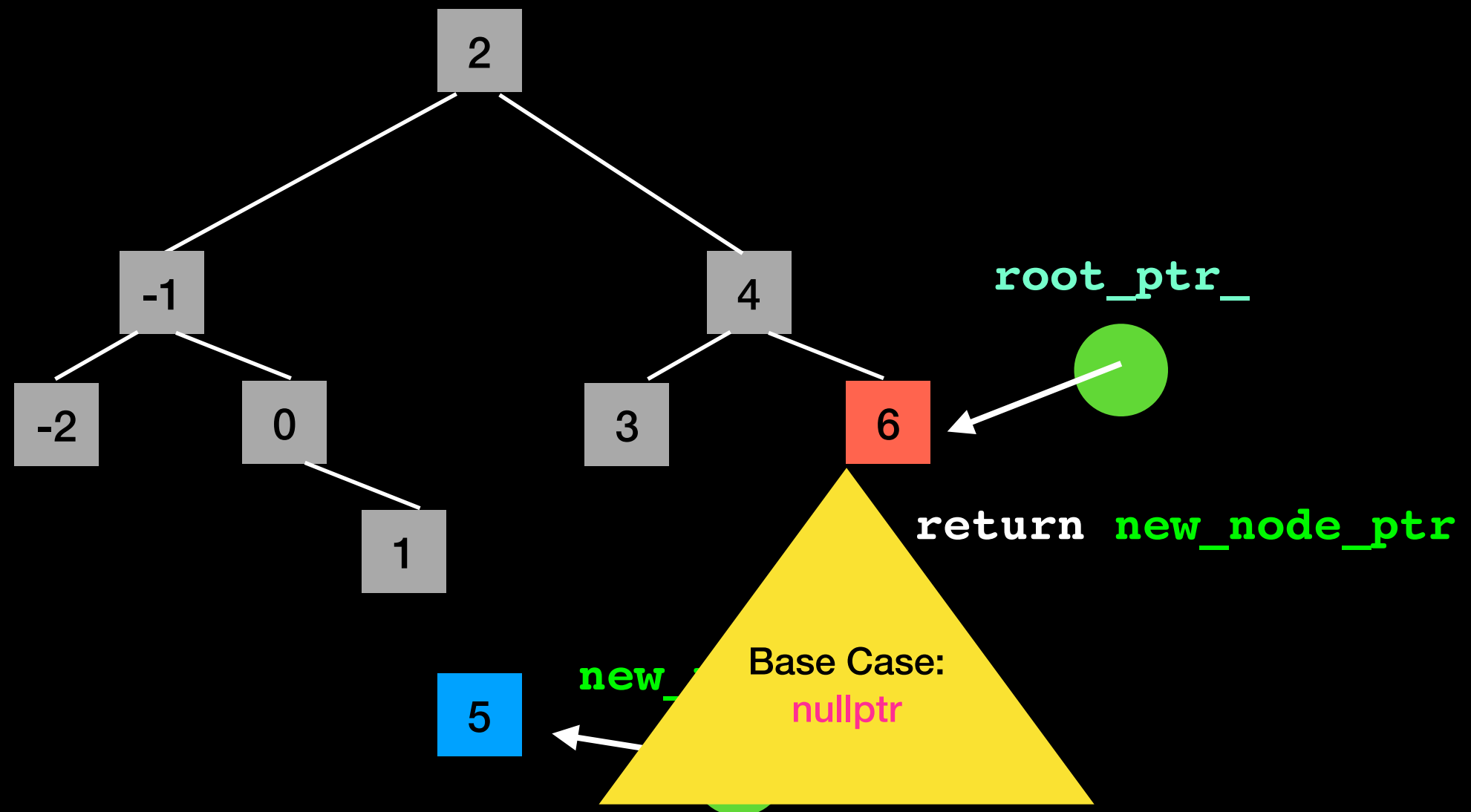

```
placeNode(root_ptr_, new_node_ptr);
```



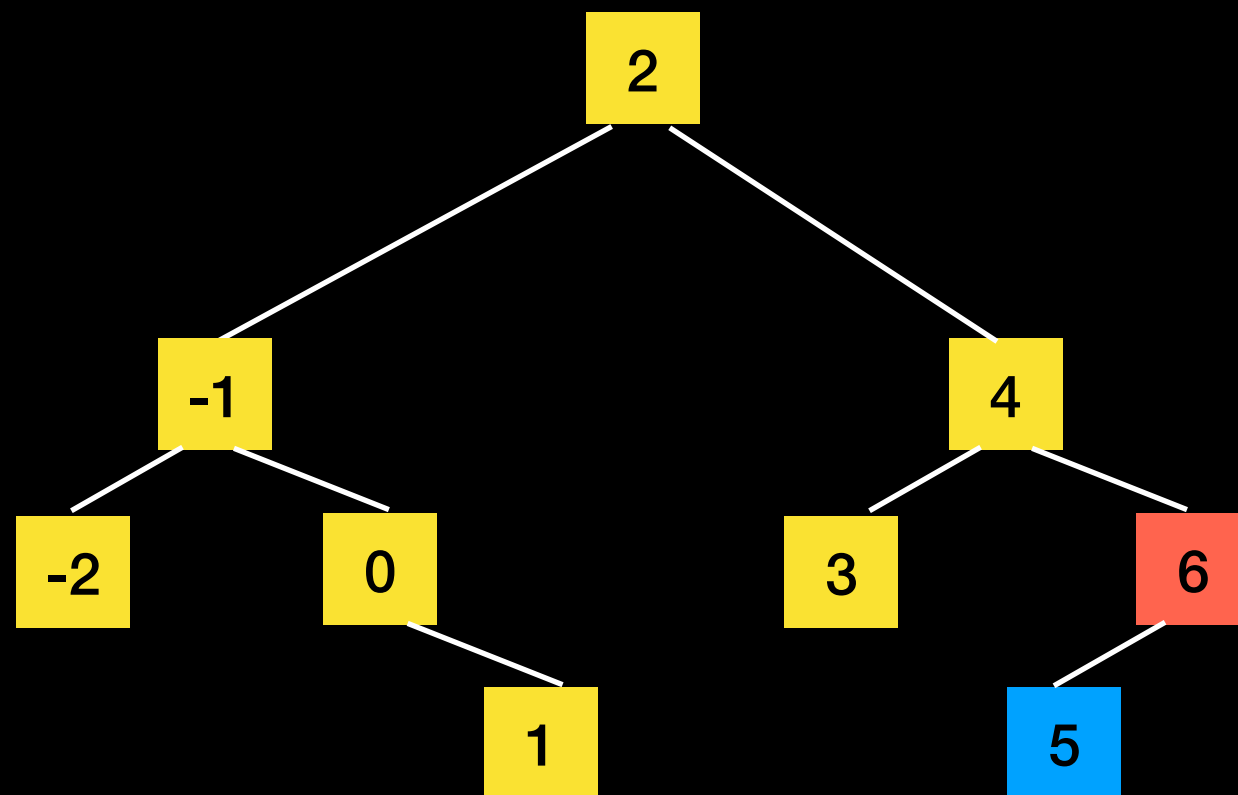
```
placeNode(root_ptr_, new_node_ptr);
```



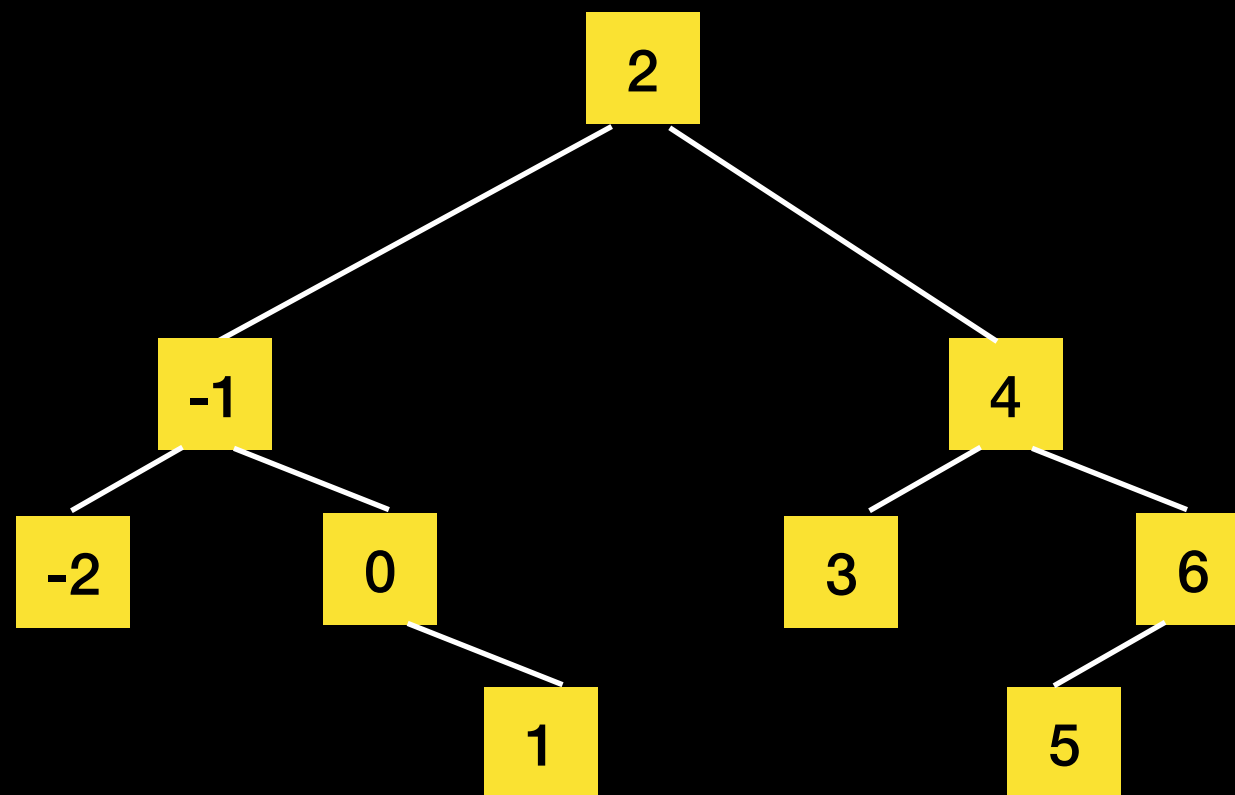
```
placeNode(root_ptr_, new_node_ptr);
```



```
placeNode(root_ptr_, new_node_ptr);
```



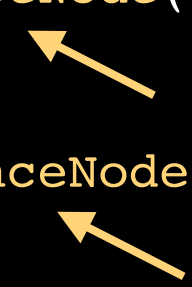
```
placeNode(root_ptr_, new_node_ptr);
```



add helper function

```
template<class T>
auto BST<T>::placeNode(std::shared_ptr<BinaryNode<T>> subtree_ptr,
                      std::shared_ptr<BinaryNode<T>> new_node_ptr)
{
    if (subtree_ptr == nullptr)
        return new_node_ptr; //base case
    else
    {
        if (subtree_ptr->getItem() > new_node_ptr->getItem())
            subtree_ptr->setLeftChildPtr(placeNode(subtree_ptr->getLeftChildPtr(),
                                                    new_node_ptr));
        else
            subtree_ptr->setRightChildPtr(placeNode(subtree_ptr->getRightChildPtr(),
                                                    new_node_ptr));

        return subtree_ptr;
    } // end if
} // end placeNode
```



remove

```
template<class T>
bool BST<T>::remove(const T& target)
{
    bool is_successful = false;
    // call may change is_successful
    root_ptr_ = removeValue(root_ptr_, target, is_successful);
    return is_successful;
} // end remove
```

Safe programming: the public method does not take pointer parameter.
Only protected/private methods have access to pointers and may modify tree structure

remove helper function

Looks for the value
to remove

```
template<class T>
auto BST<T>::removeValue(std::shared_ptr<BinaryNode<T>>
    subtree_ptr, const T target, bool& success)
{
    if (subtree_ptr == nullptr)
    {
        // Not found here
        success = false;
        return subtree_ptr;
    }
    if (subtree_ptr->getItem() == target)
    {
        // Item is in the root of this subtree
        subtree_ptr = removeNode(subtree_ptr);
        success = true;
        return subtree_ptr;
    }
}
```

target not in tree

Found target now
remove the node

remove helper function cont.ed

```
else
{
    if (subtree_ptr->getItem() > target)
    {
        // Search the left subtree
        subtree_ptr->setLeftChildPtr(removeValue(subtree_ptr
                                                ->getLeftChildPtr(), target, success));
    }
    else
    {
        // Search the right subtree
        subtree_ptr->setRightChildPtr(removeValue(subtree_ptr
                                                  ->getRightChildPtr(), target, success));
    }
    return subtree_ptr;
} // end if
} // end removeValue
```

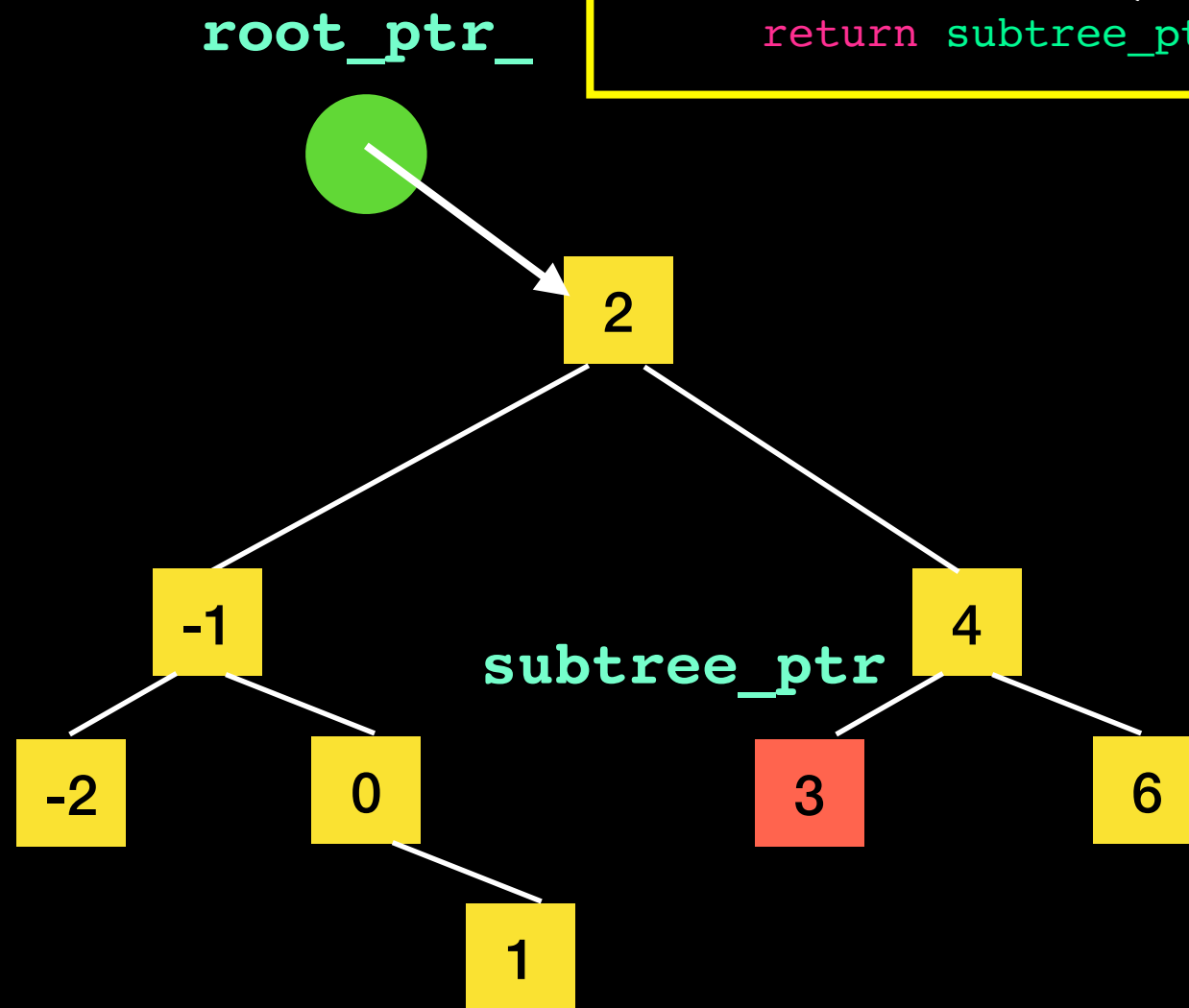
Search for target in left subtree

Search for target in right subtree

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

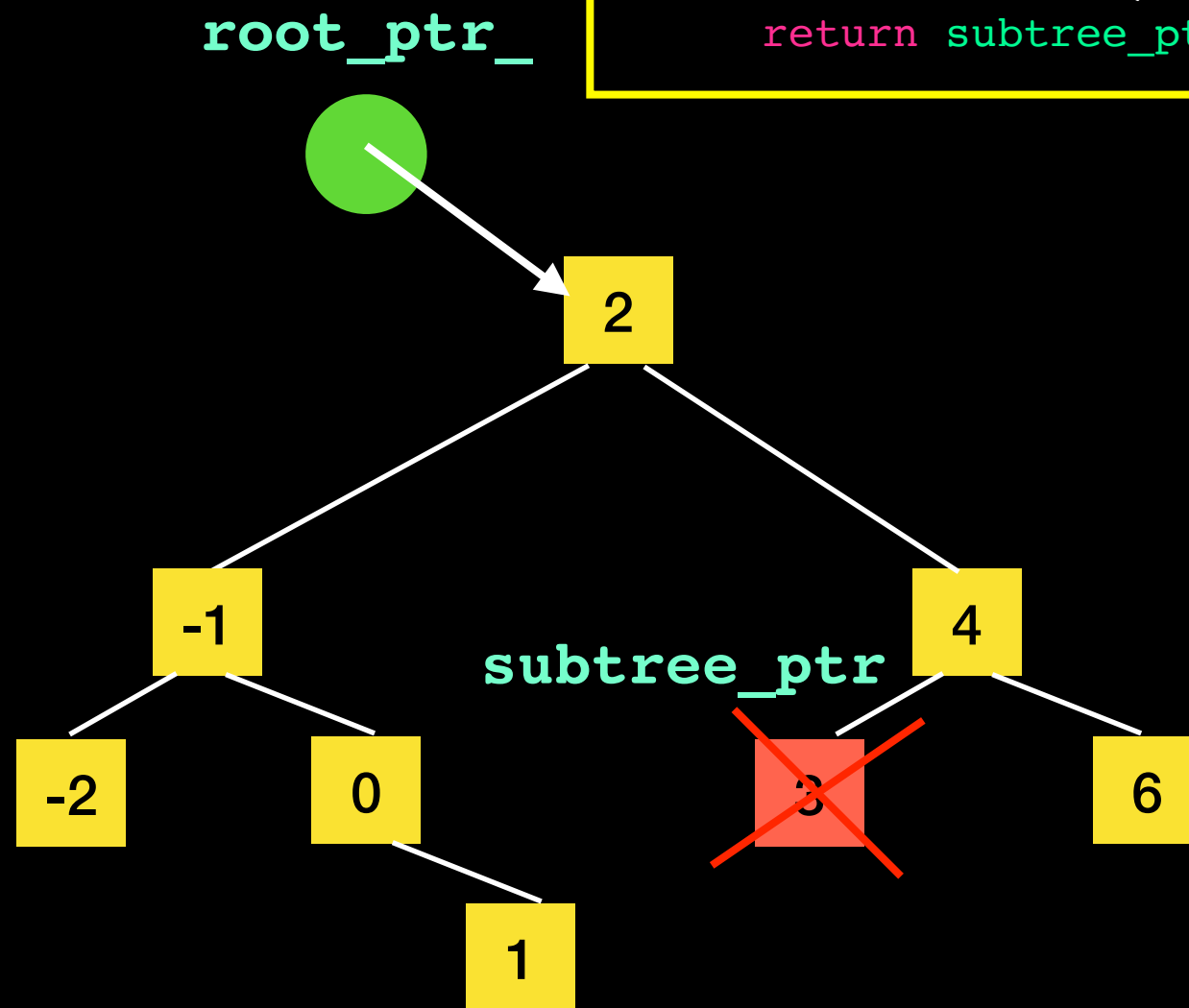
Case 1: target is a leaf



removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

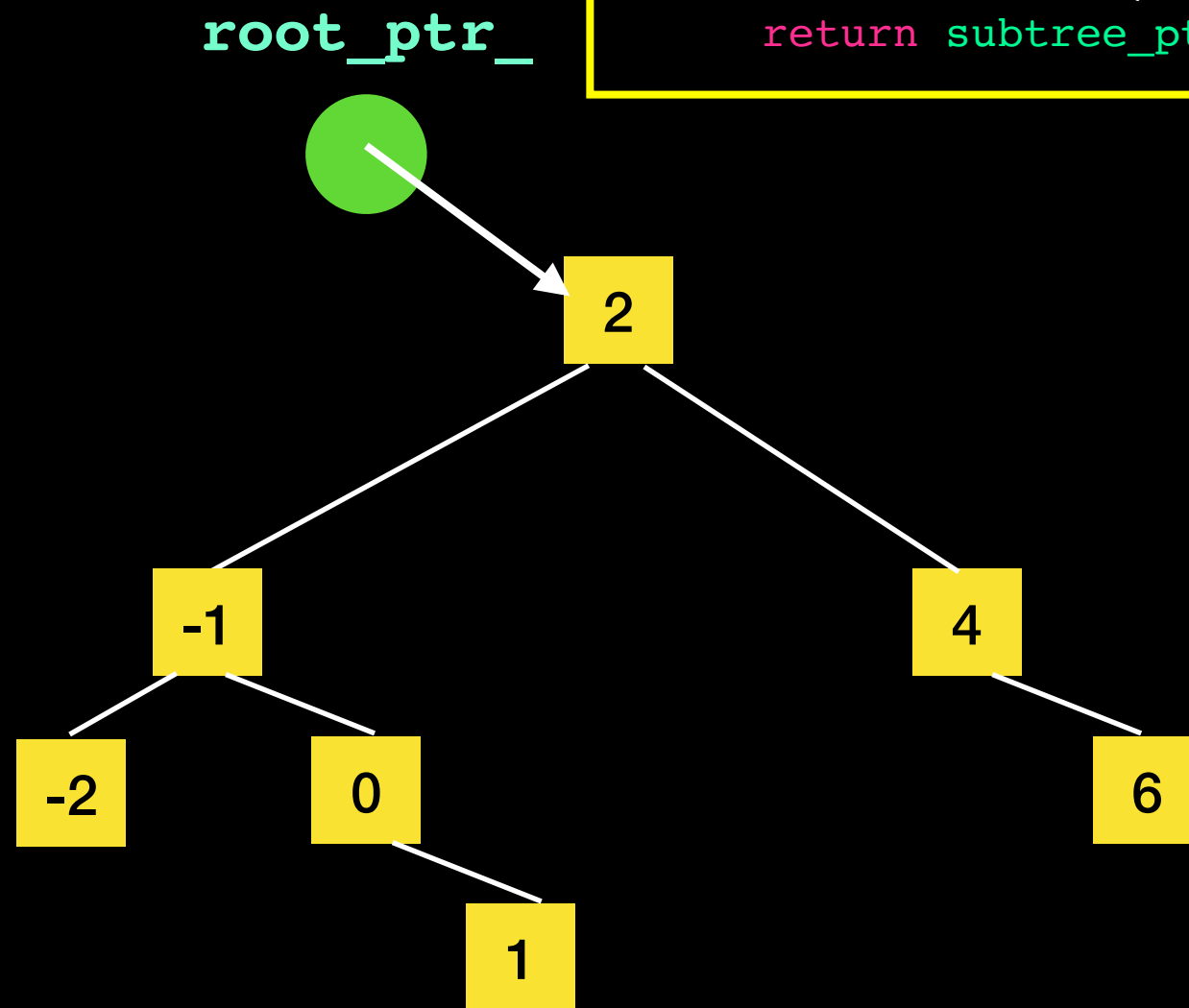
Case 1: target is a leaf



removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

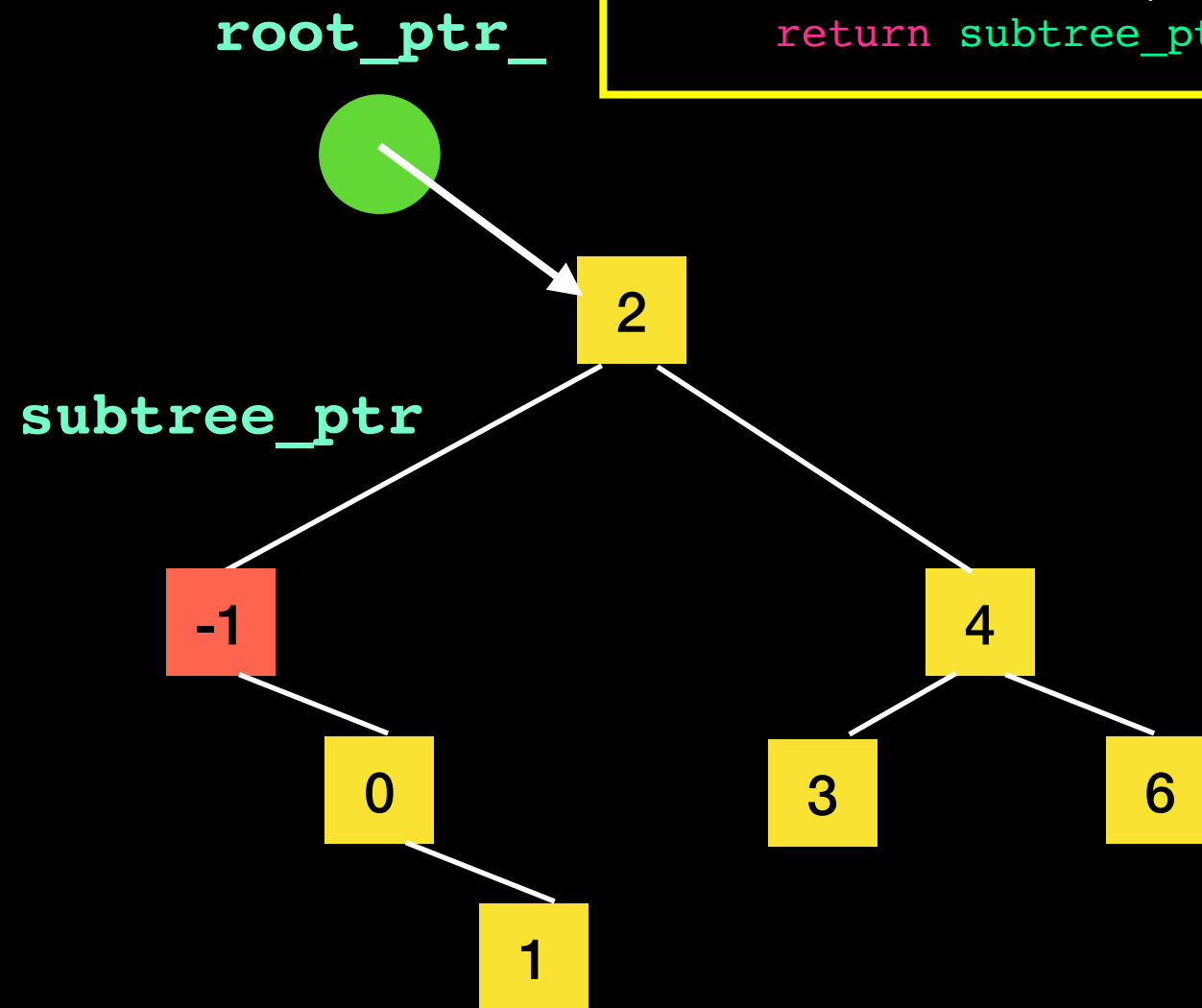
Case 1: target is a leaf



removeNode(subtree_ptr);

Case 2: target has 1 child
Left and right case are symmetric

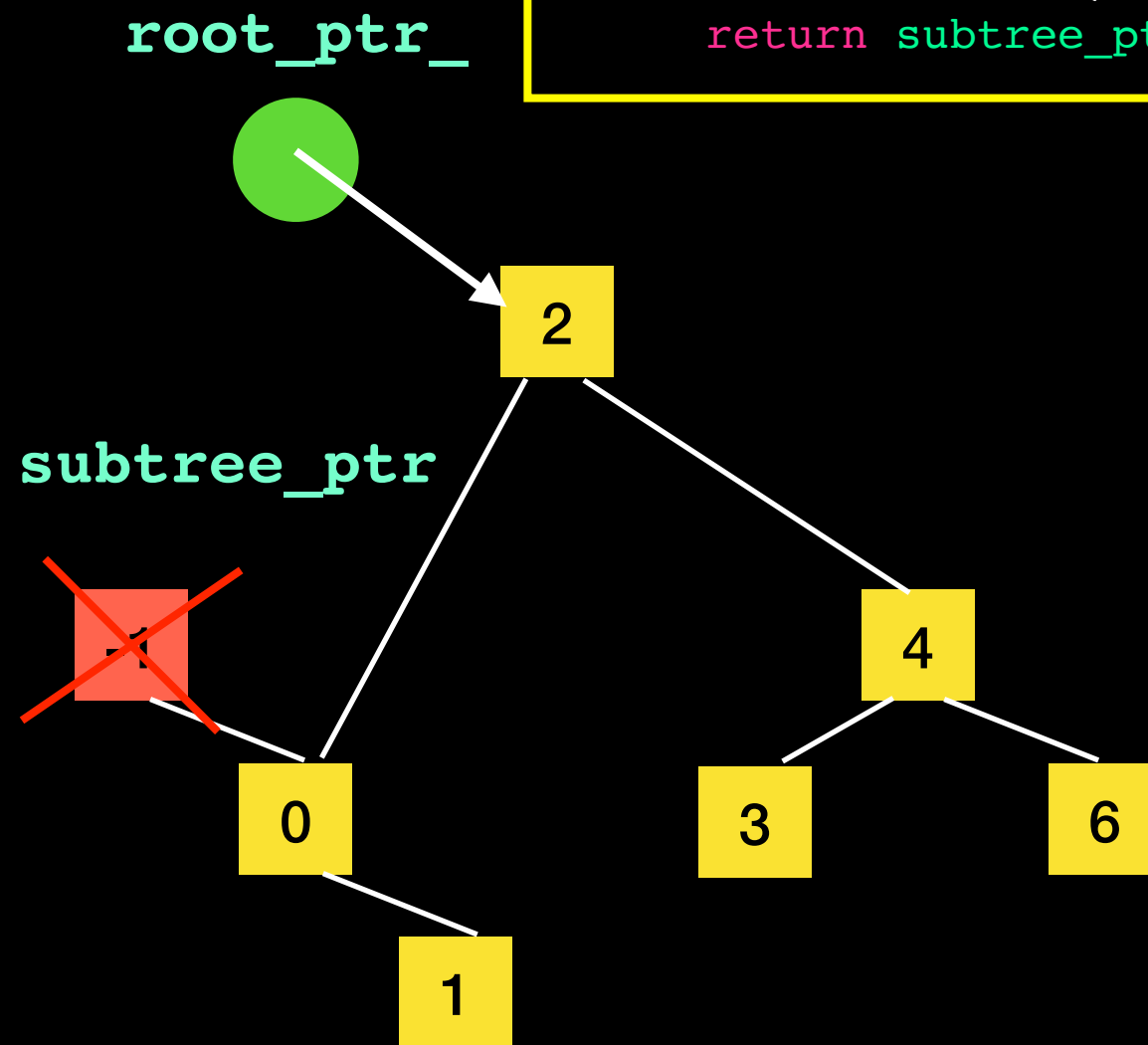
```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```



removeNode(subtree_ptr);

Case 2: target has 1 child
Left and right case are symmetric

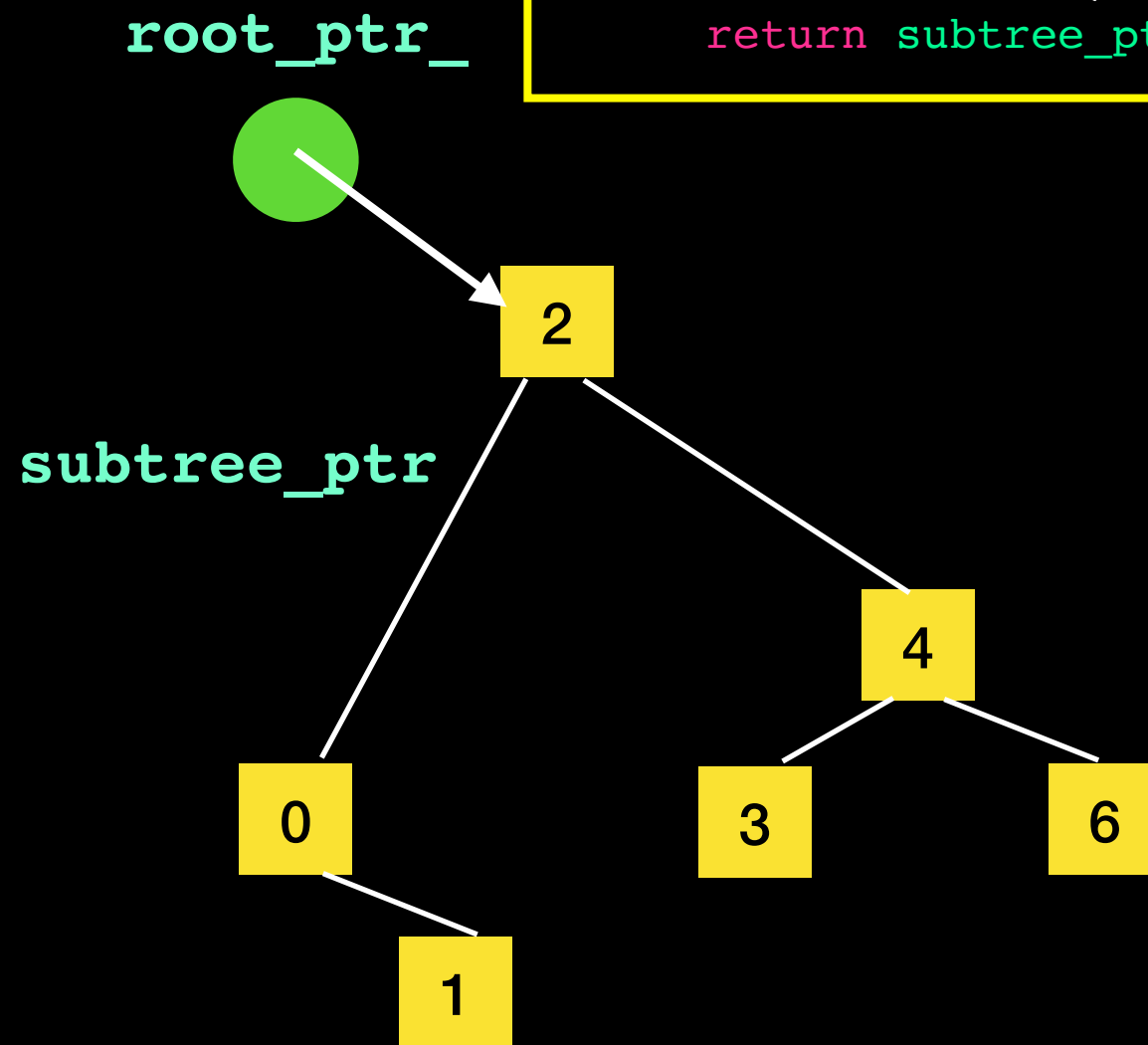
```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```



removeNode(subtree_ptr);

Case 2: target has 1 child
Left and right case are symmetric

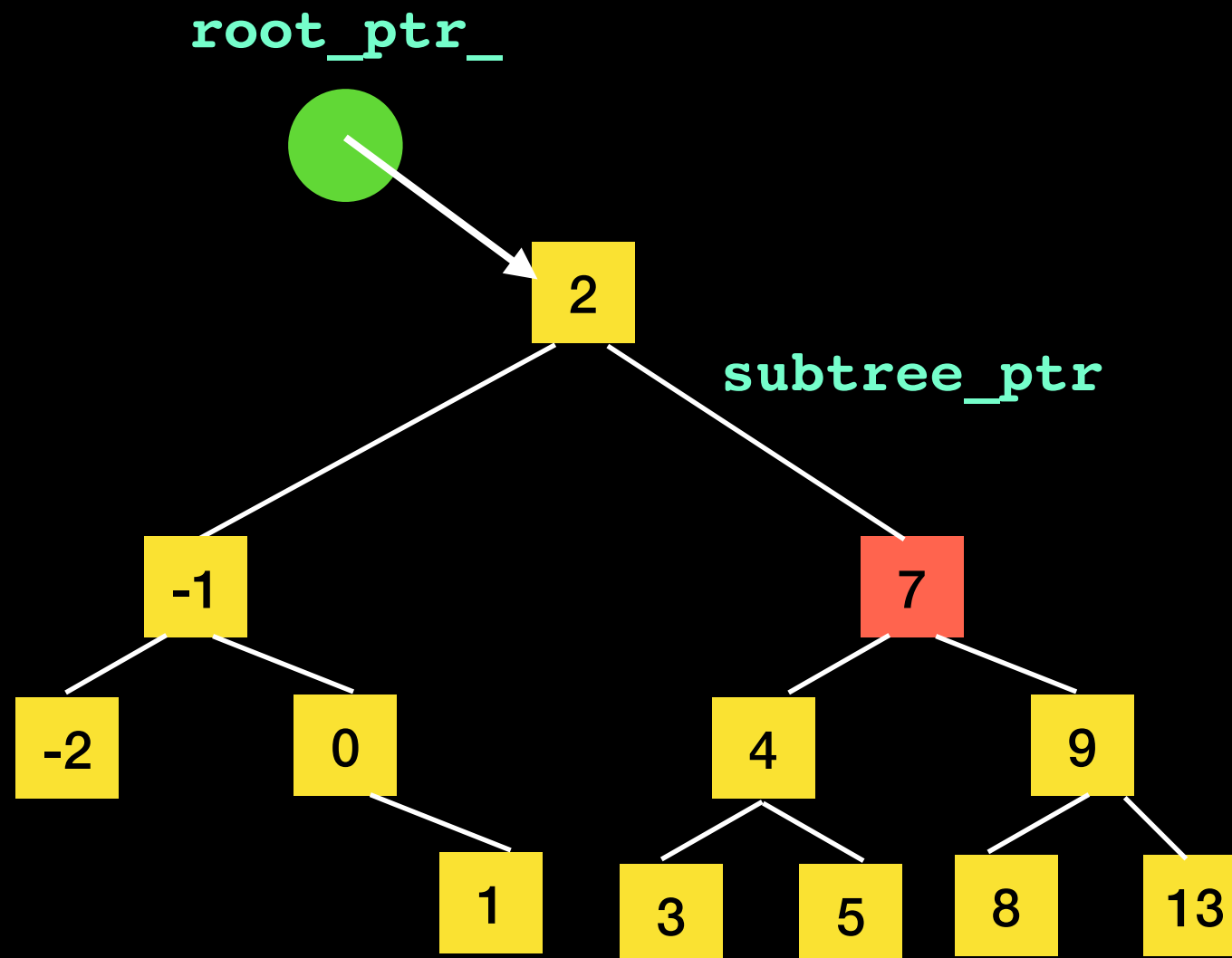
```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```



Lecture Activity

How would you remove node 7?

Case 3: target has 2 children



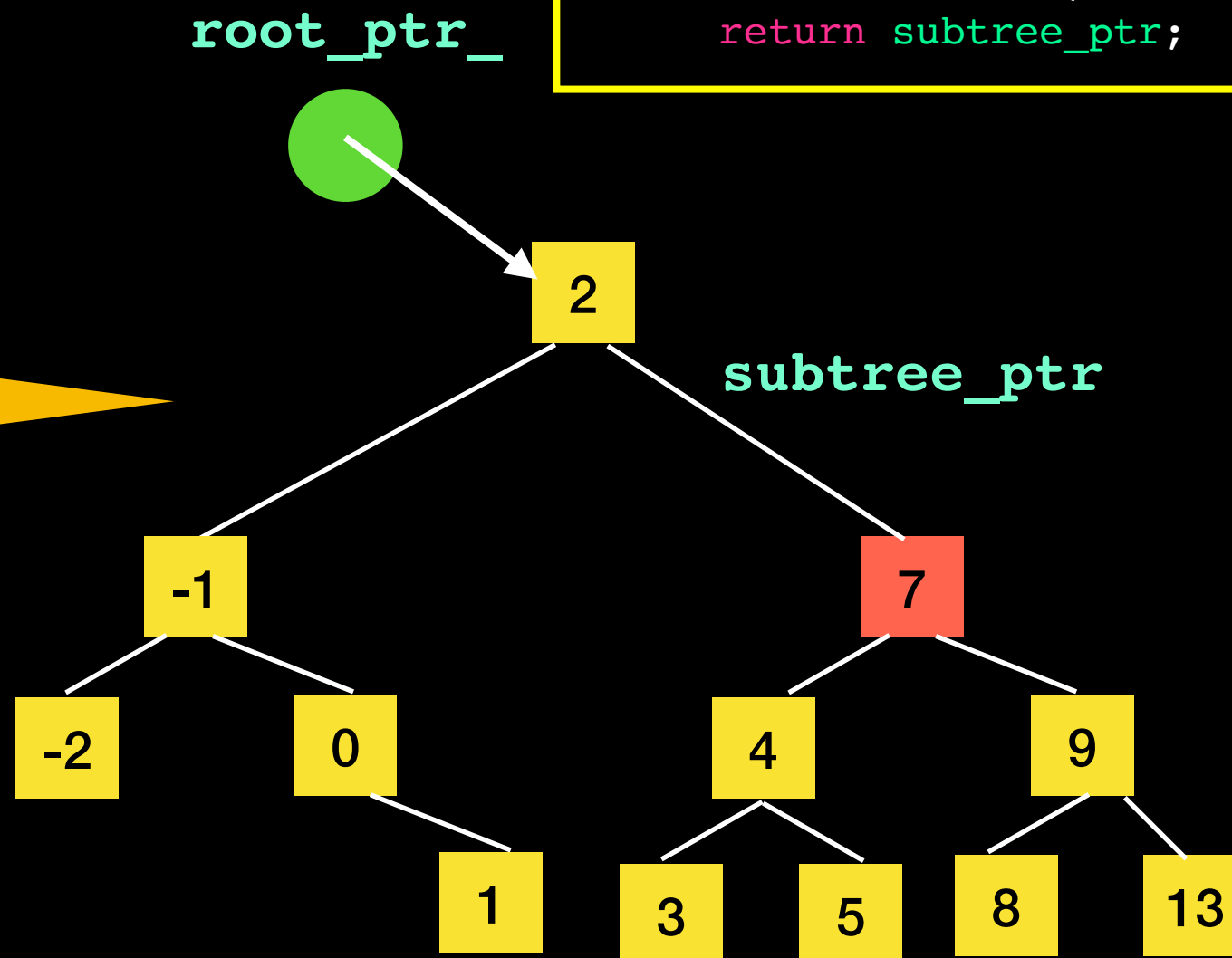
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



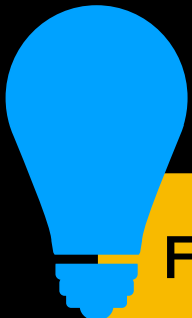
Find a node that is easy to remove and remove that one instead.



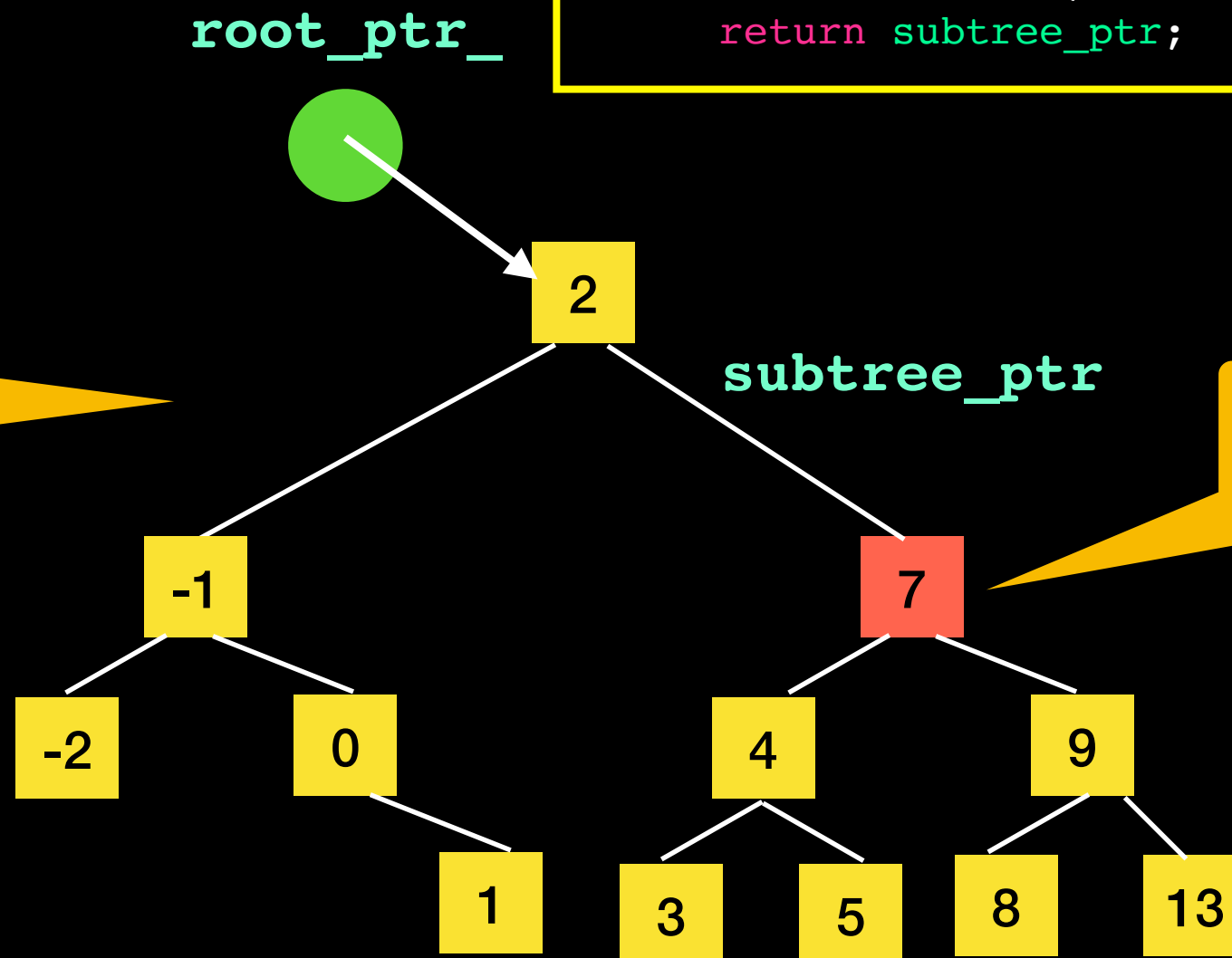
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



Find a node that is easy to remove and remove that one instead.



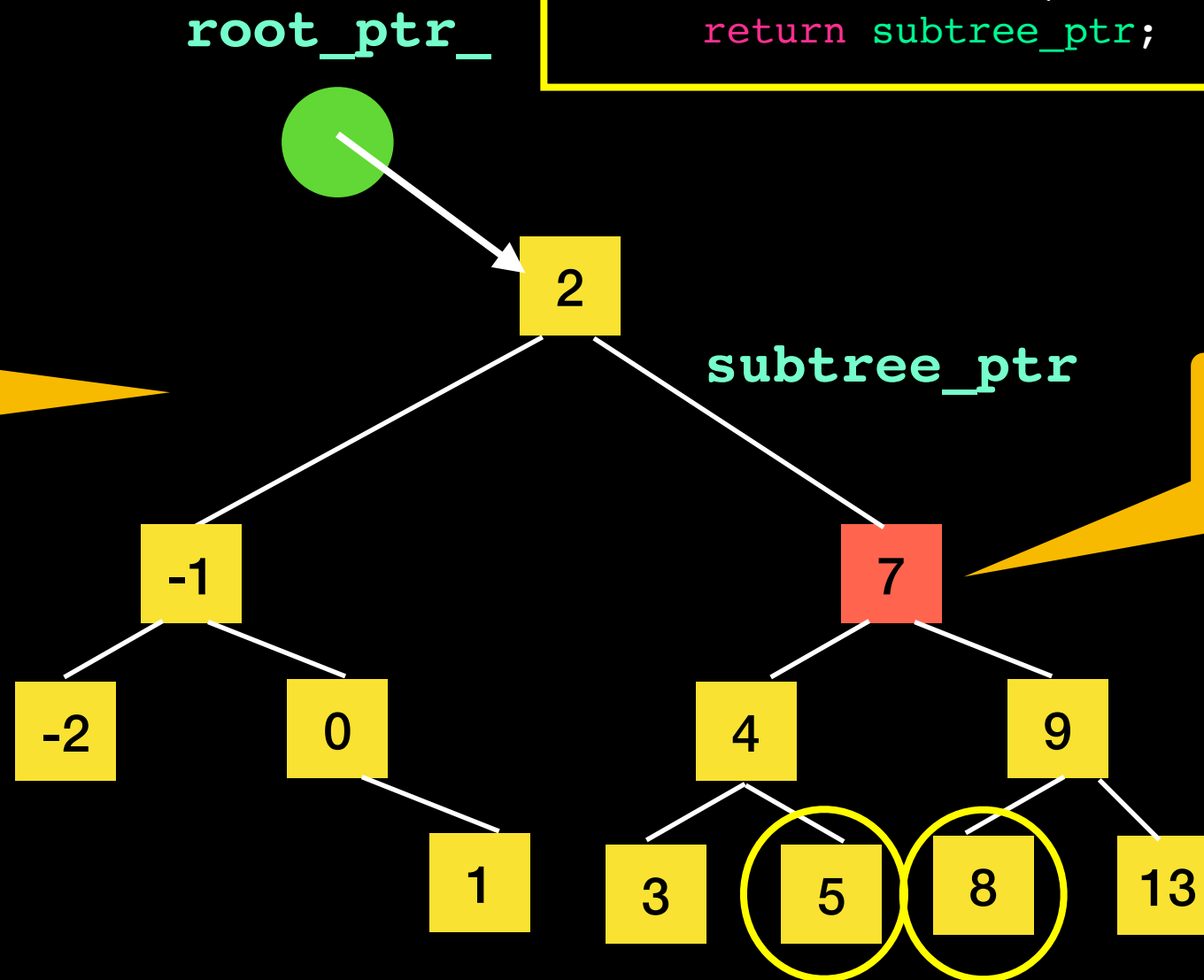
What value should we put here?

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.

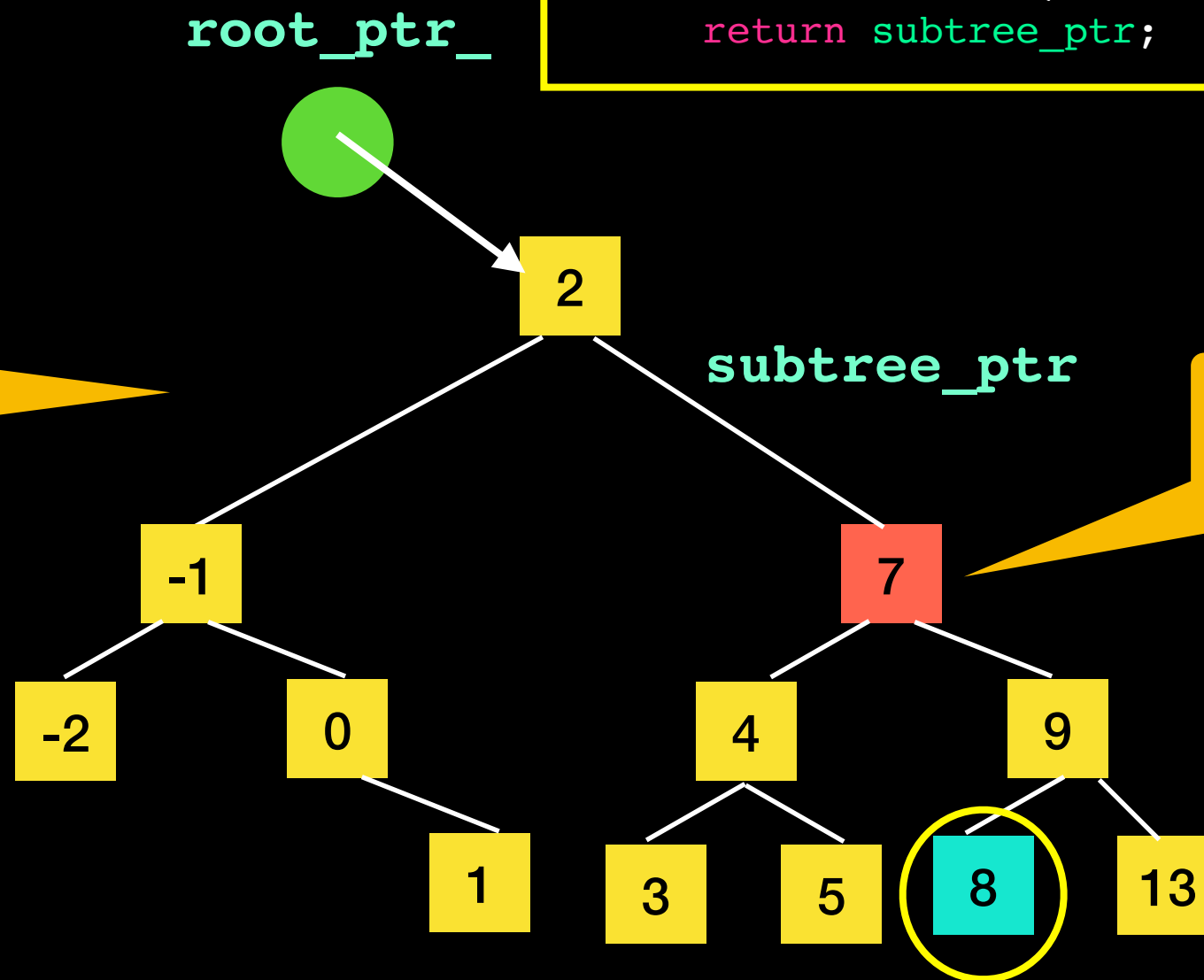


removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.



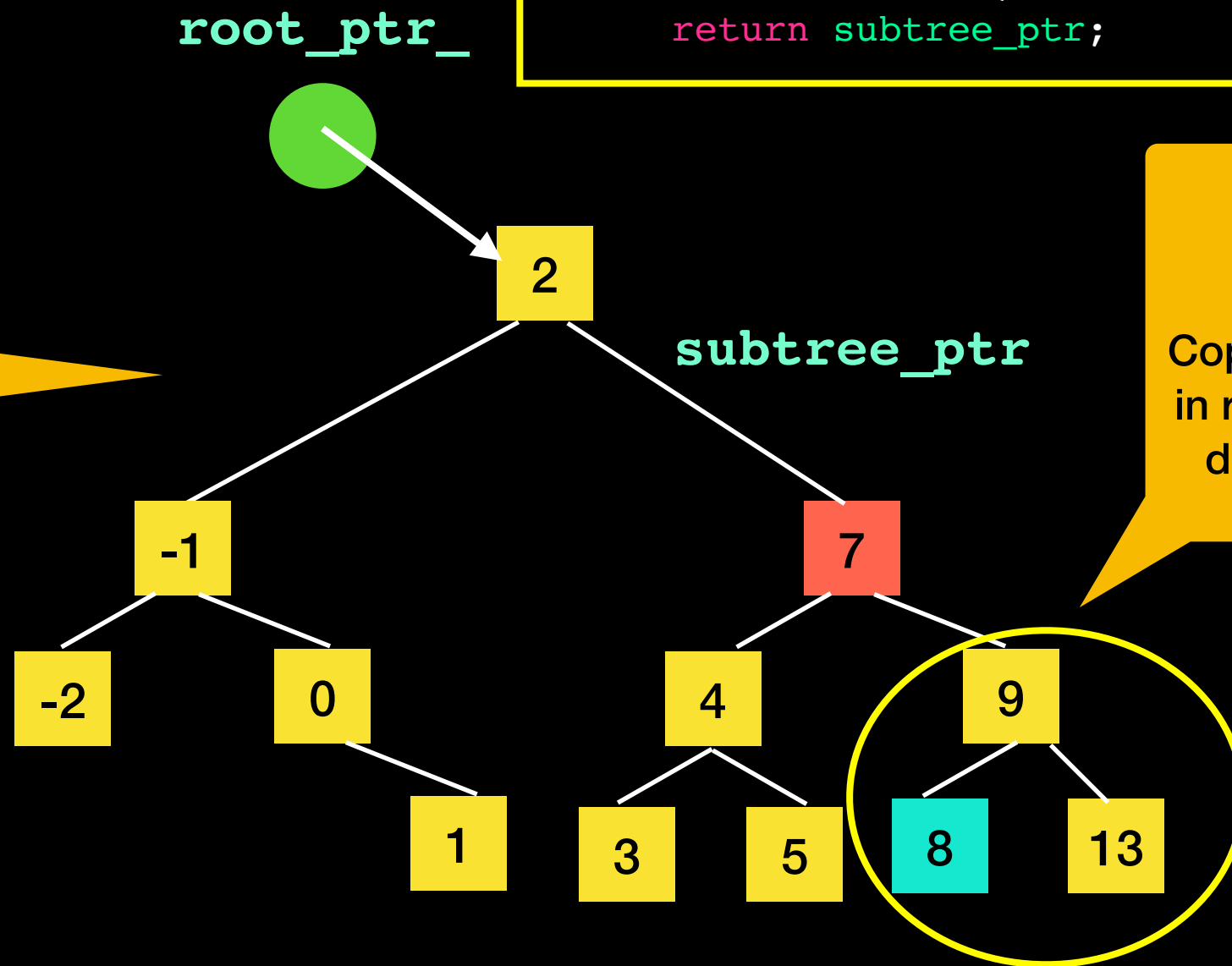
The *inorder successor*

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.



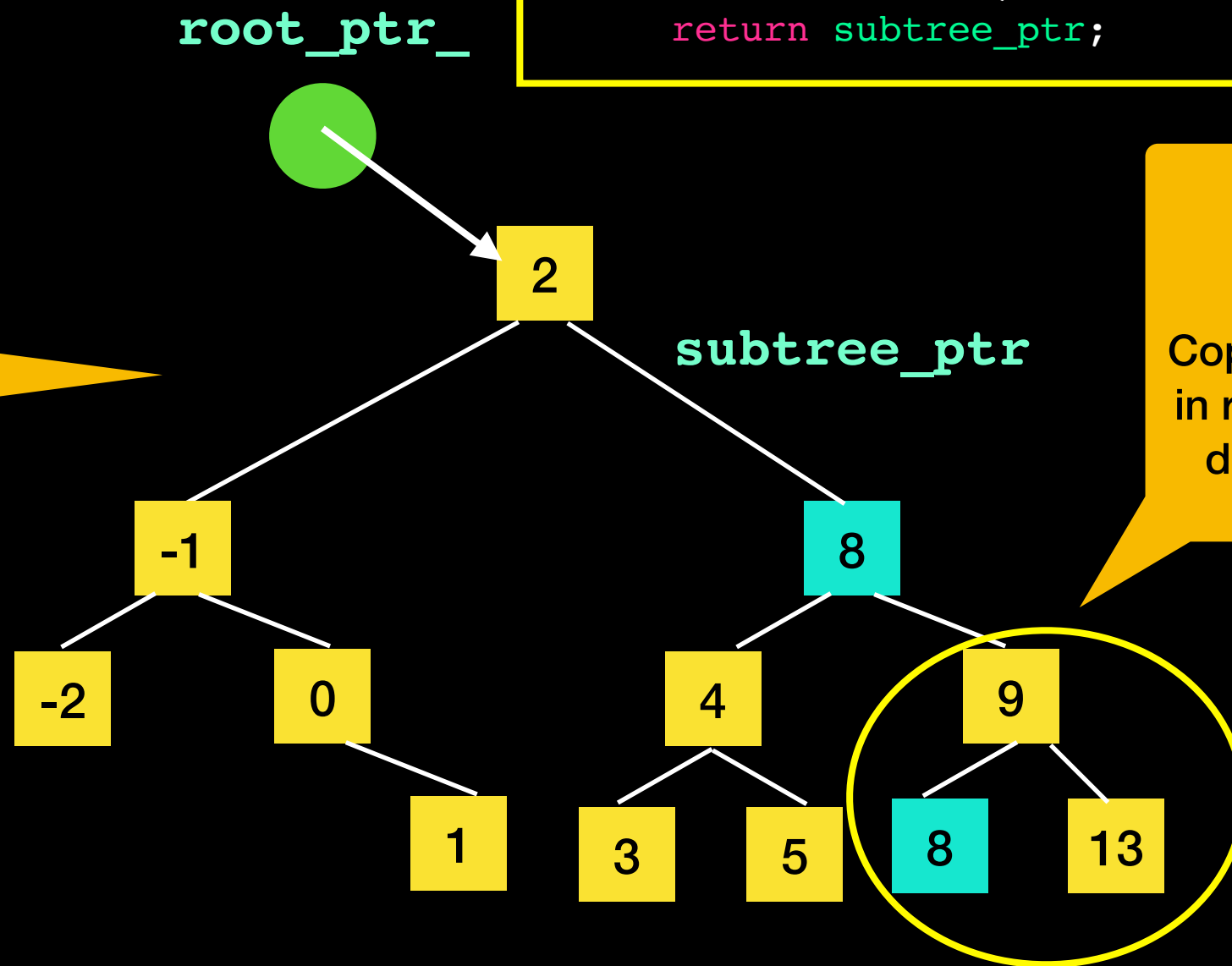
The *inorder* successor:
Copy smallest value in right subtree and delete that node

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.



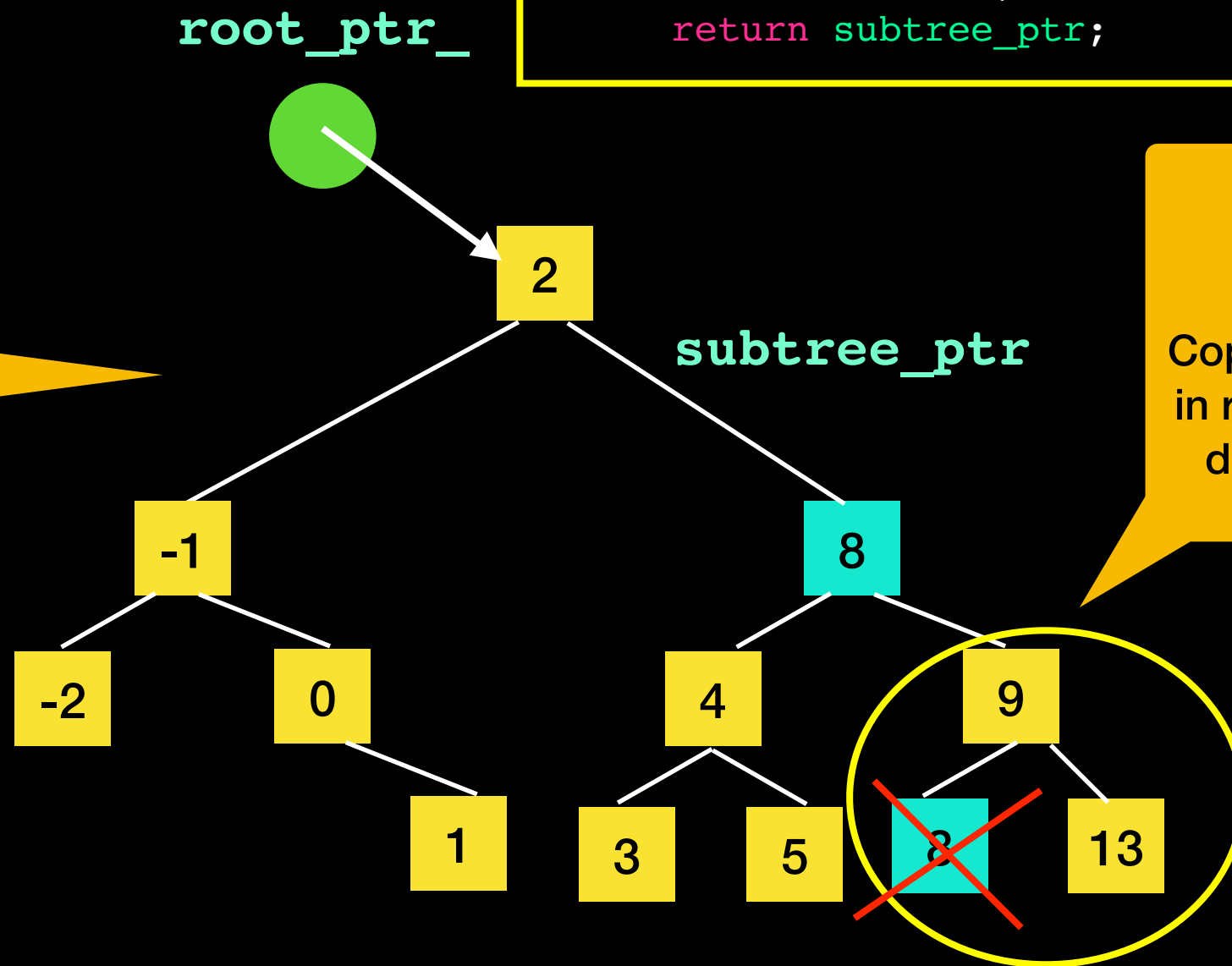
The *inorder* successor:
Copy smallest value in right subtree and delete that node

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.

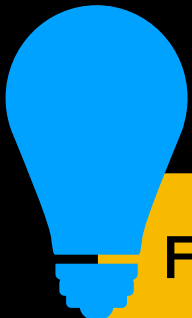


The *inorder* successor:
Copy smallest value in right subtree and delete that node

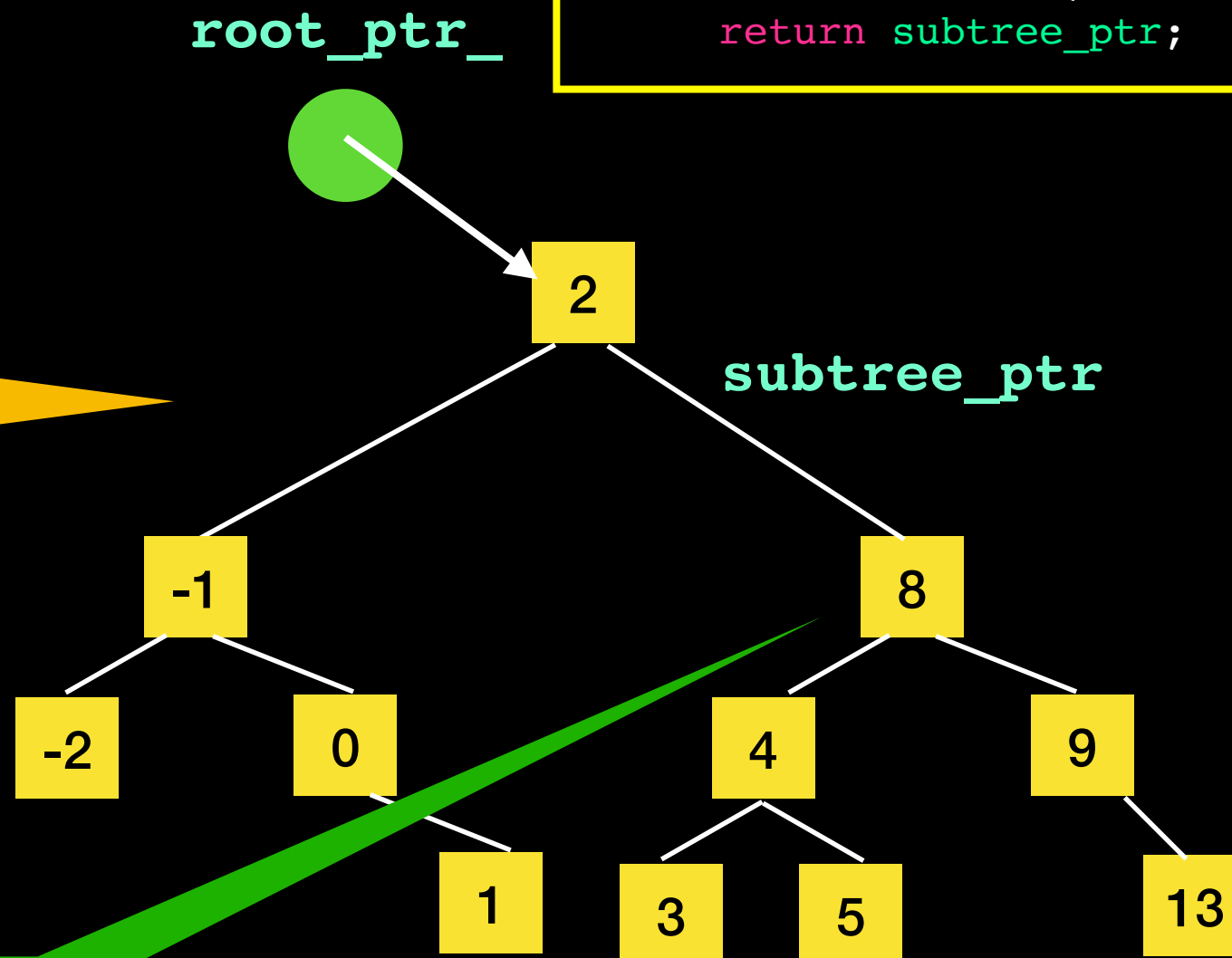
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



Find a node that is easy to remove and remove that one instead.

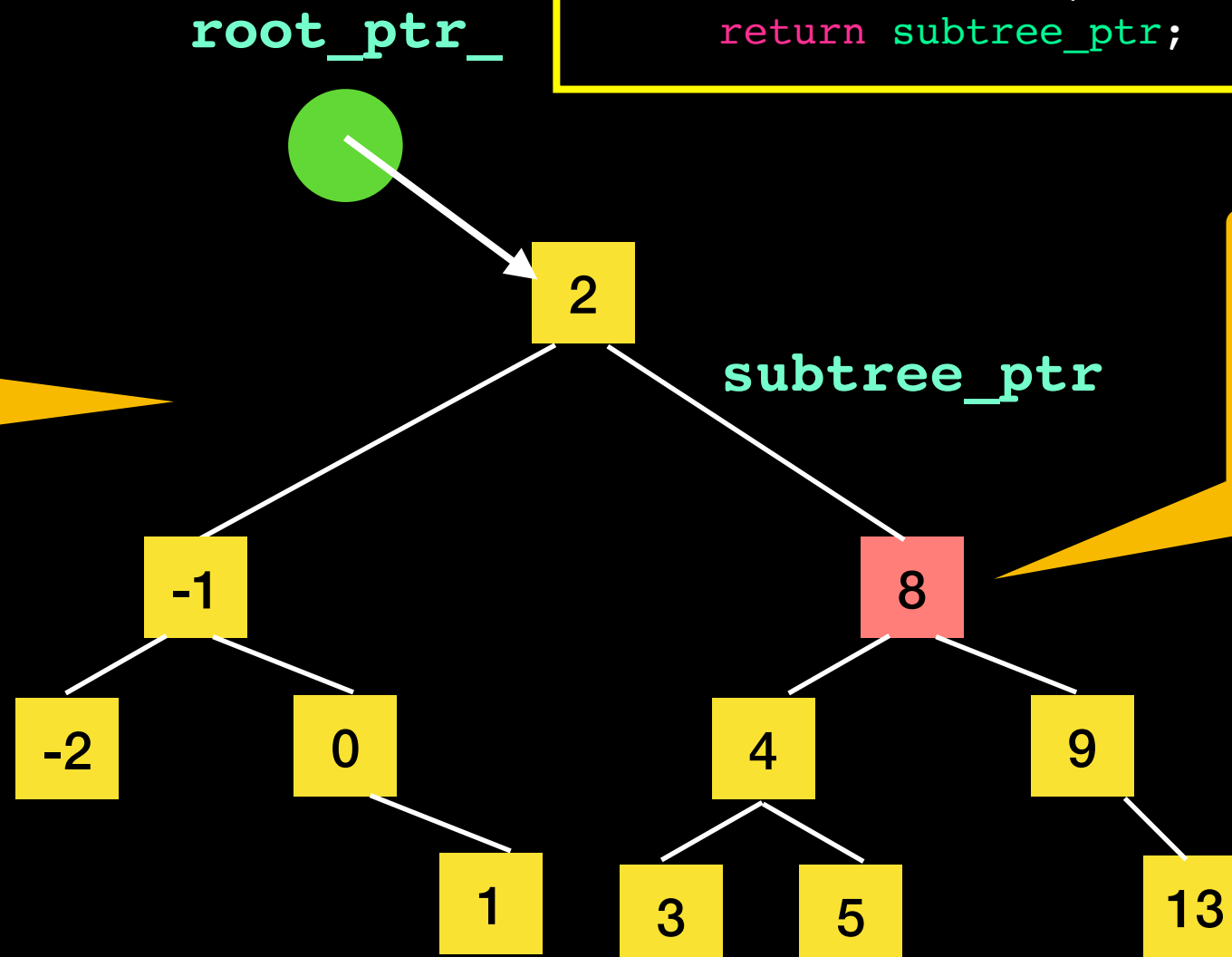


removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.



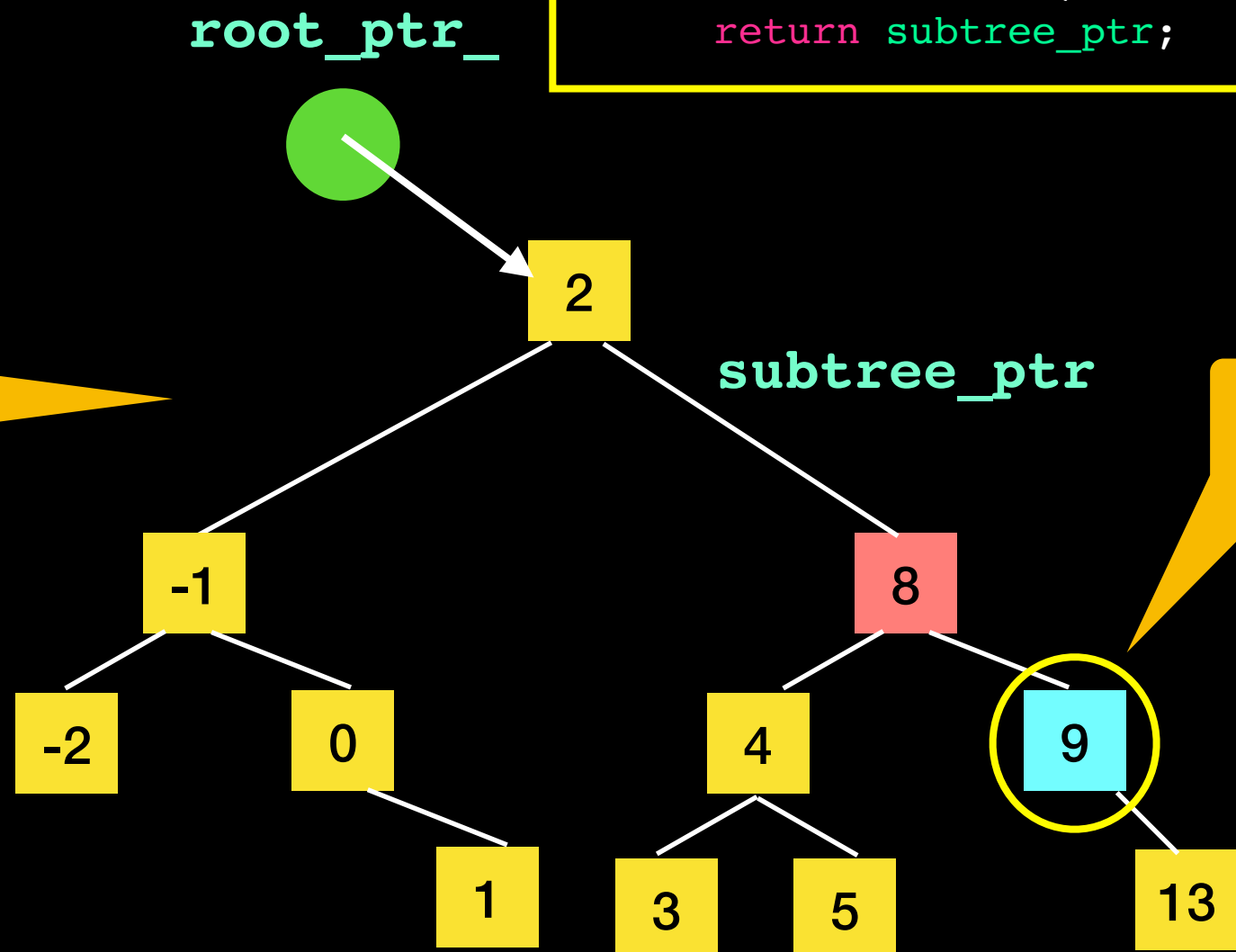
What about removing 8 now?
What value should we put here?

removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children

Find a node that is easy to remove and remove that one instead.



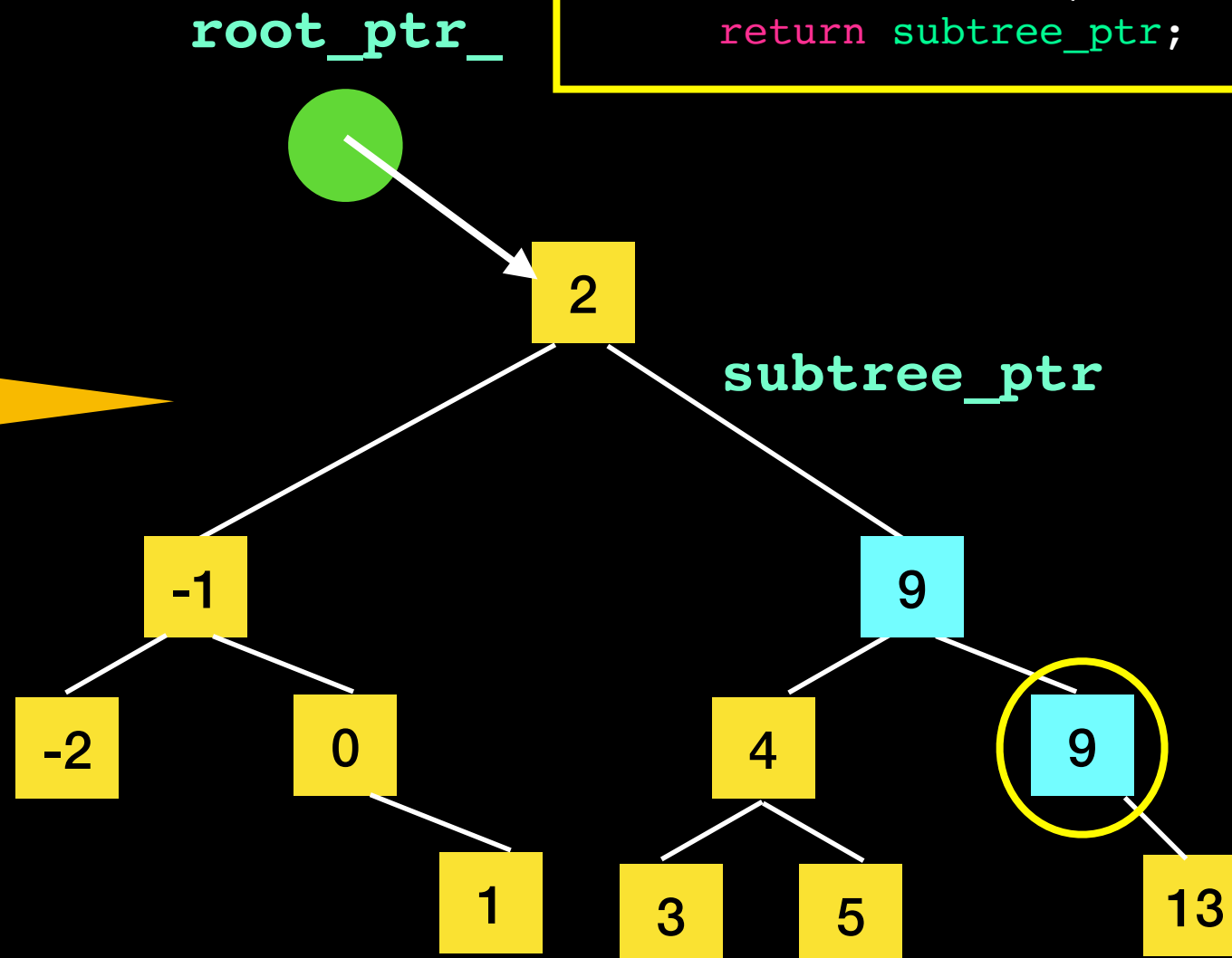
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



Find a node that is easy to remove and remove that one instead.



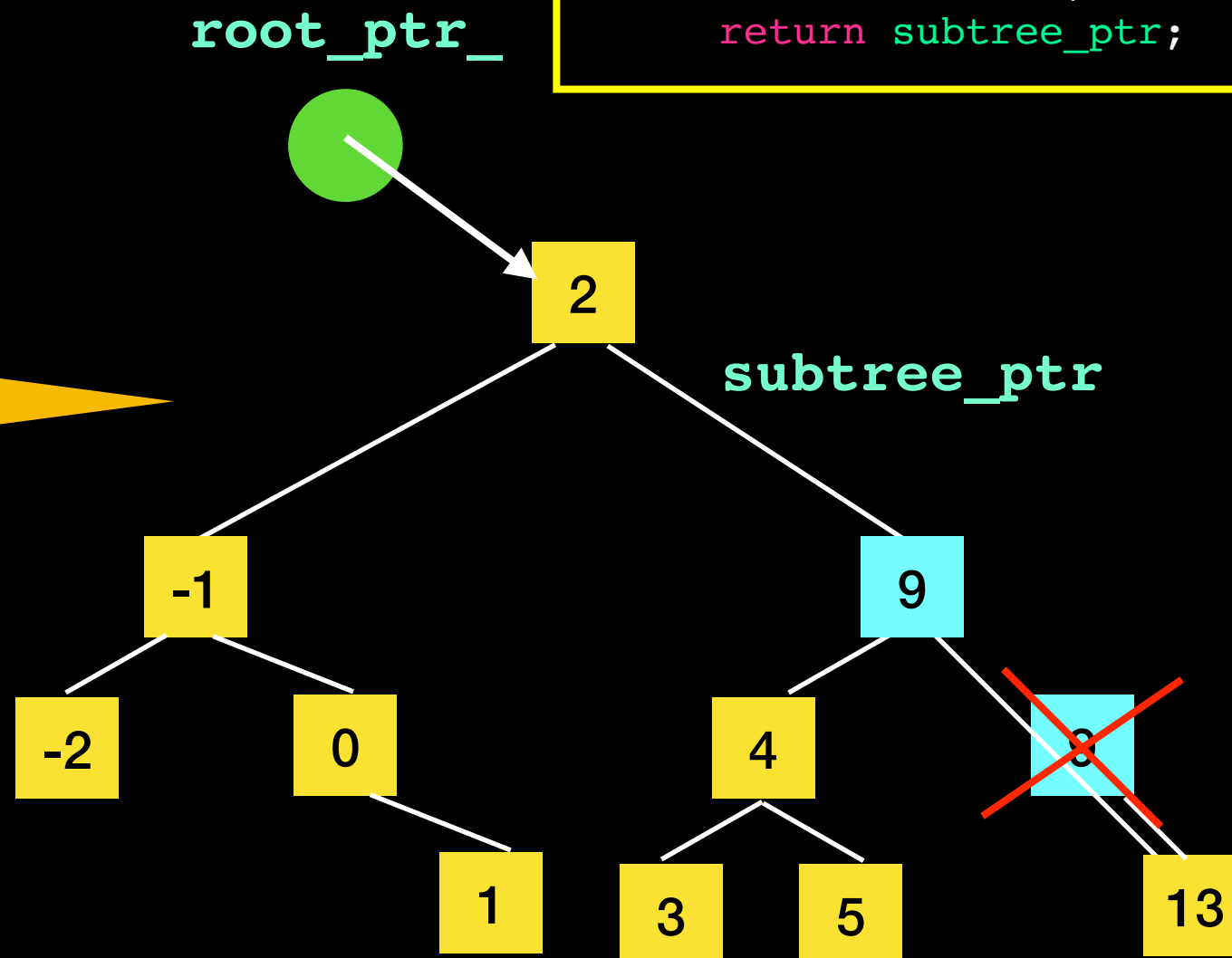
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



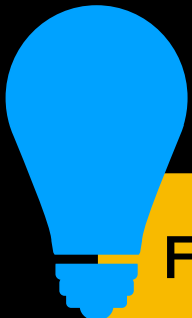
Find a node that is easy to remove and remove that one instead.



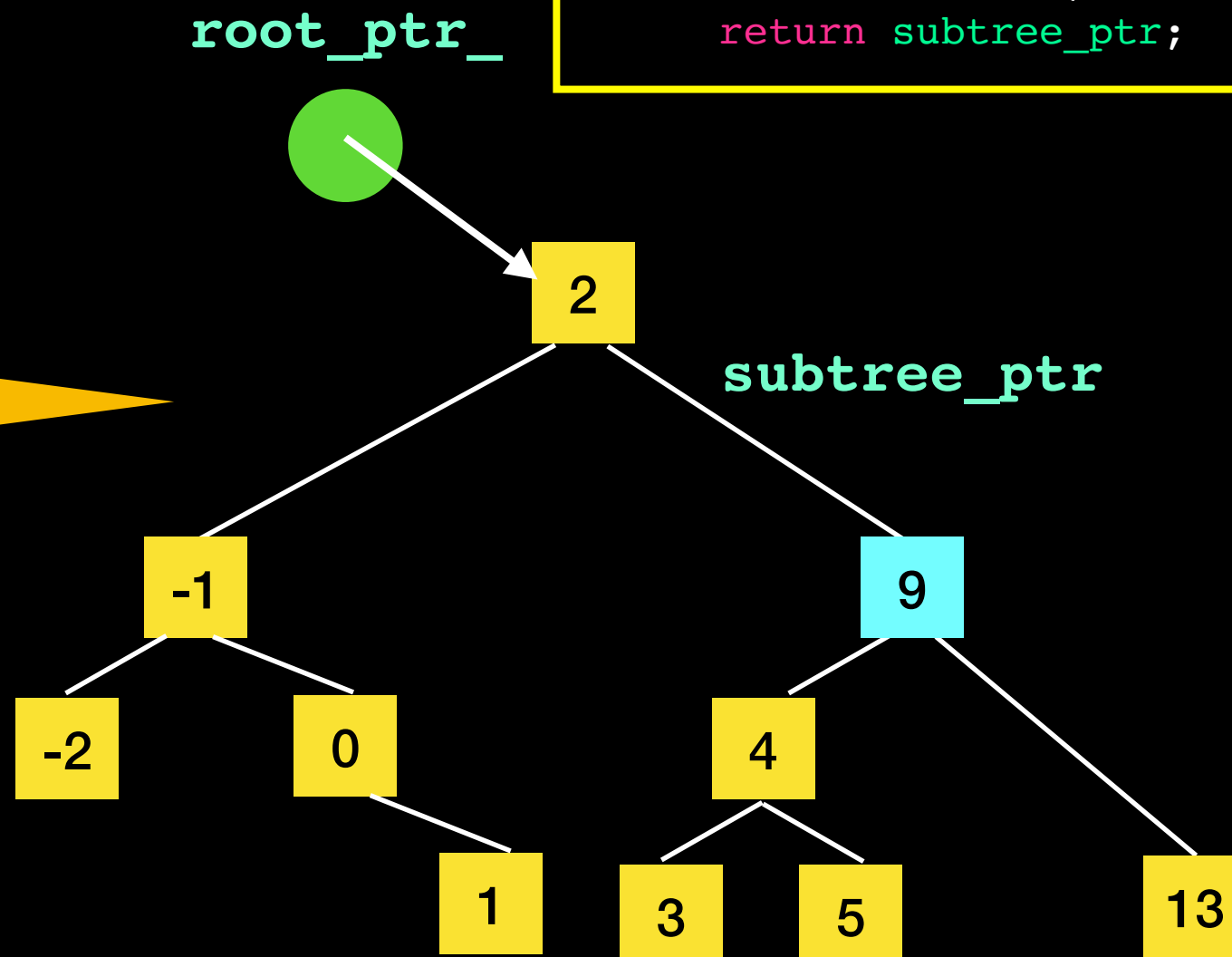
removeNode(subtree_ptr);

```
if (subtree_ptr->getItem() == target)
{
    // Item is in the root of this subtree
    subtree_ptr = removeNode(subtree_ptr);
    success = true;
    return subtree_ptr;
}
```

Case 3: target has 2 children



Find a node that is easy to remove and remove that one instead.



removeNode(node_ptr);

```
template<class T>
auto BST<T>::removeNode(std::shared_ptr<BinaryNode<T>> node_ptr)
{
    // Case 1) Node is a leaf - it is deleted
    if (node_ptr->isLeaf())
    {
        node_ptr.reset();
        return node_ptr; // delete and return nullptr
    }
    // Case 2) Node has one child - parent adopts child
    else if (node_ptr->getLeftChildPtr() == nullptr) // Has rightChild only
    {
        return node_ptr->getRightChildPtr();
    }
    else if (node_ptr->getRightChildPtr() == nullptr) // Has left child only
    {
        return node_ptr->getLeftChildPtr();
    }
    // Case 3) Node has two children:
    else
    {
        T new_node_value;
        node_ptr->setRightChildPtr(removeLeftmostNode(node_ptr->getRightChildPtr(),
                                                    new_node_value));

        node_ptr->setItem(new_node_value);
        return node_ptr;
    }
    // end if
} // end removeNode
```

Node is leaf

Node has 1 child

Node has 2 children

Will find leftmost leaf in right subtree, save value in new_node_value and delete

Safe Programming:
reference parameter is local to the private calling function

removeLeftmostNode

```
template<class T>
auto BST<T>::removeLeftmostNode(std::shared_ptr<BinaryNode<T>>
                                nodePtr, T& inorderSuccessor)
{
    if (nodePtr->getLeftChildPtr() == nullptr)
    {
        inorderSuccessor = nodePtr->getItem();
        return removeNode(nodePtr);
    }
    else
    {
        nodePtr->setLeftChildPtr(removeLeftmostNode(nodePtr->getLeftChildPtr(),
                                                    inorderSuccessor));
        return nodePtr;
    } // end if
} // end removeLeftmostNode
```


Traversals

Let's focus on the traversal for now, we will find out what Visitor does next

```
template<class T>
void BST<T>::preorderTraverse(Visitor<T>& visit) const
{
    preorder(visit, root_ptr_);
} // end preorderTraverse
```

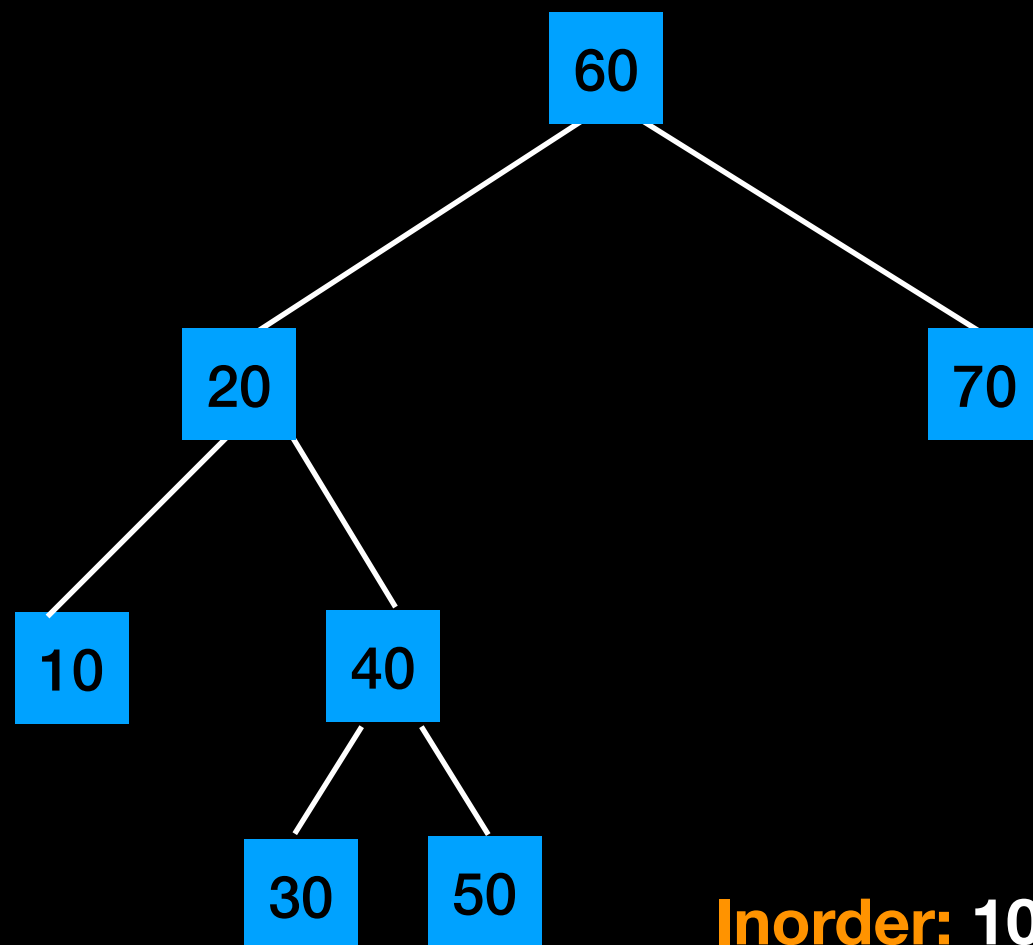
```
template<class T>
void BST<T>::inorderTraverse(Visitor<T>& visit) const
{
    inorder(visit, root_ptr_);
} // end inorderTraverse
```

```
template<class T>
void BST<T>::postorderTraverse(Visitor<T>& visit) const
{
    postorder(visit, root_ptr_);
} // end postorderTraverse
```

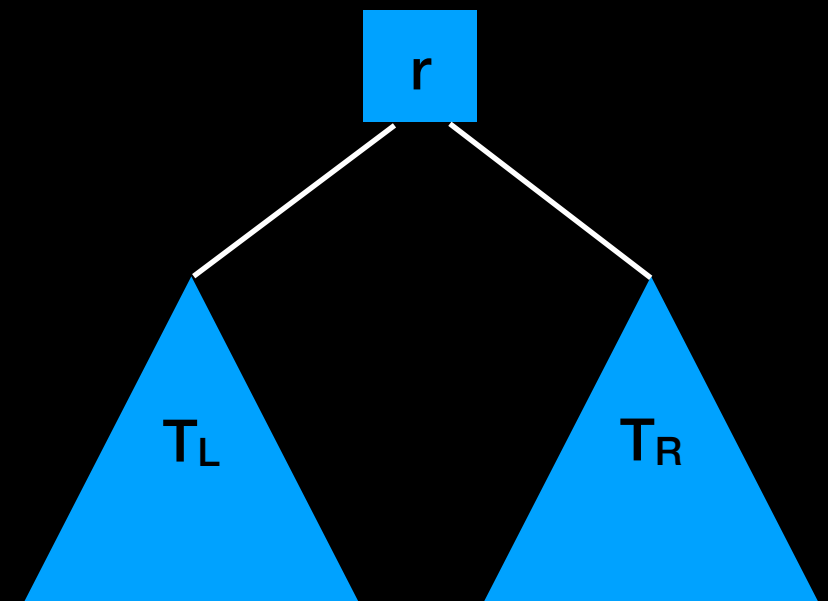

Visit (retrieve, print, modify ...) every node in the tree

Inorder Traversal:

```
if (T is not empty) //implicit base case
{
    traverse TL
    visit the root r
    traverse TR
}
```



Inorder: 10, 20, 30, 40, 50, 60, 70



inorderTraverse Helper Function

```
template<class T>
void BST<T>::inorder(Visitor<T>& visit,
    std::shared_ptr<BinaryNode<T>> tree_ptr) const
{
    if (tree_ptr != nullptr)
    {
        → inorder(visit, tree_ptr->getLeftChildPtr());
        T the_item = tree_ptr->getItem();
        visit(the_item);
        → inorder(visit, tree_ptr->getRightChildPtr());
    } // end if
} // end inorder
```

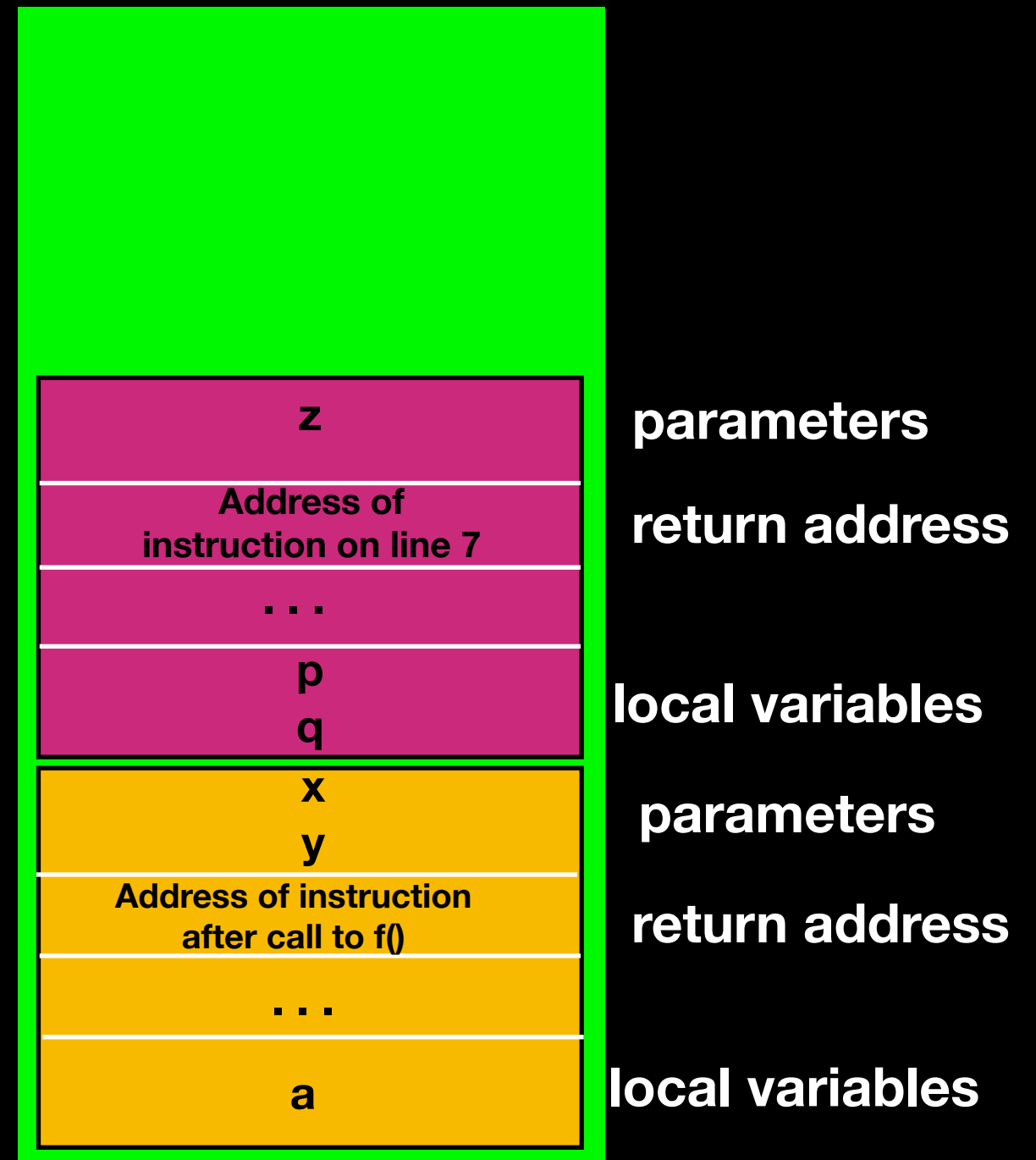
Recall: Program Stack

```
1 void f(int x, int y)
2 {
3     int a;
4     // stuff here
5     if(a<13)
6         a = g(a);
7     // stuff here
8 }
```

**Stack Frame
for g()**

```
9 int g(int z)
10 {
11     int p ,q;
12     // stuff here
13     return q;
14 }
```

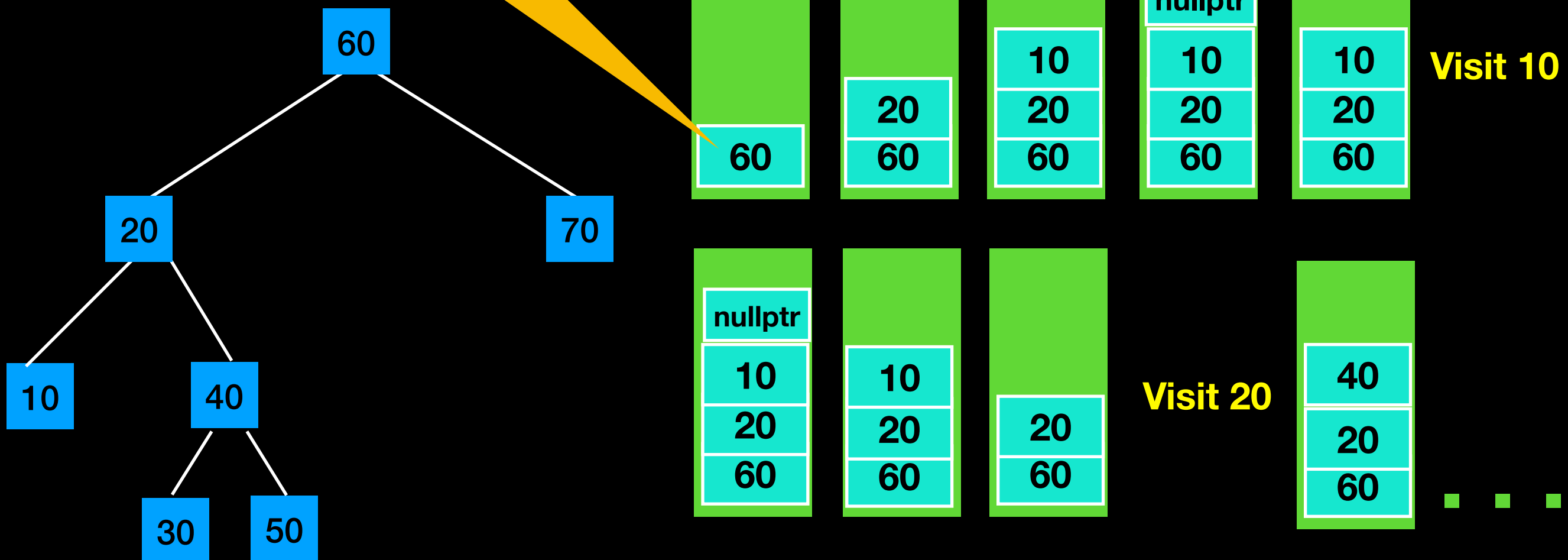
**Stack Frame
for f()**



Recursive Traversal

In recursive solution program stack keeps track of what node must be visited next

Means function call with
tree_ptr pointing to node with
data item 60



Recursive Traversal

With recursion:

- program stack **implicitly** finds node traversal must visit next
- If traversal backs up to node d from right subtree it backs up further to d 's parent as a **consequence of the recursive program execution**

Non-recursive Traversal

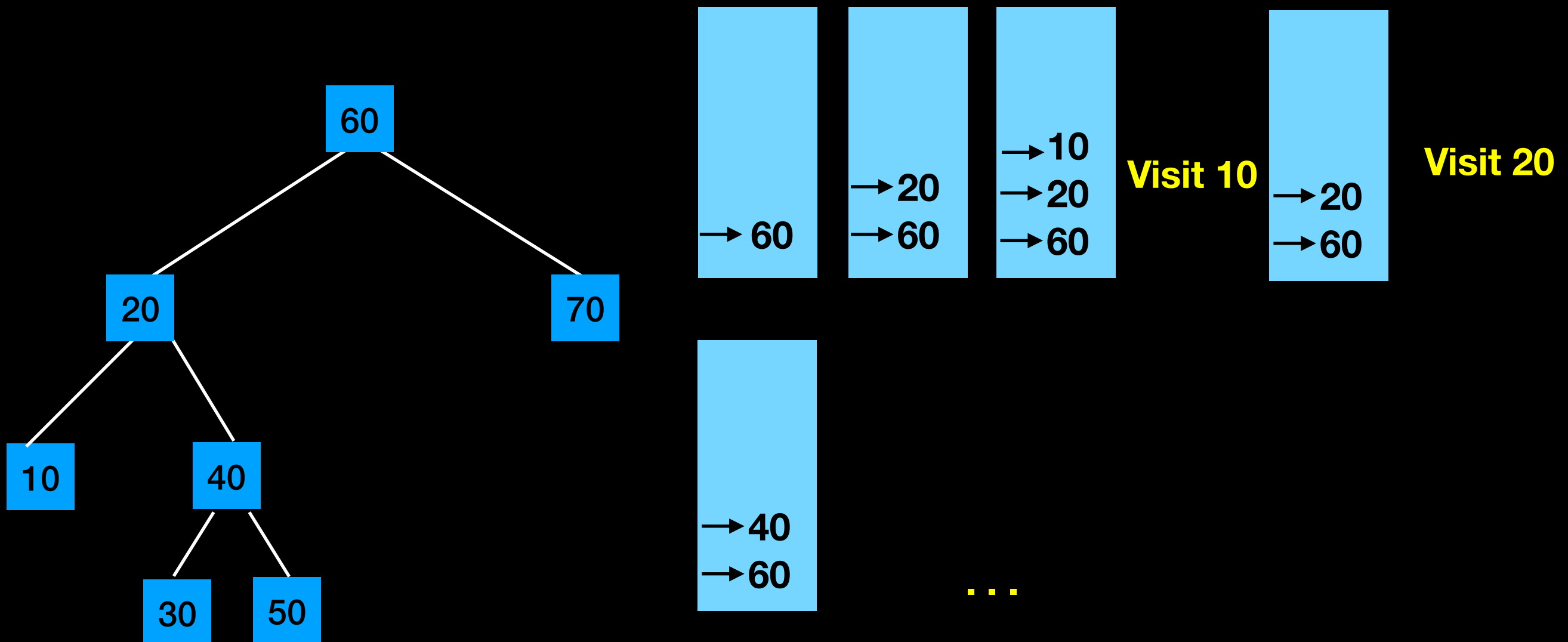
Optimize: Implement iterative approach that maintains an **explicit stack** to keep track of nodes that must be visited

Place pointer to node on stack **only** before traversing it's left subtree but **NOT** before traversing right subtree

This will also save some "steps" that were unnecessary but implicit in recursive implementation

Non-recursive Traversal

Iterative solution explicitly maintains a stack of pointers to nodes to keep track of what node must be visited next



Non-recursive Traversal

```
template<class T>
void BST<T>::inorder(Visitor<T>& visit) const
{
    std::stack<T> node_stack;
    std::shared_ptr<BinaryNode<T>> current_ptr = root_ptr_;
    bool done = false;

    while(!done)
    {
        if(current_ptr != nullptr)
        {
            node_stack.push(current_ptr);

            //traverse left subtree
            current_ptr = current_ptr->getLeftChildPtr();
        }
    }
}
```


Non-recursive Traversal cont.

```
//backtrack from empt subtree and visit the node at top of
//stack, but if stack is empty traversal is completed
else{
    done = node_stack.isEmpty();
    if(!done)
    {
        current_ptr = node_stack.top();
        visit(current_ptr->getItem());
        node_stack.pop();

        //traverse right subtree of node just visited
        current_ptr = current_ptr->getRightChildPtr();
    }
}
}
} // end inorder
```

Traversals

The last cool trick for
you this semester

Looking for
different
behavior

```
template<class T>
void BST<T>::preorderTraverse(Visitor<T>& visit) const
{
    preorder(visit, root_ptr_);
} // end preorderTraverse
```

```
template<class T>
void BST<T>::inorderTraverse(Visitor<T>& visit) const
{
    inorder(visit, root_ptr_);
} // end inorderTraverse
```

```
template<class T>
void BST<T>::postorderTraverse(Visitor<T>& visit) const
{
    postorder(visit, root_ptr_);
} // end postorderTraverse
```

Functors

Objects that by overloading `operator()` can be “called” like a function

POLYMORPHISM!
ABSTRACT CLASS!!!

```
#ifndef Visitor_hpp
#define Visitor_hpp
#include <string>

template<class T>
class Visitor
{
public:

    virtual void operator()(T&) = 0;
    virtual void operator()(T&, T&) = 0;

};

#endif /* Visitor_hpp */
```

```
#ifndef StringPrinter_hpp
#define StringPrinter_hpp

#include "Visitor.hpp"
#include <iostream>
#include <string>

class StringPrinter: public Visitor<std::string>
{
public:

    void operator()(std::string&) override;
    void operator()(std::string&, std::string&) override;

};

#endif /* StringPrinter_hpp */
```

```
#include "StringPrinter.hpp"
```

```
void StringPrinter::operator()(std::string& x)
{
    std::cout << x << std::endl;
}
```

```
void StringPrinter::operator()(std::string& a, std::string& b)
{
    std::cout << a << b << std::endl;
}
```

```
#ifndef Inverter_hpp
#define Inverter_hpp

#include "Visitor.hpp"
#include <iostream>
#include <string>
#include <algorithm>

class Inverter: public Visitor<std::string>
{
public:

    void operator()(std::string&) override;
    void operator()(std::string&, std::string&) override;

};

#endif /* Inverter_hpp */
```



```
#include "Inverter.hpp"
```

```
void Inverter::operator()(std::string& x)
{
    std::reverse(x.begin(), x.end());
    std::cout << x << std::endl;
}
```

```
void Inverter::operator()(std::string& a, std::string& b)
{
    a.swap(b);
    std::cout << a << b << std::endl;
}
```


Traversal with Functor parameter

```
template<class T>
void BST<T>::inorder(Visitor<T>& visit,
    std::shared_ptr<BinaryNode<T>> tree_ptr) const
{
    if (tree_ptr != nullptr)
    {
        inorder(visit, tree_ptr->getLeftChildPtr());
        T the_item = tree_ptr->getItem();
        visit(the_item);
        inorder(visit, tree_ptr->getRightChildPtr());
    } // end if
} // end inorder
```

```

int main() {

    std::string a_string = "a string";
    std::string anoter_string = "o string";

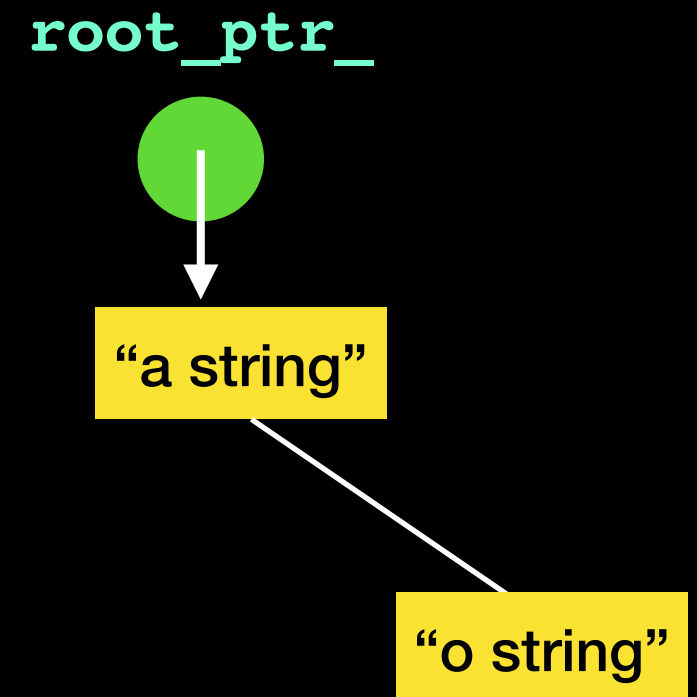
    BST<std::string> a_tree(a_string);
    a_tree.add(anoter_string);

    StringPrinter p;
    Inverter i;

    a_tree.inorderTraverse(p);
    std::cout << std::endl;
    a_tree.inorderTraverse(i);

    return 0;
}

```



```

a string
o string

gnirts a
gnirts o
Program ended with exit code: 0

```

```
int main() {
```

```
    std::string a_string = "a string";  
    std::string another_string = "o string";
```

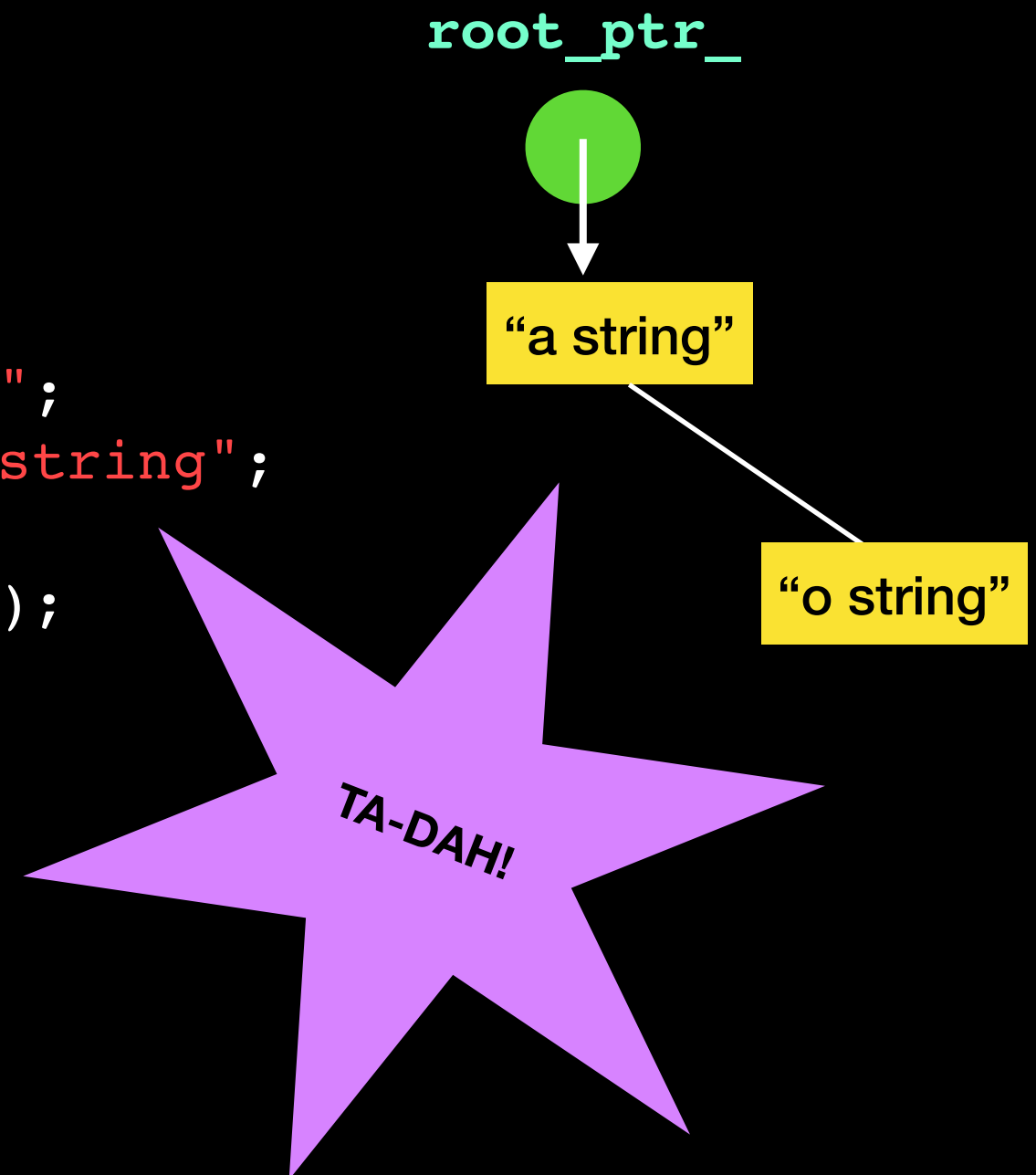
```
    BST<std::string> a_tree(a_string);  
    a_tree.add(another_string);
```

```
    StringPrinter p;  
    Inverter I;
```

```
    a_tree.inorderTraverse(p);  
    std::cout << std::endl;  
    a_tree.inorderTraverse(i);
```

```
    return 0;
```

```
}
```



```
a string  
o string  
  
gnirts a  
gnirts o  
Program ended with exit code: 0
```